Computer Basics & Organization

Basics

Machine, software, and program design

Computer Organization

- CPU - central processing unit
- Memory
- Input devices
- Output devices

CPU - Central Processing Unit

Where decisions are made, computations are performed, and input/output requests are delegated

Memory

Stores information to be processed by the CPU

Input Devices

Allows people to supply information to computers

Output Devices

Allows people to receive information from computers

Computer Organization Details

- CPU is the brain of the computer
- Arithmetic calculations are performed using the Arithmetic/Logical Unit (ALU)
- Control unit decodes and executes instructions
- Arithmetic operations are performed using binary number system

Control Unit

- The fetch/execute cycle is the steps the CPU takes to execute an instruction
- Performing the action specified by an instruction is known as executing the instruction
- The program counter (PC) holds the memory address of the next instruction

Input and Output Devices

- Accessories that allow computer to perform specific tasks
  - Receive information for processing

- Return the results of processing
- Store information

Common input and output devices

- Speakers, Mouse, Scanner
- Printer, Joystick, CD-ROM
- Keyboard, Microphone, DVD

Devices capable of both input and output

- Floppy drive, Hard drive, Magnetic tape units

Monitor

- Display device that operates like a television
- Also known as CRT (cathode ray tube)
- Controlled by an output device called a graphics card
- Displayable area measured in dots per inch (pixels)
- Standard resolution is 640 by 480
- Many cards support resolution of 1280 by 1024 or better

Application software

- Programs designed to perform specific tasks that are transparent to the user

System software

- Programs that support the execution and development of other programs
- Two major types:

1. Operating systems
2. Translation systems

Application Software Details

- Software that has made using computers indispensable and popular
- Common examples:

1. Word processors
2. Desktop publishing programs

Operating System & Translation System

Operating System

- Examples: Windows®, UNIX®, Mac OS X®
- Controls and manages computing resources
- Important services provided:

1. File system (directories, folders, files)
2. Commands for file manipulation (sort, delete, copy)
3. Ability to perform I/O on various devices
4. Management of running systems

Translation System

- Set of programs used to develop software
- Key component: translator
- Types of translators:

1. Compiler (converts from one language to another)
2. Linker (combines resources)
3. Examples: Microsoft Visual C++®, CBuilder®, g++, Code Warrior®
4. Performs compilation, linking, and other activities

Software Development Activities

- Editing
- Compiling
- Linking with precompiled files (object files, library modules)
- Loading and executing
- Viewing program behavior

Software Development Cycle

- Integrated Development Environments (IDEs) support the entire cycle
- Examples: MS Visual C++, Borland, Code Warrior
- Provides: Editor, Compiler, Linker, Loader, Debugger, Viewer

What is Next?

Programming Fundamentals ..... !!!!

MCQs

1. The CPU is often called the _____ of the computer.
   a) Heart
   b) Brain
   c) Memory
   d) Screen
   Ans. b) Brain
2. Which component performs arithmetic and logical operations?
   a) Control Unit
   b) ALU
   c) Memory
   d) Hard Drive
   Ans. b) ALU
3. Which device is NOT an input device?
   a) Mouse
   b) Printer
   c) Scanner
   d) Keyboard
   Ans. b) Printer
4. Which of these is both an input and output device?
   a) Monitor
   b) Speaker
   c) Hard Drive
   d) Keyboard
   Ans. c) Hard Drive
5. The program counter (PC) holds:
   a) Data to be processed
   b) Memory address of the next instruction
   c) Results of calculations

d) Operating system commands

Ans. b) Memory address of the next instruction

6. What does CRT stand for?

   a) Computer Remote Terminal

   b) Cathode Ray Tube

   c) Central Runtime Translator

   d) Control Register Table

   Ans. b) Cathode Ray Tube

7. Resolution is measured in:

   a) Bytes per second

   b) Dots per inch

   c) Hertz

   d) Volts

   Ans. b) Dots per inch

8. Which is an example of system software?

   a) Word Processor

   b) Operating System

   c) Spreadsheet

   d) Game

   Ans. b) Operating System

9. A compiler is a type of:

   a) Input device

   b) Translator

   c) Output device

   d) Operating system

   Ans. b) Translator

10. Which is NOT part of the software development cycle?

    a) Editing

    b) Compiling

    c) Printing

d) Linking

Ans. c) Printing

11. IDE stands for:

    a) Integrated Development Environment

    b) Input Device Evaluator

    c) Internal Data Exchange

    d) Instruction Decode Engine

    Ans. a) Integrated Development Environment

12. Which of these is an example of an IDE?

    a) Microsoft Word

    b) MS Visual C++

    c) Windows OS

    d) Google Chrome

    Ans. b) MS Visual C++

13. The fetch/execute cycle is performed by the:

    a) ALU

    b) Control Unit

    c) Memory Unit

    d) Hard Drive

    Ans. b) Control Unit

14. Standard screen resolution mentioned is:

    a) 800 x 600

    b) 1024 x 768

    c) 640 x 480

    d) 1920 x 1080

    Ans. c) 640 x 480

15. Which is a function of an operating system?

    a) File system management

    b) Compiling programs

    c) Typing documents

d) Sending emails

Ans. a) File system management

Answer Questions

Q1. What is the role of the CPU in a computer?

A: The CPU (Central Processing Unit) is the brain of the computer where decisions are made, computations are performed, and input/output requests are delegated.

Q2. Name the two main components of the CPU and briefly describe their functions.

A:

- ALU (Arithmetic/Logical Unit): Performs arithmetic calculations and logical operations.
- Control Unit: Decodes and executes instructions and manages the fetch/execute cycle.

Q3. What is the function of memory in a computer system?

A: Memory stores information (data and instructions) to be processed by the CPU.

Q4. Give three examples of input devices and three examples of output devices.

A:

- Input: Mouse, Keyboard, Scanner
- Output: Printer, Monitor, Speakers

Q5. What is the fetch/execute cycle?

A: It is the sequence of steps the CPU takes to execute an instruction: fetch the instruction from memory, decode it, execute it, and store the result.

Q6. Define the term "program counter" (PC).

A: The program counter is a register in the CPU that holds the memory address of the next instruction to be executed.

Q7. What are pixels, and how is screen resolution measured?

A: Pixels (picture elements) are tiny dots that make up the display. Screen resolution is measured in dots per inch (DPI) or as the total number of pixels horizontally × vertically (e.g., 640 × 480).

Q8. Differentiate between application software and system software.

A:

- Application software: Programs designed to perform specific tasks for the user (e.g., word processors).
- System software: Programs that support the execution and development of other programs (e.g., operating systems, compilers).

Q9. List three examples of application software.

A: Word processors, spreadsheets, drawing programs.

Q10. What is the purpose of an operating system?

A: The operating system controls and manages computing resources, provides a file system, handles input/output operations, and manages running programs.

Q11. Name two types of translators used in a translation system.

A: Compiler and linker.

Q12. What does an IDE stand for, and what does it provide?

A: Integrated Development Environment. It provides tools for the entire software development cycle: editor, compiler, linker, loader, debugger, and viewer.

Q13. What are the main activities in the software development cycle?

A: Editing, compiling, linking, loading and executing, and viewing program behavior.

Q14. What is a compiler?

A: A compiler is a translator that converts source code written in a programming language into machine code or another language.

Q15. What is the role of a linker in software development?

A: A linker combines object files and library modules into a single executable program.

Q16. Give an example of a device that can perform both input and output operations.

A: Hard drive (can read data – input, and write data – output).

Q17. What does ALU stand for and what is its function?

A: Arithmetic/Logical Unit. It performs arithmetic calculations and logical operations.

Q18. What is the binary number system used for in computers?

A: The binary number system (base-2) is used for performing arithmetic operations and representing data in digital computers.

Q19. What is the standard screen resolution mentioned in the text?

A: 640 × 480 pixels.

Q20. What comes after the basics in the learning path according to the slide?

A: Programming Fundamentals.

Programming Exercise

1. **Write a simple C++ program to print "Hello, World!" to the screen.**

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

2. **Write a C++ program that takes two integers as input and prints their sum.**

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

3. **Write a program that calculates the area of a circle. Take radius as input**

```cpp
#include <iostream>
using namespace std;

int main() {
    float radius, area;
    const float PI = 3.14159;
    cout << "Enter radius: ";
    cin >> radius;
    area = PI * radius * radius;
    cout << "Area = " << area << endl;
```

4. **Write a program that checks whether a number entered by the user is even or odd.**

```cpp
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    if (num % 2 == 0)
        cout << "Even" << endl;
    else
        cout << "Odd" << endl;

    return 0;
}
```

5. **Write a C++ program to swap two numbers without using a third variable.**

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    a = a + b;
    b = a - b;
    a = a - b;
    cout << "After swapping: a = " << a << ", b = " << b << endl;

    return 0;
}
```

Conceptual Question

What is the main purpose of learning computer organization and basics before starting programming?

Answer: Understanding computer organization and basics provides a foundational knowledge of how hardware and software interact. It helps programmers write efficient code by knowing how the CPU processes instructions, how memory stores data, and how I/O devices work. This knowledge is crucial for debugging, optimizing performance, and developing software that effectively utilizes system resources.

Learn C++ Programming Lecture #2

Objectives

- To learn fundamental problem solving techniques
- To learn how to design a program
- To learn what is the C++ programming language and how to use it

What is C++?

- C++ is a programming language.
- C++ ranks 4th in popularity according to 2017 IEEE Spectrum Top Programming
- Language ranking. Learning C++ is a wise investment for all programmers.
- Creator: Bjarne Stroustrup

What is C++? (Definition)

"C++ is a statically-typed, free-form, (usually) compiled, multi-paradigm, intermediate-level general-purpose middle-level programming language."

- In simple terms, C++ is a sophisticated, efficient and a general-purpose programming language based on C. It was developed by Bjarne Stroustrup in 1979.
- Many of today's operating systems, system drivers, browsers and games use C++ as their core language. This makes C++ one of the most popular languages today.
- Since it is an enhanced/extended version of C programming language, C and C++ are often denoted together as C/C++.

Features of C++

- C++ is fast – Since C++ is an extended version of C, the C part of it is very low level. This offers a huge boost in speed that high-level languages like Python, Java don't give you.
- C++ is statically typed – C++ doesn't allow the compiler to make assumptions about the type of data. E.g., 10 is different from "10" and you have to let C++ know which one you are talking about.
- C++ is a multi-paradigm programming language – C++ supports at least 7 different styles of programming and gives developers the freedom to choose one at their will. Unlike Java and Python, you don't need to use objects to solve every task (if it's not necessary). You can choose the programming style that fits your use case.
- Object oriented programming with C++ – Object oriented programming helps you solve a complex problem intuitively. With its use in C++, you are able to divide these complex problems into smaller sets by creating objects.
- Power of standard library (Standard Template Library - STL) – The power of C++ extends with the use of standard libraries contained in it. These libraries contain efficient algorithms that you use extensively while coding. This saves ample amount of programming effort, which otherwise would have been wasted reinventing the wheel.

Reasons Why You Should Learn C++?

1. C++ is irreplaceable – Used in modern games, operating systems, browsers, and more. Major applications like Adobe Products (Photoshop, Illustrator, InDesign), Amazon, Autodesk products, and Facebook are heavy C++ centric products.

2. You learn the internal architecture of a computer – Since C++ is a middle-level language, you will write code that interacts directly with the internal hardware of the computer. You'll learn how computer memory really works, how information is stored and retrieved.
3. Over 600,000 C++ repositories on GitHub – This metric proves the worth of C++ in the open source community. Tons of great open source projects exist in gaming, graphics, windows applications, etc.
4. 60% StackOverflow Answer rate and active community – Over 400,000 C++ questions asked on StackOverflow, with more than 60% answered. Shows active support and interest.
5. C++ job opportunities and salary – C++ developers can expect an average yearly salary of $100,000 with over 7,700 jobs advertised every month. Jobs come mostly from game development, rendering engines, and Windows applications.

Compile and Run C++ Programming on Your OS

To run C++ Programming in Windows, you'd need to download Code::Blocks.

- Go to the binary release download page of Code::Blocks official site.
- Under Windows XP / Vista / 7 / 8.x / 10 section, click the link with mingw-setup highlighted row either from Sourceforge.net or FossHub.
- Open the Code::Blocks Setup file and follow the instructions (Next > I agree > Next > Install); you don't need to change anything. This installs Code::Blocks with GNU GCC compiler, which is the best compiler to start with for beginners.

Example C++ Program

```cpp
#include <iostream>
using namespace std;
int main()
{
    cout << "Hello World!";
    return 0;
}
```

MCQs

Multiple Choice Questions (MCQs) – C++ Lecture 2

1. Who created C++?

   a) Dennis Ritchie

   b) James Gosling

   c) Bjarne Stroustrup

   d) Guido van Rossum

   Ans: c) Bjarne Stroustrup

2. C++ was developed in which year?

   a) 1985

   b) 1979

   c) 1995

   d) 2000

   Ans: b) 1979

3. C++ is based on which language?

   a) Java

   b) Python

   c) C

   d) Fortran

   Ans: c) C

4. C++ is a _____ language.

   a) Dynamically typed

   b) Statically typed

   c) Scripting

   d) Markup

   Ans: b) Statically typed

5. C++ supports how many programming paradigms?

   a) 1

   b) 3

   c) 5

   d) At least 7

   Ans: d) At least 7

6. STL stands for:

   a) Standard Type Library

   b) Standard Template Library

   c) System Template Library

   d) Structured Template Library

   Ans: b) Standard Template Library

7. Which of these is NOT a major application using C++?

   a) Photoshop

   b) Amazon

   c) Facebook

   d) Google Search (only)

   Ans: d) Google Search (only)

8. C++ is often called a _____ level language.

   a) Low

   b) High

   c) Middle

   d) Very high

   Ans: c) Middle

9. Approximately how many C++ repositories are on GitHub?

   a) 60,000

   b) 200,000

   c) 400,000

   d) Over 600,000

   Ans: d) Over 600,000

10. What is the average yearly salary for a C++ developer as per the slide?
    a) $70,000
    b) $85,000
    c) $100,000
    d) $120,000
    Ans: c) $100,000
11. Which IDE is recommended for beginners to run C++ on Windows?
    a) Visual Studio
    b) Code::Blocks
    c) Eclipse
    d) Dev-C++
    Ans: b) Code::Blocks
12. What compiler comes with Code::Blocks for beginners?
    a) Clang
    b) Intel C++
    c) GNU GCC
    d) Turbo C++
    Ans: c) GNU GCC
13. Which of the following is a characteristic of C++?
    a) Interpreted
    b) Only object-oriented
    c) Multi-paradigm
    d) Used only for web development
    Ans: c) Multi-paradigm
14. C++ is especially popular in which fields?
    a) Game development
    b) Mobile apps only
    c) Web design only
    d) Database management only
    Ans: a) Game development

15. What does cout do in C++?
    a) Takes input
    b) Prints output
    c) Declares a variable
    d) Includes a library
    Ans: b) Prints output

Answer Questions

Short Answer Questions – C++ Lecture 2

Q1. Who created C++ and in which year?
A: C++ was created by Bjarne Stroustrup in 1979.

Q2. What rank did C++ hold in the 2017 IEEE Spectrum Top Programming Language ranking?
A: C++ ranked 4th in popularity.

Q3. Define C++ in your own words.
A: C++ is a powerful, efficient, and versatile programming language based on C, supporting multiple programming styles and used for system/software development.

Q4. Why is C++ considered a fast programming language?
A: Because it is low-level (close to hardware) and compiles directly to machine code, giving it a performance advantage over interpreted languages like Python.

Q5. What does "statically typed" mean in the context of C++?
A: It means data types must be explicitly declared and checked at compile time, not at runtime.

Q6. How many programming paradigms does C++ support?

A: C++ supports at least 7 different programming paradigms.

Q7. What is the role of Object-Oriented Programming (OOP) in C++?

A: OOP helps in breaking complex problems into smaller, manageable objects, making code modular, reusable, and easier to maintain.

Q8. What is STL, and why is it useful?

A: STL stands for Standard Template Library. It provides reusable components like containers and algorithms, saving time and effort in coding.

Q9. Name three major real-world applications that use C++.

A: Adobe Photoshop, Amazon, Facebook.

Q10. How does learning C++ help you understand computer architecture?

A: C++ allows direct interaction with hardware and memory, helping programmers understand how data is stored, retrieved, and processed.

Q11. How many C++ repositories exist on GitHub according to the slide?

A: Over 600,000 repositories.

Q12. What percentage of C++ questions on StackOverflow have been answered?

A: More than 60%.

Q13. What is the average yearly salary for a C++ developer mentioned in the slide?

A: About $100,000 per year.

Q14. Which IDE is recommended for beginners to run C++ on Windows?

A: Code::Blocks.

Q15. What compiler does Code::Blocks come with for beginners?

A: GNU GCC compiler.

Q16. Write the basic structure of a C++ program that prints "Hello World".

A:

```cpp
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World";
    return 0;
}
```

Q17. What is the purpose of `#include <iostream>` in a C++ program?

A: It includes the input/output stream library, allowing the program to use `cin` and `cout`.

Q18. What does `using namespace std;` do?

A: It allows us to use standard library functions (like `cout`, `cin`) without prefixing them with `std::`.

Q19. Why is C++ often referred to as C/C++?

A: Because C++ is an extension of C, and both are closely related; many concepts and syntax are shared.

Q20. What type of jobs most commonly require C++ skills?

A: Jobs in game development, operating systems, browsers, rendering engines, and Windows applications.

Programming Exercise

Programming Exercises – Sample Solutions (C++)

1. **Write a C++ program to print your name and age.**

```cpp
#include <iostream>
using namespace std;
int main() {
    cout << "Name: John Doe" << endl;
```

```
    cout << "Age: 25" << endl;
    return 0;
}
```

2. **Write a program that takes two integers from the user and prints their product.**

```cpp
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    cout << "Product = " << a * b << endl;
    return 0;
}
```

3. **Write a program to check if a number is positive, negative, or zero.**

```cpp
#include <iostream>
using namespace std;
int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    if (num > 0)
        cout << "Positive" << endl;
    else if (num < 0)
        cout << "Negative" << endl;
    else
        cout << "Zero" << endl;
    return 0;
}
```

4. **Write a program to print the first 5 natural numbers using a loop.**

```cpp
#include <iostream>
using namespace std;
int main() {
```

```
    for (int i = 1; i <= 5; i++) {
        cout << i << " ";
    }
    return 0;
}
```

5. **Write a program to calculate the factorial of a number.**

```cpp
#include <iostream>
using namespace std;
int main() {
    int n, fact = 1;
    cout << "Enter a number: ";
    cin >> n;
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    cout << "Factorial = " << fact << endl;
    return 0;
}
```

Conceptual Question

Considering C++'s features of being both high-performance (low-level) and supporting multiple paradigms (high-level), what is the primary advantage this offers to a software developer compared to using a purely high-level language like Python or a purely low-level language like Assembly?

A: The primary advantage is **balance and control**. C++ allows developers to write high-level, abstract, and maintainable code (like Python) for most of the application, while still having the ability to drop down to a lower level for performance-critical sections (like Assembly). This "best of both worlds" approach enables the creation of software that is both efficient (fast execution, minimal memory overhead) and manageable (easier to

write, read, and debug than pure Assembly). A developer isn't forced to choose between productivity and performance.

Basic Elements of C++ – Chapter 2

The Basics of a C++ Program

A C++ program is a collection of one or more subprograms (functions).

- Function: Collection of statements
- Statements: Accomplish a task
- Every C++ program has a function called main

Example Program

```cpp
#include <iostream>
using namespace std;
int main()
{
    cout << "Welcome to C++ Programming" << endl;
    return 0;
}
```

Program Output

```
Welcome to C++ Programming
```

The Basics of a C++ Program

- Programming language: A set of rules, symbols, special words
- Rules: Syntax – specifies legal instructions
- Symbols: Special symbols (+, -, *, !, ...)
- Word symbols: Reserved words (int, float, double, char, ...)

Identifiers

- Must begin with a letter or underscore _
- Followed by any combination of numerals or letters
- Recommend meaningful identifiers

Evaluate the following identifiers:

`ElectricCharge`

`23Skidoo`

`snarFbLat`

Data Types

Simple data types include:

- Integers
- Floating point
- Enumeration

Integer data types include:

- char
- short
- int
- long
- bool

Floating-Point Types

Stored using scientific notation:

- The sign of the number
- The significant digits of the number
- The sign of the power of 10
- The power of 10

Data Types

Different floating-point types:

- Note that various types will have different ranges of values
- Require different amounts of memory

The string Type

- A programmer-defined type
- Requires `#include <string>`
- A string is a sequence of characters
- Examples:

  `"Hi Jack"`

  `"We're Number 11"`

  `"75607"`

Arithmetic Operators and Operator Precedence

Common operators for calculations: `+, -, *, /, %`

Precedence same as in algebraic usage:

1. Inside parentheses done first
2. Next `*, /, %` from left to right
3. Then `+` and `-` from left to right

   Note: See operator precedence chart, page 1035.

Expressions

An expression includes:

- Constants
- Variables
- Function calls
- Combined with operators

  Examples:

`3/2+5.0`

`sin(x)+sqrt(y)`

Expressions

Expressions can include:

- Values all of the same type: `3+5*12-7`
- Values of different (compatible) types: `1.23*18/9.5`

  An operation is evaluated according to the types of the operands:
- If they are the same, the result is the type of the operands
- If the operands are different (int and float) then the result is float

Type Casting

- Implicit change of type can occur when operands are of different types
- It is possible to explicitly specify that an expression be converted to a different type

Input

Storing data in the computer's memory requires two steps:

1. Allocate the memory by declaring a variable
2. Have the program fetch a value from the input device and place it in the allocated memory location

   Example: `x ← 123` via `cin >> x`

Allocating Memory

Variable: A memory location whose content may change during program execution

- Declaration Syntax: `type identifier;`
- Example: `double x; int y = 45;`

  Note: Optional initialization of the variable

Named Constant: A memory location whose content cannot be changed

- Declaration Syntax: `const type identifier = value;`
- Example: `const double PI = 3.14159;`

  Note: Required initialization of the named constant

Putting Data Into Variables

At initialization time:

- Assignment statement:

  Syntax: `variable = expression;`

  Example: `x = 1.234;`
- Input (read) statement:

  Syntax: `cin >> variable;`

  Example: `cin >> height;`

Increment and Decrement Operators

- Pre-increment: `++x;` equivalent to `x = x + 1;`
- Pre-decrement: `--x;`

  Changes the value before execution of a statement

  Example: `y = ++x;`
- Post-increment: `intVal++;`
- Post-decrement: `intVal--;`

  Changes the value after execution of the statement

  Example: `y = x++;`

Output

Values sent to an output device (usually the screen, can also be a file or other device)

- Syntax for screen output: `cout << expression << …`
- Example: `cout << "The total is " << sum << endl;`

Sample Program Output

Escape sequences also used to manipulate output:

`cout << "The total is\t " << sum << endl;`

Preprocessor Directives

Commands supplied to the preprocessor (runs before the compiler, modifies source code before compilation)

- Syntax: Start with `#` symbol

  `#include <headerFileName>`
- Example: `#include <iostream>`

Preprocessor Directives

Note the preprocessor step in the sequence.

Namespace

- `#include <iostream>` is where `cin` and `cout` are declared
- They are declared within a namespace called `std`
- When we specify `using namespace std;` we do not need to preface `cin` and `cout` with `std::`

Program Style and Form

Every program must contain a function called main:

- `int main(void) { … }`
  - `int` specifies that it returns an integer value
  - `void` specifies there will be no arguments
- Also can say: `void main() { … }`

Program Style and Form

- Variables usually declared inside main at the beginning of the program
- Use blanks and space to make the program easy for humans to read

- Semicolons `;` required to end a statement
- Commas used to separate things in a list

Program Style and Form – Documentation:

- Comments specified between `/* this is a comment */`
- And following `//` also a comment

  Always put at the beginning of program:

```cpp
/* name,
   date,
   cpo,
   purpose of program */
```

Program Style and Form

- Names of identifiers should help document program

  Example: `double electricCharge;` instead of `ec`
- Prompt keyboard entry:

  `cout << "Enter the value for x -> "; cin >> x;`

MCQs

Multiple Choice Questions (MCQs) – C++ Basics

1. Every C++ program must contain:

   a) `#include <iostream>`

   b) A function named `main`

   c) `using namespace std;`

   d) All of the above

   Ans: b) A function named main

2. Which of these is a valid identifier?

   a) `23Skidoo`

   b) `__variable`

   c) `float`

   d) `my-van`

   Ans: b) __variable

3. Which is NOT an integer data type?

   a) `char`

   b) `bool`

   c) `float`

   d) `short`

   Ans: c) float

4. What header is required to use strings in C++?

   a) `<string>`

   b) `<iostream>`

   c) `<string>`

   d) `<str>`

   Ans: a) `<string>`

5. Which operator has the highest precedence?

   a) `+`

   b) `*`

   c) `( )`

   d) `-`

   Ans: c) ( )

6. What will be the result of `5 / 2` in C++?

   a) `2.5`

   b) `2`

   c) `3`

   d) `2.0`

   Ans: b) 2

7. Which is used to declare a constant?

   a) `const`

   b) `#define`

   c) `static`

   d) `final`

   Ans: a) const

8. What does `cin >> x;` do?

   a) Prints the value of `x`

   b) Reads a value into `x`

   c) Increments `x`

   d) Declares `x`

   Ans: b) Reads a value into x

9. What is the output of `y = ++x;` if `x = 5` initially?

   a) `y = 5, x = 5`

   b) `y = 6, x = 6`

   c) `y = 5, x = 6`

   d) `y = 6, x = 5`

   Ans: b) y = 6, x = 6

10. Which escape sequence inserts a tab in output?

    a) `\n`

    b) `\t`

    c) `\\`

    d) `"`

    Ans: b) \t

11. Preprocessor directives start with:

    a) `//`

    b) `/*`

    c) `#`

    d) `::`

    Ans: c) #

12. What does `using namespace std;` allow?
    a) Use of `cin` and `cout` without `std::`
    b) Use of all C++ keywords
    c) Use of only standard functions
    d) None of the above
    Ans: a) Use of cin and cout without std::
13. Which is the correct way to end a statement?
    a) `.`
    b) `:`
    c) `;`
    d) `,`
    Ans: c) ;
14. What is the purpose of `//` in C++?
    a) Division
    b) Comment
    c) Include
    d) Output
    Ans: b) Comment
15. Which of these improves program readability?
    a) Meaningful variable names
    b) Comments
    c) Proper indentation
    d) All of the above
    Ans: d) All of the above

Answer Questions

Short Answer Questions – C++ Basics

Q1. What is a function in C++?

A: A function is a collection of statements that perform a specific task.

Q2. Every C++ program must contain which function?

A: Every C++ program must contain the `main` function.

Q3. What is the purpose of `#include <iostream>`?

A: It includes the input/output stream library, which allows the program to use `cin` and `cout`.

Q4. What are the two main types of symbols in C++?

A: Special symbols (e.g., +, -, *) and reserved words (e.g., int, float).

Q5. List three valid identifiers and three invalid identifiers.

A:

Valid: `sum`, `__count`, `totalAmount`

Invalid: `23Skidoo`, `my-var`, `float`

Q6. What are the simple data types in C++?

A: Integers, floating-point numbers, and enumeration types.

Q7. Name three integer data types.

A: `int`, `short`, `long`

Q8. What is the difference between `float` and `double`?

A: `float` is single-precision (less memory, less precise), `double` is double-precision (more memory, more precise).

Q9. What header is required to use the `string` data type?

A: `#include <string>`

Q10. List the five basic arithmetic operators in C++.

A: `+, -, *, /, %`

Q11. What is operator precedence? Give an example.

A: Operator precedence determines the order in which operators are evaluated.
Example: In `3 + 5 * 2`, multiplication is done first.

Q12. What is an expression? Give two examples.

A: An expression is a combination of constants, variables, operators, and function calls.
Examples: `x + 5`, `sin(y) + 3`

Q13. What is type casting?

A: Type casting is converting a value from one data type to another, either implicitly or explicitly.

Q14. What are the two steps to store data in memory?

A: 1. Declare a variable (allocate memory) 2. Use input (e.g., `cin`) to store a value in that variable.

Q15. Differentiate between a variable and a named constant.

A: A variable's value can change during program execution, while a named constant's value cannot.

Q16. What is the difference between `++x` and `x++`?

A: `++x` increments `x` before using its value (pre-increment), `x++` increments `x` after using its value (post-increment).

Q17. What is the purpose of the `cout` statement?

A: `cout` is used to output data to the screen (or another output device).

Q18. What does `endl` do in output?

A: `endl` inserts a newline and flushes the output buffer.

Q19. What is a preprocessor directive? Give an example.

A: A preprocessor directive is a command processed before compilation.
Example: `#include <iostream>`

Q20. What is the role of the `using namespace std;` statement?

A: It allows the use of standard library names (like `cout`, `cin`) without the `std::` prefix.

Q21. How should variables be declared for good program style?

A: Variables should be declared at the beginning of the function, with meaningful names and proper indentation.

Q22. What are the two ways to write comments in C++?

A: `//` for single-line comments and `/* */` for multi-line comments.

Q23. Why is meaningful naming of identifiers important?

A: It makes code easier to read, understand, and maintain.

Q24. What is the purpose of prompting the user before input?

A: To inform the user what kind of input is expected, improving usability.

Q25. What is the difference between `int main(void)` and `void main()`?

A: `int main(void)` returns an integer and takes no arguments, while `void main()` returns nothing (non-standard in C++).

Programming Exercise

Programming Exercises – Sample Solutions (C++ Basics)

1. **Hello World**

```cpp
#include <iostream>
using namespace std;
int main() {
    cout << "Hello, C++ World!" << endl;
    return 0;
}
```

2. **Declare and Print Integer**

```cpp
#include <iostream>
using namespace std;
int main() {
    int num = 10;
    cout << "Number: " << num << endl;
    return 0;
}
```

3. **Area of Rectangle**

```cpp
#include <iostream>
using namespace std;
int main() {
    float length, width;
    cout << "Enter length: ";
    cin >> length;
    cout << "Enter width: ";
    cin >> width;
    cout << "Area: " << length * width << endl;
    return 0;
}
```

4. **Sum and Product**

```cpp
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cout << "Enter two integers: ";
    cin >> a >> b;
    cout << "Sum: " << a + b << endl;
    cout << "Product: " << a * b << endl;
    return 0;
}
```

5. **Swap Two Numbers**

```cpp
#include <iostream>
using namespace std;
int main() {
    int a, b, temp;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    temp = a;
    a = b;
    b = temp;
    cout << "After swap: a = " << a << ", b = " << b << endl;
    return 0;
}
```

Conceptual Question

Why is it considered good programming practice to use meaningful variable names (like `electricCharge`) and to add comments, even though the program might work perfectly fine with short, cryptic names and no comments?

A: While the computer executes the code based on syntax alone, humans write, read, debug, and maintain it. Meaningful names and comments serve as **documentation for human readers**. They make the code's purpose and logic immediately clear, which drastically reduces the time needed for someone else (or the original programmer after some time) to understand what the code does. This improves collaboration, reduces bugs, and makes long-term maintenance feasible. Code is read far more often than it is written.

Input and Output – Chapter 3

Chapter Topics

1. I/O Streams, Devices
2. Predefined Functions
3. Input Failure
4. Formatting Output
5. Formatting Tools
6. File I/O

I/O Streams, Devices

Input stream: A sequence of characters/bytes from an input device into the computer

Output stream: A sequence of characters/bytes from the computer to an output device

Input Streams

Input data is considered to be an endless sequence of characters/bytes coming into a program from an input device (keyboard, file, etc.)

`cin` and the Extraction Operator `>>`

A binary operator

Takes two operands:

- Name of an input stream on the left (`cin` for standard input from keyboard)
- Variable on the right

  Variables can be "cascaded":

```cpp
cin >> amount >> count >> direction;
```

- Variables should generally be simple types

The Extraction Operator `>>`

- Enables you to do input with the `cin` command
- Think of `>>` as pointing to where the data will end up

- C++ can handle different types of data and multiple inputs correctly

The Reading Marker

- Keeps track of the point in the input stream where the computer should continue reading
- Extraction `>>` operator leaves the reading marker following the last piece of data read

The Reading Marker Behavior
During execution of a `cin` command:

- As long as it keeps finding data, it keeps reading
- When the reading marker hits something not data, it quits reading
- Things in the input stream that `cin` considers not data:

  - Spaces
  - Tab `\t`
  - Newline character `\n` (pressing the RETURN key)
  - For numeric input, something non-numeric

Input to a Simple Variable

- **char**:

  - Skips any white space characters
  - Reads one character
  - Any other characters in the stream are held for later input
- **int**:

  - Skips white space characters
  - Reads leading `+` or `-`
  - Reads numerals
  - Quits reading when it hits a non-numeral
- **double (or float)**:

  - Skips leading white space

- Reads leading `+` or `-`
- Reads numerals and at most one decimal point
- Quits reading when it hits something not numeric

Predefined Functions

- A function in a computer language is similar to a function in mathematics
- The function is sent value(s) called arguments or parameters
- It manipulates the value and returns a value
- Some functions "do a task"

Reading C-String Data with `cin`

- Keyboard response of two words (separated by a space) causes the `cin` command to quit reading because the space is considered non-data

Reading C-String Data

- The `getline()` function allows the programmer to access all characters

`cin` and the `get` Function

- Syntax:

```cpp
cin.get(varChar);
```

- Example:

```cpp
cin.get(chVal);
```

`chVal` is the parameter. The `.get` function retrieves a character from the keyboard and stores it in `chVal`

`cin` and the `ignore` Function

- Syntax:

```cpp
cin.ignore(intValue, charVal);
```

- Example:

```cpp
cin.ignore(10, '\n');
```

The `ignore` function causes characters in the input stream to be ignored (discarded). In this example: for 10 characters or until a newline character occurs. It also discards the newline character. `ignore` and `get` also work for other input streams (e.g., file input streams).

Using `getline()`
Problem: `getline()` quits reading when it finds a newline.
Suppose you have terminated previous input with the `<RETURN>` key (newline still in input stream). `getline()` finds the newline immediately and declares its task finished. We must somehow discard the newline in the input stream.

Using `ignore()`
Solution: the `ignore()` command

- Tells the program to skip either the next 10 characters or until it reaches a newline, whichever comes first
- This effectively discards the newline

Input Failure

- Happens when a value in the input stream is invalid for the variable
- Example:

```cpp
int x, y;
```

```
cin >> x >> y; // User enters: B 37
```

Value of 'B' is not valid for an `int`.

- When an input stream fails, the system ignores all further I/O

The `clear` Function

- Use `clear` to return the input stream to a working state
- Example:

```cpp
cin.clear();
cin.ignore(200, '\n'); // to empty out input stream
```

Formatting Output

- Producing proper output in the proper format is important
- Specify decimal precision, left/right justification, align columns of numbers
- C++ provides I/O manipulators
- Syntax:

```cpp
cout << manipulator << expression …;
```

Manipulators

- Must first `#include <iomanip>`
- For decimal precision:

```cpp
cout << setprecision(n) << …;
```

- To output floating-point numbers in fixed decimal format:

```cpp
cout << fixed << …;
```

- To force decimal zeros to show:

```cpp
cout << showpoint << …;
```

Manipulators (Continued)

- To specify right justification in a specified number of blanks:

```cpp
cout << setw(n) << …;
```

If the number of blanks required to print the expression exceeds the specified size, the size is ignored.

- Problem: print series of names left-justified followed by right-justified numbers Example:

```text
Osgood Smart    1.23
Joe Schmo    456.78
```

Names are of different lengths; need variable number of spaces

Manipulators Example

- Print name, then variable number of spaces using `setw()`:

```cpp
cout << showpoint << fixed;
cout << name << setw(25 - strlen(name)) << " ";
cout << setw(8) << setprecision(2) << amt;
```

Formatting Tools

- Possible to specify a character to fill leading spaces:

```cpp
cout.fill('*');
```

```
cout << setw(10) << setprecision(2);
cout << pmtAmount;
```

- Result: `*****12.34`

File I/O

- Previous discussion considered input from the keyboard (fine for limited input)
- Larger amounts of data require file input
- File: An area of secondary storage used to hold information
- Keyboard I/O: `#include <iostream>`
- File I/O: `#include <fstream>`

File I/O Requirements

1. `#include <fstream>`
2. Declare a file stream variable (`ifstream` or `ofstream`)
3. Open the file:

```
cpp

whateverFile.open("filename.xxx");
```

4. Use the stream variable with `>>` or `<<`
5. Close the file:

```
cpp

whateverFile.close();
```

File Open Modes

- In some situations, you must specify one or more modes for the file:

```
cpp

fileVariable.open("filename.xxx", fileOpenMode);
```

Using Files in Programs

1. Specify `#include <fstream>` header file
2. Declare an instance of the file to be used
3. Prepare for access with `.open()` command
4. Use the name of the file in place of `cin` or `cout`

   Example:

```cpp
#include <fstream>
```

MCQs

Multiple Choice Questions (MCQs) – I/O Chapter 3

1. Which of these is an input stream?

   a) `cout`

   b) `cin`

   c) `cern`

   d) `clog`

   Ans: b) cin

2. What does `cin >> x >> y;` do?

   a) Reads two values into `x` and `y`

   b) Prints `x` and `y`

   c) Swaps `x` and `y`

   d) None of the above

   Ans: a) Reads two values into x and y

3. When reading an `int`, `cin` stops at:

   a) A space

   b) A newline

   c) A non-numeric character

d) All of the above

Ans: d) All of the above

4. Which function reads a single character from input?

   a) `cin.read()`

   b) `cin.get()`

   c) `cin.fetch()`

   d) `cin.grab()`

   Ans: b) cin.get()

5. What does `cin.ignore(100, '\n')` do?

   a) Ignores 100 characters or until newline

   b) Ignores only newlines

   c) Reads 100 characters

   d) Skips the next 100 lines

   Ans: a) Ignores 100 characters or until newline

6. Input failure occurs when:

   a) The user presses Enter

   b) The input type doesn't match the variable type

   c) The file is empty

   d) The program ends

   Ans: b) The input type doesn't match the variable type

7. To reset the input stream after failure, use:

   a) `cin.reset()`

   b) `cin.clear()`

   c) `cin.fix()`

   d) `cin.repair()`

   Ans: b) cin.clear()

8. Which header is required for `setprecision`?

   a) `<iostream>`

   b) `<iomanip>`

   c) `<cmath>`

d) `<fstream>`

Ans: b) `<iomanip>`

9. `setw(5)` affects:

   a) The next output only

   b) All following outputs

   c) Only integers

   d) Only strings

   Ans: a) The next output only

10. To output floating-point numbers in fixed notation, use:

    a) `fixed`

    b) `showpoint`

    c) `setfill`

    d) `right`

    Ans: a) fixed

11. Which manipulator fills unused spaces with a character?

    a) `setfill()`

    b) `fill()`

    c) `setw()`

    d) `pad()`

    Ans: a) setfill()

12. For file I/O, you must include:

    a) `<file>`

    b) `<fstream>`

    c) `<ifstream>`

    d) `<ofstream>`

    **Ans: b) `<fstream>`\`

13. To read from a file, you declare:

    a) `ofstream`

    b) `ifstream`

    c) `fstream`

d) `file`

Ans: b) ifstream

14. To open a file named "data.txt" for reading, you write:

    a) `open("data.txt")`

    b) `file.open("data.txt")`

    c) `ifstream myFile("data.txt")`

    d) Both b and c

    Ans: d) Both b and c

15. What does `myFile.close()` do?

    a) Deletes the file

    b) Saves and closes the file

    c) Clears file content

    d) Opens the file again

    Ans: b) Saves and closes the file

Answer Questions

Short Answer Questions – I/O Chapter 3

Q1. What is an input stream in C++?

A: An input stream is a sequence of characters/bytes coming from an input device (like a keyboard or file) into the program.

Q2. What is an output stream?

A: An output stream is a sequence of characters/bytes sent from the program to an output device (like a screen or file).

Q3. What operator is used with `cin` for input?

A: The extraction operator `>>`.

Q4. How does the extraction operator `>>` handle multiple inputs in one statement?
A: It reads values sequentially from left to right, storing each in the corresponding variable.

Q5. What is the "reading marker" in C++ input?
A: It is an invisible pointer that tracks where in the input stream the next read operation should start.

Q6. List three things that cause `cin` to stop reading input.
A: 1. A space, 2. A newline (`\n`), 3. A non-numeric character when reading a number.

Q7. How does `cin` handle input for a `char` variable?
A: It skips whitespace, reads one character, and leaves the rest in the input stream.

Q8. What happens when `cin` reads an `int` and encounters a non-numeric character?
A: It stops reading, leaves the non-numeric character in the stream, and the variable may contain an undefined value.

Q9. What is a predefined function? Give an example.
A: A function built into the language. Example: `cin.get()`.

Q10. Why might `cin >>` fail when reading strings with spaces?
A: Because `>>` stops reading at whitespace (space, tab, newline).

Q11. What function can read an entire line of text including spaces?
A: `getline(cin, str)`.

Q12. What does `cin.get(ch)` do?
A: Reads a single character (including whitespace) and stores it in `ch`.

Q13. What is the purpose of `cin.ignore()`?
A: To discard characters from the input stream, often used to clear leftover newlines or invalid input.

Q14. What is input failure and when does it occur?

A: Input failure occurs when the input data type does not match the variable type, causing the stream to enter a failed state.

Q15. How can you recover from an input stream failure?

A: Use `cin.clear()` to reset the stream state, then `cin.ignore()` to remove invalid input.

Q16. What header is needed for I/O manipulators?

A: `<iomanip>`.

Q17. How do you set floating-point output to show exactly two decimal places?

A: `cout << fixed << setprecision(2);`

Q18. What does `setw(10)` do?

A: Sets the field width to 10 characters for the next output.

Q19. How can you fill leading spaces with a specific character?

A: Use `cout.fill('*');` before outputting with `setw()`.

Q20. What header is required for file I/O?

A: `<fstream>`.

Q21. Name the two file stream types used in C++.

A: `ifstream` (input file stream) and `ofstream` (output file stream).

Q22. What are the steps to read from a file?

A: 1. Include `<fstream>`, 2. Declare `ifstream` variable, 3. Open file, 4. Read using `>>` or `getline()`, 5. Close file.

Q23. How do you open a file for writing?

A: `ofstream outfile("filename.txt");`

Q24. Why should you close a file after using it?

A: To ensure all data is written, free system resources, and allow other programs to access the file.

Q25. What is a file open mode?

A: A parameter that specifies how a file should be opened (e.g., `ios::app` for append).

Programming Exercise

Programming Exercises – I/O Chapter 3 (Solutions)

1. **Read integer and character using `cin`**

```cpp
#include <iostream>
using namespace std;
int main() {
    int num;
    char ch;
    cout << "Enter an integer and a character: ";
    cin >> num >> ch;
    cout << "Integer: " << num << ", Character: " << ch << endl;
    return 0;
}
```

2. **Read full line with `getline()`**

```cpp
#include <iostream>
#include <string>
using namespace std;
int main() {
    string line;
    cout << "Enter a line: ";
    getline(cin, line);
    cout << "You entered: " << line << endl;
    return 0;
}
```

3. **Read single character with `cin.get()`**

```cpp
#include <iostream>
using namespace std;
int main() {
    char ch;
    cout << "Enter a character: ";
    cin.get(ch);
    cout << "You entered: " << ch << endl;
    return 0;
}
```

4. **Ignore characters with** `cin.ignore()`

```cpp
#include <iostream>
using namespace std;
int main() {
    char ch;
    cout << "Enter some text: ";
    cin.ignore(50, '\n'); // Ignore up to 50 chars or until newline
    cout << "Now enter a character: ";
    cin.get(ch);
    cout << "Character: " << ch << endl;
    return 0;
}
```

5. **Handle input failure**

```cpp
#include <iostream>
using namespace std;
int main() {
    int num;
    cout << "Enter an integer: ";
    while (!(cin >> num)) {
        cin.clear();
        cin.ignore(100, '\n');
        cout << "Invalid input. Enter an integer: ";
    }
    cout << "You entered: " << num << endl;
```

```
    return 0;
}
```

Conceptual Question

Explain the difference between formatted I/O (using `>>` and `<<`) and unformatted I/O (using `get()`, `getline()`, `put()`). When would you choose one over the other?
A: Formatted I/O (`>>`, `<<`) automatically handles data type conversion, skips whitespace (for `>>`), and is convenient for reading/writing simple, delimited data. Unformatted I/O (`get()`, `getline()`, `put()`) reads/writes raw characters/bytes without interpretation, preserving whitespace and providing precise control over the stream. You would choose **formatted I/O** for typical user input (numbers, words) and structured output. You would choose **unformatted I/O** when you need to read exact characters (including spaces, newlines), process binary data, or handle input that doesn't follow standard formatting rules (e.g., reading a full line with spaces, parsing complex file formats).

Control Structures – Selection (Chapter 4)

Chapter Topics

- Control Structures
- Relational Operators
- Logical (Boolean) Operators
- Logical Expressions
- Selection:
  - `if()`

- `if() … else`
- `switch`
- The `assert` Function

Control Structures

- Statements can be executed in sequence (one after another)
- A selection structure selects which statement to execute based on whether an expression is true or false
- Statements can be repeated based on when an expression becomes false

Relational Operators

Used for comparisons in selection and repetition

Expression	Meaning	Value
`8 < 15`	8 is less than 15	`true`
`6 != 6`	6 is not equal to 6	`false`
`2.5 > 5.8`	2.5 is greater than 5.8	`false`
`5.9 <= 7.5`	5.9 is less than or equal to 7.5	`true`

String comparisons example:

```cpp
string str1 = "Hello";
string str2 = "Hi";
string str3 = "Ain";
string str4 = "Bill";
string str5 = "Big";
```

Logical (Boolean) Operators

Combine logical expressions; operands and results are logical (true/false)

- `&&` (logical AND): True only if both operands are true

- `||` (logical OR): True if at least one operand is true
- `!` (logical NOT): Reverses the logical value

Logical Expressions

Order of precedence (highest to lowest):

1. `!`
2. `&&`
3. `||`

Example: `12 > 7 || 9 * 5 >= 6 && 5 < 9`

Short-Circuit Evaluation

- For `&&`: If the first operand is false, the second is not evaluated
- For `||`: If the first operand is true, the second is not evaluated

  Example: `(x != 0) && (1.0 / x < 0.25)`

  If `x == 0`, the second part is not evaluated, avoiding division by zero

Selection: `if()`

- Syntax:

```cpp
if (logicalExpression)
    statement;
```

- Example:

```cpp
if (x < 5)
    cout << "low value for x";
```

Selection: `if() … else`

- Syntax:

```cpp
if (condition)
    statement1;
else
    statement2;
```

- Example:

```cpp
if (x < 5)
    cout << "low x";
else
    cout << "high x";
```

Compound Statements

- Use `{}` to group multiple statements
- Example:

```cpp
if (x < 5) {
    x = x + 10;
    cout << x;
}
```

Nested `if`

An `if` statement inside another `if`

```cpp
if (abs(x - 7))
    if (x < 7)
        cout << "x approaches 7 from left";
    else
        cout << "x approaches 7 from the right";
else
    cout << "x not close to 7";
```

The Dangling `else`

- Rule: An `else` pairs with the closest unmatched `if`
- To force an `else` to match a previous `if`, use `{}`

  Example:

```cpp
if (x < y) {
    if (y > 3)
        cout << "message about y > 3";
} else {
    cout << "message about x and y";
}
```

Multiple Selections

- Use `if _ else if _` for multiway branching
- Example (grading):

```cpp
if (score >= 90) grade = 'A';
else if (score >= 80) grade = 'B';
else if (score >= 70) grade = 'C';
else if (score >= 60) grade = 'D';
else grade = 'F';
```

`switch` Structures

- Syntax:

```cpp
switch (selector) {
    case L1: statements1; break;
    case L2: statements2; break;
    ...
    default: statements_n;
}
```

- Selector must be `int` or `char` (not floating-point)
- `case` labels must be constants

- `break` exits the switch; without it, execution falls through

Example Programs

Example 4: Music Notes

```cpp
switch (ch) {
    case 'c': cout << "do"; break;
    case 'd': cout << "re"; break;
    case 'e': cout << "mi"; break;
    case 'f': cout << "f"; break;
    case 'g': cout << "sol"; break;
    case 'a': cout << "la"; break;
    case 'b': cout << "ti"; break;
    default: cout << "Invalid note";
}
```

Example 5: Circle Area/Circumference

```cpp
switch (ch) {
    case 'a':
        area = 3.14f * radius * radius;
        cout << "Area = " << area; break;
    case 'c':
        circum = 2 * radius * 3.14f;
        cout << "Circumference = " << circum; break;
    default: cout << "Invalid letter";
}
```

Example 6: Grade Evaluation

```cpp
switch (ch) {
    case 'A':
    case 'a': cout << "Excellent"; break;
    case 'B':
    case 'b': cout << "Good"; break;
    case 'C':
```

```
    case 'c': cout << "O.K."; break;
    case 'D':
    case 'd':
    case 'F':
    case 'f': cout << "Poor"; break;
    default: cout << "Invalid grade";
}
```

Example 7: Sum Switch

```cpp
switch (x + y) {
    case 7: cout << "Too small"; break;
    case 5: cout << "Good job"; break;
    case 4: cout << "Nice Pick";
    case 3: cout << "Excellent"; break;
    case 2: cout << "Masterful"; break;
    case 1: cout << "Incredible"; break;
    default: cout << "Too large";
}
```

MCQs

Multiple Choice Questions (MCQs) with Answers – Control Structures

1. Which control structure executes statements one after another?

   a) Selection

   b) Repetition

   c) Sequence

   d) Branching

   Answer: c) Sequence

2. Which operator checks for equality?

   a) `=`

b) `==`

c) `!=`

d) `<=`

Answer: b) ==

3. What is the result of `!(5 > 3)`?

   a) `true`

   b) `false`

   c) `0`

   d) `1`

   Answer: b) false

4. Which logical operator returns true only if both operands are true?

   a) `||`

   b) `&&`

   c) `!`

   d) `&`

   Answer: b) &&

5. What is short-circuited in `(x != 0) && (y/x > 2)` if `x == 0`?

   a) First expression

   b) Second expression

   c) Both

   d) None

   Answer: b) Second expression

6. How many statements can follow an `if` without braces?

   a) One

   b) Two

   c) Many

   d) Unlimited

   Answer: a) One

7. In nested `if-else`, an `else` matches with:

   a) The first `if`

b) The last `if`

c) The nearest unmatched `if`

d) All `if`s

Answer: c) The nearest unmatched if

8. Which is NOT a valid `switch` selector type?

   a) `int`

   b) `char`

   c) `float`

   d) `enum`

   Answer: c) float

9. What happens without `break` in a `switch` case?

   a) Error

   b) Stops

   c) Falls through

   d) Returns

   Answer: c) Falls through

10. Which case handles unmatched values in `switch`?

    a) `else`

    b) `other`

    c) `default`

    d) `case`

    Answer: c) default

Answer Questions

Short Answer Questions – Control Structures (with Answers)

Q1. What are the three types of control structures in C++?

A: Sequence, selection, and repetition.

Q2. What is a selection structure?

A: A structure that selects which statement to execute based on whether an expression is true or false.

Q3. List all six relational operators in C++.

A: `<`, `>`, `<=`, `>=`, `==`, `!=`.

Q4. What is the difference between `=` and `==`?

A: `=` is the assignment operator; `==` is the equality comparison operator.

Q5. What are the three logical operators?

A: `&&` (AND), `||` (OR), `!` (NOT).

Q6. What is short-circuit evaluation?

A: When the second operand of `&&` or `||` is not evaluated because the first operand already determines the result.

Q7. What happens if you omit the braces `{}` in a compound `if` statement?

A: Only the first statement after the `if` is considered part of the condition; subsequent statements execute regardless.

Q8. What is a dangling `else` problem?

A: When an `else` could match multiple `if` statements; it pairs with the nearest unmatched `if`.

Q9. How do you force an `else` to match a specific `if`?

A: Use braces `{}` to group the `if` statement.

Q10. What is the main advantage of `switch` over multiple `if-else` statements?

A: Better readability and efficiency for multiway branching based on a single integer or character value.

Q11. What types can be used in a `switch` selector?

A: `int`, `char`, and `enum` (not floating-point types).

Q12. What happens if you forget `break` in a `switch` case?

A: Execution "falls through" to the next case until a `break` is encountered.

Q13. What is the purpose of the `default` case in `switch`?

A: To handle values not matched by any case label.

Q14. Can `case` labels be variables?

A: No, they must be constant expressions.

Q15. What is the `assert()` function used for?

A: To check for conditions that should always be true, typically for debugging.

Programming Exercise

Programming Exercises – Control Structures (with Solutions)

1. **Check Even or Odd**

```cpp
#include <iostream>
using namespace std;
int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    if (num % 2 == 0)
        cout << "Even" << endl;
    else
        cout << "Odd" << endl;
    return 0;
}
```

2. **Find Largest of Three Numbers**

```cpp
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int a, b, c;
    cout << "Enter three numbers: ";
    cin >> a >> b >> c;
    if (a >= b && a >= c)
        cout << a << " is largest" << endl;
    else if (b >= a && b >= c)
        cout << b << " is largest" << endl;
    else
        cout << c << " is largest" << endl;
    return 0;
}
```

3. **Check Leap Year**

```cpp
#include <iostream>
using namespace std;
int main() {
    int year;
    cout << "Enter year: ";
    cin >> year;
    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        cout << "Leap year" << endl;
    else
        cout << "Not leap year" << endl;
    return 0;
}
```

4. **Simple Calculator using `switch`**

```cpp
#include <iostream>
using namespace std;
int main() {
    char op;
    double a, b;
    cout << "Enter operator (+, -, *, /): ";
```

```cpp
    cin >> op;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    switch (op) {
        case '+': cout << a + b; break;
        case '-': cout << a - b; break;
        case '*': cout << a * b; break;
        case '/':
            if (b != 0)
                cout << a / b;
            else
                cout << "Division by zero!";
            break;
        default: cout << "Invalid operator";
    }
    return 0;
}
```

5. **Grade Calculator**

```cpp
#include <iostream>
using namespace std;
int main() {
    int score;
    cout << "Enter score (0-100): ";
    cin >> score;
    if (score >= 90) cout << "Grade: A" << endl;
    else if (score >= 80) cout << "Grade: B" << endl;
    else if (score >= 70) cout << "Grade: C" << endl;
    else if (score >= 60) cout << "Grade: D" << endl;
    else cout << "Grade: F" << endl;
    return 0;
}
```

Conceptual Question

Compare and contrast the use of an `if-else if` ladder versus a `switch` statement. Under what circumstances is each most appropriate?

A: An `if-else if` **ladder** is versatile and can handle complex conditions involving ranges (`score >= 90`), multiple variables, relational operators, and logical expressions. It is most appropriate when: 1) Conditions are not based on a single integer/character value, 2) Ranges need to be tested, 3) Conditions involve different variables.

A `switch` **statement** is optimized for testing a single integer or character expression against discrete constant values. It is most appropriate when: 1) You are branching based on one expression, 2) The expression has discrete possible values, 3) You need a clean, readable way to handle many cases. `switch` is generally faster for such multiway branching and provides better readability, but it is less flexible than `if-else if`.

Chapter 1 – Overview of Computers & Programming Languages

Chapter Contents

- Computer History
- Elements of a Computer System
- Hardware
- Software
- Language of a Computer
- Evolution of Programming Languages
- High Level Languages
- Analysis-Coding-Execution
- Object-Oriented Programming

Computer History

- 1950s: Large devices, accessible to few people
- 1960s: Commercial usage emerges; operated by experts
- 1970s: Computers cheaper, smaller
- 1990s: Computers fast, small, inexpensive; owned and used by many people

Elements of a Computer System

Hardware

Physical components of a computer system

Includes CPU, memory, input/output devices, storage

Software

- Systems programs: Control the computer
  Includes Operating System
- Applications programs:
  Word processors, compilers, spreadsheets, databases

The Language of a Computer

- Uses digital signals: all 0's and 1's (binary)
- Bits: Binary digiTs
- Data and commands stored in binary
- 8 bits = 1 byte
- ASCII character stored in 1 byte
- Integers stored in 2 or 4 bytes

Evolution of Programming Languages

1. **Machine language:** Binary numbers
2. **Assembly language:** Uses mnemonic codes
   Translated into machine language by an assembler

3. **High-level languages:** Read like English/algebra

   Translated into machine language by a compiler

   Example high-level code:

```cpp
write_string(outfile, cust_name, '1', 23);
first_line = 1;
ord.read_order(infile);
while (!ord.done()) {
    if (!first_line) write_string(outfile, " ", '1', 23);
    ord.print_order(outfile, part_list);
    first_line = 0;
    ord.read_order(infile);
}
```

Processing a High-Level Language Program

1. Source program created with an editor
2. Compiler translates source code → object code (.obj file)
3. Linker combines object code with library routines → executable code (.exe file)
4. Loader brings executable code into memory; program runs

Analysis-Coding-Execution

- **Algorithm:** A step-by-step problem-solving process that arrives at a solution in a finite amount of time
  - Steps must be simple, unambiguous
  - Steps must be performed in specified order
  - Steps must solve the problem

Problem-Solving Process

1. Analyze problem, design solution algorithm
2. Implement algorithm in a programming language, verify
3. Maintain program, adapting it to changes in requirements

Structured Programming

1. Thoroughly understand the problem
2. Determine:

- The output desired
- The required input
- Processing that will occur

3. Divide the problem into sub-problems
   (Also called: structured design, top-down design, stepwise refinement, modular programming)

Object-Oriented Programming

1. Identify objects (usually nouns in the problem description)
2. Identify operations performed on the objects (usually verbs in the problem description)

MCQs

✅ Multiple Choice Questions (MCQs) – Chapter 1

1. In which decade did computers become fast, small, and inexpensive?
   a) 1950s
   b) 1960s
   c) 1970s
   d) 1990s
   Ans. d) 1990s
2. Which of these is NOT part of hardware?
   a) CPU
   b) Operating System
   c) Memory

d) Keyboard

Ans. b) Operating System

3. What is the language of a computer?

   a) Decimal

   b) Binary

   c) ASCII

   d) English

   Ans. b) Binary

4. How many bits are in a byte?

   a) 4

   b) 8

   c) 16

   d) 32

   Ans. b) 8

5. What translates assembly language into machine language?

   a) Compiler

   b) Interpreter

   c) Assembler

   d) Linker

   Ans. c) Assembler

6. What translates high-level language into machine language?

   a) Assembler

   b) Compiler

   c) Loader

   d) Editor

   Ans. b) Compiler

7. What combines object code with library routines?

   a) Compiler

   b) Linker

   c) Loader

d) Interpreter

Ans. b) Linker

8. What is the final output of the compilation process called?

   a) Source code

   b) Object code

   c) Executable code

   d) Byte code

   Ans. c) Executable code

9. Which of these is an application program?

   a) Operating System

   b) Compiler

   c) Word Processor

   d) Both b and c

   Ans. d) Both b and c

10. What is the first step in structured programming?

    a) Divide the problem

    b) Write code

    c) Understand the problem

    d) Determine output

    Ans. c) Understand the problem

11. In OOP, objects are usually the _____ in the problem description.

    a) verbs

    b) adjectives

    c) nouns

    d) adverbs

    Ans. c) nouns

12. Which of these is a high-level language?

    a) Machine language

    b) Assembly language

    c) C++

d) Binary code

Ans. c) C++

13. What does ASCII stand for?

    a) American Standard Code for Information Interchange

    b) Automated System for Computer Information

    c) Assembly System Code for Input

    d) None of the above

    Ans. a) American Standard Code for Information Interchange

14. Which of these is NOT a characteristic of an algorithm?

    a) Unambiguous steps

    b) Finite number of steps

    c) Written in English

    d) Solves the problem

    Ans. c) Written in English

15. What brings executable code into memory?

    a) Compiler

    b) Linker

    c) Loader

    d) Editor

    Ans. c) Loader

Answer Questions

Short Answer Questions – Chapter 1

Q1. What were computers like in the 1950s?

A: They were large, expensive devices accessible only to a few people, mainly in research institutions and large corporations.

Q2. When did commercial usage of computers emerge?

A: In the 1960s.

Q3. What happened to computers in the 1970s?

A: They became smaller, cheaper, and more accessible to businesses and individuals.

Q4. What are the two main elements of a computer system?

A: Hardware and software.

Q5. Define hardware.

A: The physical components of a computer system including the CPU, memory, storage devices, and input/output devices.

Q6. What are systems programs? Give an example.

A: Programs that control and manage computer operations. Example: Operating System (like Windows or Linux).

Q7. What are applications programs? List three examples.

A: Programs designed to perform specific tasks for users. Examples: Word processors, spreadsheets, and databases.

Q8. What language does a computer understand directly?

A: Machine language (binary code consisting of 0s and 1s).

Q9. What is a bit?

A: A binary digit - the smallest unit of data in computing, represented as either 0 or 1.

Q10. How many bits are in a byte?

A: 8 bits.

Q11. How is an ASCII character stored?

A: In one byte (8 bits) of memory.

Q12. What is machine language?

A: The lowest-level programming language consisting of binary numbers (0s and 1s) that the computer's CPU executes directly.

Q13. What is assembly language?

A: A low-level programming language that uses mnemonic codes (like ADD, MOV) to represent machine instructions, translated by an assembler.

Q14. What is a compiler?

A: A program that translates source code written in a high-level programming language into machine language (object code).

Q15. What are the four steps in processing a high-level language program?

A: 1. Editing (creating source code), 2. Compiling (translating to object code), 3. Linking (combining with libraries), 4. Loading and executing.

Q16. Define an algorithm.

A: A step-by-step problem-solving procedure with a finite number of steps that produces a solution to a given problem.

Q17. What are the characteristics of a good algorithm?

A: Steps must be simple and unambiguous; steps must be performed in specified order; must solve the problem; must be finite (eventually stops).

Q18. What are the three stages of the problem-solving process?

A: 1. Analyze problem and design solution algorithm, 2. Implement algorithm in programming language and verify, 3. Maintain program and adapt to changes.

Q19. What is structured programming?

A: A programming paradigm that emphasizes breaking down problems into smaller sub-problems (also called top-down design, stepwise refinement, or modular programming).

Q20. What are the two main steps in object-oriented programming?
A: 1. Identify objects (usually nouns in the problem description), 2. Identify operations performed on those objects (usually verbs in the problem description).

Programming Exercise

Programming/Conceptual Exercises – Chapter 1 (Answers)

1. **Binary representation of decimal 10:**
   Answer: `1010`
2. **Bytes needed for "HELLO":**
   Answer: 5 bytes (one per ASCII character: H=1, E=1, L=1, L=1, O=1)
3. **Steps to create and run a C++ program:**
   Answer: Write source code in an editor → Compile using a compiler (e.g., g++) → Link object files with libraries → Load and execute the program.
4. **Algorithm to find the largest of three numbers:**
   Answer:

```text

1. Start
2. Input three numbers: a, b, c
3. If a ≥ b and a ≥ c then
     largest = a
   Else if b ≥ a and b ≥ c then
     largest = b
   Else
     largest = c
4. Display "The largest number is: " + largest
5. End
```

5. **Objects and operations for a library system:**
   Answer:

- Objects: Book, Member, Librarian
- Operations: Borrow, Return, Search, Add

Conceptual Question

Describe the evolution of programming languages from machine language to high-level languages. What fundamental problem does each successive generation of languages aim to solve?
A: The evolution progresses from **machine language** (1st gen: binary codes) which is tedious and error-prone, to **assembly language** (2nd gen: mnemonics) which improves readability but is still hardware-specific, to **high-level languages** (3rd gen: like C++, Python) which use English-like syntax and are portable. Each generation aims to solve the problem of **programmer productivity and abstraction**. Machine language is for the machine's convenience; assembly adds a thin layer of human readability; high-level languages abstract away hardware details entirely, allowing programmers to focus on problem-solving rather than machine architecture. This progression makes programming more accessible, reduces errors, and increases code portability across different computer systems.

Introduction to Programming – Lecture 1

Program

- "A precise sequence of steps to solve a particular problem" – Alan Perlis, Yale University

- "It goes against the grain of modern education to teach children to program. What fun is there in making plans, acquiring discipline in organizing thoughts, devoting attention to detail and learning to be self-critical?"

Critical Skills

- Analysis
- Critical Thinking
- Attention to Detail

Design Recipe

To design a program properly, we must:

1. Analyze a problem statement (typically expressed as a word problem)
2. Express its essence, abstractly and with examples
3. Formulate statements and comments in a precise language
4. Evaluate and revise the activities in light of checks and tests
5. PAY ATTENTION TO DETAIL

   These skills are useful for anybody. All assignments in this course should follow these guidelines.

Observations

- Computers are STUPID
- Humans are even more.....

Think Reuse

- Example: Area of the Ring = Area of Outer Circle
- Think Reuse
- Think User Interface
- Comments liberally

Logical Error Example

- Lewis Carroll – "Through the Looking Glass"
  "Twas brillig, and the slithy toves
  Did gyre and gimble in the wabe"

Course Policy

Policy for the distribution of marks and examination:

- Assignments: 15%
- Group discussion: 5%
- Midterm: 35%
- Final: 45%

Books

- Deitel & Deitel: C++ How to Program
- Kernighan and Ritchie: The C Programming Language

Course Objectives

Objectives of this course are threefold:

1. To appreciate the need for a programming language
2. To introduce the concept and usability of structured programming methodology
3. To develop proficiency in making useful software using the C language

Course Contents

To achieve our first two objectives, we will discuss:

- Basic Programming constructs and building blocks
- Structured programming
- Structured flowcharts, pseudo-code

Additional Course Topics

- History of C Language

- Variables and expressions in C
- Control structures and functions
- Arrays and Pointers
- Dynamic Memory Allocation
- File handling
- Structures and Unions
- Flavor of Object-Oriented Programming

MCQs

Multiple Choice Questions (MCQs) – Lecture 1 (with Answers)

1. According to the lecture, a program is:
   a) A machine that solves problems
   b) A precise sequence of steps to solve a problem
   c) A set of mathematical equations
   d) A computer language
   Answer: b) A precise sequence of steps to solve a problem
2. Which skill is NOT listed as critical for programming?
   a) Analysis
   b) Creativity
   c) Critical Thinking
   d) Attention to Detail
   Answer: b) Creativity
3. What is the first step in the "Design Recipe"?
   a) Write code
   b) Analyze the problem
   c) Test the program

d) Comment the code

Answer: b) Analyze the problem

4. What percentage of the total grade is the final exam?

   a) 35%

   b) 45%

   c) 15%

   d) 5%

   Answer: b) 45%

5. Which book is recommended for learning C?

   a) C++ How to Program

   b) The C Programming Language

   c) Both a and b

   d) Neither

   Answer: c) Both a and b

6. What does "Think Reuse" encourage?

   a) Writing longer code

   b) Using more variables

   c) Reusing code components

   d) Avoiding comments

   Answer: c) Reusing code components

7. Where is the logical error example taken from?

   a) Shakespeare

   b) Lewis Carroll

   c) Mark Twain

   d) George Orwell

   Answer: b) Lewis Carroll

8. Which of these is a course objective?

   a) To build a computer

   b) To appreciate the need for a programming language

   c) To learn hardware design

d) To write a novel

Answer: b) To appreciate the need for a programming language

9. What is the weightage of assignments?

   a) 5%

   b) 15%

   c) 35%

   d) 45%

   Answer: b) 15%

10. Which topic is NOT listed in the course contents?

    a) Dynamic memory allocation

    b) Web development

    c) File handling

    d) Structures and unions

    Answer: b) Web development

11. What should be added "liberally" to code?

    a) Variables

    b) Comments

    c) Functions

    d) Loops

    Answer: b) Comments

12. What does the lecture say about computers and humans?

    a) Computers are smart, humans are smarter

    b) Computers are stupid, humans are even more

    c) Both are equally intelligent

    d) Humans are stupid, computers are smart

    Answer: b) Computers are stupid, humans are even more

13. Which of these is part of structured programming?

    a) Pseudo-code

    b) Binary code

    c) Machine code

d) Assembly code

Answer: a) Pseudo-code

14. What is the last step in the Design Recipe?

    a) Analyze

    b) Express

    c) Formulate

    d) Evaluate and revise

    Answer: d) Evaluate and revise

15. Which author wrote the C programming book with Ritchie?

    a) Deitel

    b) Kernighan

    c) Stroustrup

    d) Perlis

    Answer: b) Kernighan

Answer Questions

Short Answer Questions – Lecture 1 (with Answers)

Q1. How is a program defined in this lecture?

A: A precise sequence of steps to solve a particular problem.

Q2. What did Alan Perlis say about teaching programming to children?

A: He said it goes against the grain of modern education, implying that planning, discipline, organization, attention to detail, and self-criticism are not fun for children.

Q3. What are the three critical skills mentioned for programming?

A: Analysis, Critical Thinking, and Attention to Detail.

Q4. What are the five steps in the "Design Recipe"?

A: 1. Analyze the problem statement, 2. Express its essence abstractly and with examples, 3. Formulate statements and comments in a precise language, 4. Evaluate and revise activities in light of checks and tests, 5. Pay attention to detail.

Q5. What should you always do while programming according to the lecture?

A: Pay attention to detail.

Q6. What are the two groups compared in terms of intelligence in the lecture?

A: Computers and humans (humans are implied to be even more "stupid").

Q7. What does "Think Reuse" mean?

A: To design code in a way that it can be reused in different parts of a program or in future programs.

Q8. What example is given for "Think Reuse"?

A: Area of the Ring = Area of Outer Circle.

Q9. What should you do "liberally" in your code?

A: Add comments.

Q10. What is the "Logical Error" example from literature?

A: A quote from Lewis Carroll's "Through the Looking Glass": "Twas brillig, and the slithy toves / Did gyre and gimble in the wabe."

Q11. What are the four components of the course grading policy?

A: Assignments (15%), Group Discussion (5%), Midterm (35%), Final (45%).

Q12. Name the two textbooks recommended for the course.

A: Deitel & Deitel: C++ How to Program and Kernighan and Ritchie: The C Programming Language.

Q13. What are the three main objectives of this course?
A: 1. Appreciate the need for a programming language, 2. Introduce structured programming methodology, 3. Develop proficiency in making useful software using C.

Q14. What are the three main topics covered to achieve the first two objectives?
A: Basic Programming constructs, Structured programming, Structured flowcharts and pseudo-code.

Q15. List at least five specific topics covered in the course.
A: History of C, Variables and expressions, Control structures and functions, Arrays and pointers, Dynamic memory allocation, File handling, Structures and unions, Object-oriented programming (flavor).

Programming Exercise

💻 Programming Concept Exercises – Lecture 1 (with Answers)

1. **Write a simple pseudo-code for calculating the area of a circle.**
   Answer:

```text

START
INPUT radius
area = 3.14 * radius * radius
OUTPUT area
END
```

2. **Why is attention to detail important in programming?**
   A: Because a single small mistake (like a missing semicolon or incorrect variable name) can cause the entire program to fail or produce incorrect results.

3. **Give an example of code reuse in a real program.**
   A: A function that calculates the average of numbers can be reused in different parts of a program (e.g., in a grade calculator, weather app, or financial software).
4. **What is the purpose of comments in code?**
   A: Comments explain what the code does, making it easier for others (and yourself) to understand, debug, and maintain the code later.
5. **How does structured programming differ from unstructured programming?**
   A: Structured programming uses clear, logical structures like sequences, selections (if/else), and loops (while/for), making code easier to read and maintain. Unstructured programming uses goto statements and lacks clear organization.
6. **Write a structured flowchart for a program that checks if a number is positive or negative.**
   Answer:

```text

[Start] → [Input number] → [If number >= 0] → Yes → [Print "Positive"] → [End]
                                ↓ No
                          [Print "Negative"] → [End]
```

7. **What is the relationship between analysis and programming?**
   A: Analysis is the process of understanding the problem and breaking it down into solvable parts before writing code. Good analysis leads to better, more efficient programs.
8. **Why should you "Think User Interface" when designing a program?**
   A: Because a good user interface makes the program easier to use, reduces errors, and improves the overall user experience.
9. **How can logical errors be identified in a program?**
   A: Through careful testing, debugging, code reviews, and comparing expected output with actual output.
10. **What is the benefit of learning structured programming before object-oriented programming?**
    A: Structured programming teaches fundamental problem-solving skills, logical

thinking, and code organization, which form a strong foundation for learning more advanced concepts like OOP.

Conceptual Question

🎯 **Critical Thinking Question – Lecture 1**
Why does Alan Perlis suggest that teaching programming is against modern education trends? What essential life skills does programming develop that might be overlooked in conventional education?
A: Perlis suggests modern education often prioritizes immediate engagement and entertainment, while programming requires delayed gratification through planning, discipline, systematic thinking, and meticulous attention to detail. Programming develops **problem decomposition** (breaking complex issues into manageable parts), **logical reasoning** (if-then-else thinking), **persistence** in debugging, and **precision** in communication with a literal-minded machine. These are fundamental cognitive skills applicable to any analytical field, but they require patience and practice that might not align with "fun-focused" educational approaches.

Introduction to Programming – Lecture 2

Today's Lecture Topics

1. Software Categories
   - System Software
   - Application Software
2. Introduction to 'C' Language

- History
- Evolution
- Justification

3. Development Environment of ‘C’

Software Categories

There are two main categories of software:

1. System Software (e.g., operating systems, drivers, utilities)
2. Application Software (e.g., word processors, games, databases)

Introduction to ‘C’ Language

- History
- Evolution
- Justification (why learn C)

TWAIN

- Technology Without An Interesting Name
- ANSI C (refers to the standardized version of C)

Tools of the Trade

- Editor (for writing code)
- Interpreters and Compilers (for translating code)
- Debuggers (for finding and fixing errors)

Integrated Development Environment (IDE)

An IDE contains:

- Editor
- Compilers
- Debugger

- Linkers
- Loaders

MCQs

✅ Multiple Choice Questions (MCQs) – Lecture 2 (with Answers)

1. Which of these is a system software?
   a) Microsoft Word
   b) Windows OS
   c) Photoshop
   d) Google Chrome
   Answer: b) Windows OS
2. Which of these is an application software?
   a) Device Driver
   b) Compiler
   c) Spreadsheet
   d) Operating System
   Answer: c) Spreadsheet
3. What does TWAIN stand for?
   a) Technology With An Interesting Name
   b) Technology Without An Interesting Name
   c) Tool Without An Interesting Name
   d) None of the above
   Answer: b) Technology Without An Interesting Name
4. ANSI C refers to:
   a) A version of C++
   b) The standardized version of C
   c) A type of compiler

d) A programming tool

Answer: b) The standardized version of C

5. Which tool is used to write source code?

   a) Compiler

   b) Debugger

   c) Editor

   d) Linker

   Answer: c) Editor

6. Which tool translates source code to machine code?

   a) Editor

   b) Debugger

   c) Compiler

   d) Loader

   Answer: c) Compiler

7. Which tool helps find errors in code?

   a) Editor

   b) Debugger

   c) Linker

   d) Loader

   Answer: b) Debugger

8. What does IDE stand for?

   a) Integrated Development Environment

   b) Internal Development Editor

   c) Integrated Debugging Environment

   d) International Development Environment

   Answer: a) Integrated Development Environment

9. Which of these is NOT typically part of an IDE?

   a) Editor

   b) Compiler

   c) Web Browser

d) Debugger

Answer: c) Web Browser

10. What is the main purpose of a linker?

    a) Write code

    b) Find errors

    c) Combine object files

    d) Execute program

    Answer: c) Combine object files

11. Which component loads the program into memory?

    a) Compiler

    b) Linker

    c) Loader

    d) Debugger

    Answer: c) Loader

12. C language is the foundation for which popular language?

    a) Python

    b) Java

    c) C++

    d) HTML

    Answer: c) C++

13. Which tool executes code line by line?

    a) Compiler

    b) Interpreter

    c) Linker

    d) Loader

    Answer: b) Interpreter

14. What does "justification" refer to in the context of C language?

    a) Why C is still used

    b) How C was created

    c) Who created C

d) Where C is used

Answer: a) Why C is still used

15. Which of these is a benefit of using an IDE?

a) All tools in one place

b) Only for experts

c) Makes code run faster

d) Replaces the need for a compiler

Answer: a) All tools in one place

Answer Questions

Short Answer Questions – Lecture 2 (with Answers)

Q1. What are the two main categories of software discussed in Lecture 2?

A: System Software and Application Software.

Q2. Give one example of System Software.

A: Operating System (e.g., Windows, Linux).

Q3. Give one example of Application Software.

A: Word Processor (e.g., Microsoft Word).

Q4. What three aspects of the C language are introduced in this lecture?

A: History, Evolution, and Justification.

Q5. What does TWAIN stand for?

A: Technology Without An Interesting Name.

Q6. What is ANSI C?

A: The standardized version of the C programming language.

Q7. Name three tools used in C programming.

A: Editor, Compiler, Debugger.

Q8. What is the difference between an interpreter and a compiler?

A: An interpreter translates and executes code line by line, while a compiler translates the entire program at once into machine code.

Q9. What does an IDE stand for?

A: Integrated Development Environment.

Q10. List five components typically found in an IDE.

A: Editor, Compiler, Debugger, Linker, Loader.

Q11. Why is C language still relevant today?

A: It is efficient, portable, and forms the foundation for many other languages and systems.

Q12. What is the purpose of a debugger?

A: To help find and fix errors (bugs) in the code.

Q13. What does a linker do?

A: It combines object files and libraries into a single executable program.

Q14. What does a loader do?

A: It loads the executable program into memory so it can run.

Q15. Which organization standardized C as ANSI C?

A: The American National Standards Institute (ANSI).

Programming Exercise

💻 Programming Concept Exercises – Lecture 2 (with Answers)

1. **Match the tool with its function:**

- Editor → c) Writes and modifies source code
- Compiler → a) Translates source code to machine code
- Debugger → b) Helps find and fix errors
- Linker → d) Combines object files into an executable
- Loader → e) Loads executable into memory

2. **Explain the difference between system software and application software with examples.**

   A: System Software: Manages and controls computer hardware (e.g., Windows OS, Linux, device drivers). Application Software: Performs specific tasks for users (e.g., Microsoft Word, Google Chrome, Photoshop).

3. **Why is C called a "middle-level" language?**

   A: Because it combines features of low-level languages (like assembly) with high-level languages (like Python), allowing both hardware control and ease of programming.

4. **What is the role of ANSI in C programming?**

   A: ANSI created a standard for C (ANSI C) to ensure consistency and portability across different compilers and platforms.

5. **List three popular IDEs for C programming.**

   A: 1. Code::Blocks, 2. Dev-C++, 3. Visual Studio Code, 4. Eclipse, 5. Turbo C (any three).

6. **Why might a programmer prefer an IDE over separate tools?**

   A: Because an IDE integrates all necessary tools (editor, compiler, debugger, etc.) into one interface, improving productivity and simplifying the development process.

7. **What is the purpose of the "History" and "Evolution" topics when learning C?**

   A: To understand how C developed, its design principles, and why it remains influential in modern programming.

8. **Give two reasons why C is still taught and used today.**

   A: 1. Efficiency: C programs are fast and use minimal resources. 2. Foundation: Many modern languages (C++, Java, C#) and operating systems are built on C.

9. **What does "portability" mean in programming, and how does C achieve it?**
   A: Portability means code can run on different systems with little or no modification. C achieves this through standardization (ANSI C) and compiler support across platforms.
10. **Describe the typical workflow when developing a C program using an IDE.**
    A: 1. Write code in the editor, 2. Compile the code using the compiler, 3. Debug errors using the debugger, 4. Link object files using the linker, 5. Load and run the program using the loader.

Conceptual Question

🎯 Critical Thinking Questions – Lecture 2 (with Answers)

1. **Why do you think C has evolved but not been replaced entirely by newer languages?**
   A: C is close to hardware, highly efficient, and forms the core of many systems (OS kernels, embedded systems). Newer languages often build on C concepts rather than replace it.
2. **What might be the disadvantages of using an IDE for beginners?**
   A: IDEs can be complex, hide underlying processes, and may encourage dependency on automated tools rather than understanding how compilation and linking work.
3. **Why is standardization (like ANSI C) important in programming languages?**
   A: It ensures code written on one system works on another, promotes best practices, and allows developers to share and maintain code reliably.
4. **How does understanding the history of C help a programmer?**
   A: It provides context for why C was designed the way it was, reveals its strengths and limitations, and helps in understanding modern languages that evolved from C.
5. **If you were to design a new IDE for C, what features would you include?**
   A: Smart code completion, real-time error checking, integrated documentation, version control support, project management tools, cross-platform compatibility.

Introduction to Programming – Lesson 3

Sample C++ Program

```cpp
#include <iostream.h>

main ( )
{
    cout << " Welcome to Virtual University ";
}
```

Variable

- A variable is a named location in memory.
- Example: `x = 2;`

Variable Naming Rules

- Must start with:
  - A character (letter)
  - An underscore _ (not recommended)
- Think of a variable like a small post box in memory.

Variable Properties

A variable in a program has:

1. Name
2. Type
3. Size
4. Value

Assignment Operator

- Symbol: `=`
- Example: `x = 2;`

Assignment Rules

- Left-hand side (LHS) = Right-hand side (RHS)
- Correct: `z = x + 4;`
- Incorrect: `x + 4 = z;`

Reassignment Example

```cpp
x = 10;
x = 30; // x is now 30
```

Self-Assignment Example

```cpp
x = x + 1; // Increases x by 1
```

Data Types

- Declaration example: `int i;`

Sample Program with Variables

```cpp
#include <iostream.h>
main ( )
{
    int x;
    int y;
    int z;
    x = 10;
    y = 20;
    z = x + y;
```

```
    cout << " x = ";
    cout << x;
    cout << " y = ";
    cout << y;
    cout << " z = x + y = ";
    cout << z;
}
```

Multiple Variable Declarations

```cpp
int x, y, z; // Single Line
int x; int y; int z; // Multiple Lines
```

Common Data Types

- `int`
- `short`
- `long`
- `float`
- `double`
- `char`

Arithmetic Operators

Operator	Symbol
Plus	`+`
Minus	`-`
Multiply	`*`
Divide	`/`
Modulus	`%`

Examples

```cpp
```

```
i + j
x * y
a / b
a % b  // Modulus (remainder)
```

Modulus Examples

- `5 % 2 = 1`
- `2 % 2 = 0`
- `4 / 2 = 2`
- `5 / 2 = ?` (Integer division result: 2)

Operator Precedence (Highest to Lowest)

1. Highest: `( )` Parentheses
2. Next: `*, /, %`
3. Lowest: `+, -`

MCQs

✅ Multiple Choice Questions (MCQs) – Lesson 3 (with Answers)

1. What is a variable?

   a) A constant value

   b) A named memory location

   c) A type of operator

   d) A function

   Answer: b) A named memory location
2. Which of these is a valid variable name?

   a) `1var`

   b) `var-1`

c) `_var`

d) `var name`

Answer: c) _var

3. What does the assignment operator do?

   a) Compares values

   b) Stores a value in a variable

   c) Adds two values

   d) Prints output

   Answer: b) Stores a value in a variable

4. Which side of an assignment must be a variable?

   a) Left-hand side

   b) Right-hand side

   c) Both sides

   d) Neither side

   Answer: a) Left-hand side

5. What is the result of `7 % 3`?

   a) `2`

   b) `1`

   c) `0`

   d) `3`

   Answer: a) 2

6. Which operator has the highest precedence?

   a) `+`

   b) `*`

   c) `()`

   d) `%`

   Answer: c) ()

7. What is `5 / 2` in integer division?

   a) `2.5`

   b) `2`

c) `3`

d) `2.0`

Answer: b) 2

8. Which is NOT an arithmetic operator?

   a) `+`

   b) `-`

   c) `=`

   d) `*`

   Answer: c) = (it's an assignment operator)

9. Which data type is used for whole numbers?

   a) `float`

   b) `double`

   c) `int`

   d) `char`

   Answer: c) int

10. What does `x = x + 1` mean?

    a) `x` is compared to `x + 1`

    b) `x` is assigned the value of `x + 1`

    c) `x` is printed

    d) Error

    Answer: b) x is assigned the value of x + 1

Answer Questions

Short Answer Questions – Lesson 3 (with Answers)

Q1. What is a variable in programming?

A: A named location in memory used to store data.

Q2. What must a variable name start with?

A: A letter (character) or an underscore (though underscore is not recommended).

Q3. What are the four properties of a variable?

A: Name, type, size, and value.

Q4. What is the assignment operator in C++?

A: `=` (equals sign).

Q5. In the assignment `z = x + 4`, which side is the LHS and which is the RHS?

A: LHS = `z`, RHS = `x + 4`.

Q6. Why is `x + 4 = z` incorrect?

A: Because the LHS must be a single variable, not an expression.

Q7. What does `x = x + 1` do?

A: It increases the value of `x` by 1.

Q8. What does the `%` operator do?

A: It gives the remainder of a division (modulus).

Q9. What is the result of `5 % 2`?

A: `1`.

Q10. What is integer division? Give an example.

A: Division where the fractional part is discarded. Example: `5 / 2 = 2`.

Q11. List three integer data types in C++.

A: `int`, `short`, `long`.

Q12. List two floating-point data types.

A: `float`, `double`.

Q13. What is the character data type called?

A: `char`.

Q14. What is the order of precedence for arithmetic operators?

A: Parentheses `()` first, then `*, /, %`, then `+, -`.

Q15. How do you declare multiple variables of the same type in one line?

A: `int x, y, z;`

Programming Exercise

💻 Programming Exercises – Lesson 3 (with Solutions)

1. **Variable Declaration and Initialization**

```cpp
#include <iostream>
using namespace std;
int main() {
    int a = 5, b = 10;
    int sum = a + b;
    cout << "Sum = " << sum << endl;
    return 0;
}
```

2. **Modulus Operator Practice**

```cpp
#include <iostream>
using namespace std;
int main() {
    int num1 = 17, num2 = 5;
    int remainder = num1 % num2;
    cout << "Remainder = " << remainder << endl;
    return 0;
}
```

3. **Operator Precedence Demonstration**

```cpp
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int result = 3 + 4 * 2;  // 4*2 is done first
    cout << "Result = " << result << endl;  // Output: 11
    return 0;
}
```

4. **Using Parentheses to Change Precedence**

```cpp
#include <iostream>
using namespace std;
int main() {
    int result = (3 + 4) * 2;  // 3+4 is done first
    cout << "Result = " << result << endl;  // Output: 14
    return 0;
}
```

5. **Self-Assignment Practice**

```cpp
#include <iostream>
using namespace std;
int main() {
    int x = 5;
    x = x + 3;  // x becomes 8
    cout << "x = " << x << endl;
    x = x * 2;  // x becomes 16
    cout << "x = " << x << endl;
    return 0;
}
```

Conceptual Question

📊 Concept Practice Problems (with Answers)

1. **What is the output?**

```cpp
int x = 10;
x = x + 5;
cout << x;
```

Answer: `15`

2. **What is the output?**

```cpp
int a = 8, b = 3;
cout << a % b;
```

Answer: `2`

3. **What is the output?**

```cpp
int result = 2 + 3 * 4;
cout << result;
```

Answer: `14` (Multiplication first: `3*4=12`, then `2+12=14`)

4. **Correct this invalid assignment:**

```cpp
5 + x = y;
```

Answer: `y = 5 + x;`

5. **Declare three integer variables in one line.**

Answer: `int a, b, c;`

Conceptual Question:

Why is understanding operator precedence crucial when writing arithmetic expressions in programming? Provide an example where incorrect precedence would lead to a logical error.

A: Operator precedence determines the order in which operations are performed, which

directly affects the result of an expression. Without proper understanding, a programmer might write an expression expecting one computation order while the compiler follows another, leading to subtle bugs.

Example: Suppose a programmer writes `average = a + b + c / 3` to calculate the average of three numbers. Due to precedence (`/` before `+`), this actually computes `a + b + (c / 3)` instead of `(a + b + c) / 3`. The result would be incorrect unless parentheses are used: `average = (a + b + c) / 3`.

Introduction to Programming – Lecture 4

Keywords of C

- `main`
- `if`
- `else`
- `while`
- `do`
- `for`

Assignment Statements

- Example: `x = 2 + 4; // x becomes 6`
- Memory representation: Variable x stores the value 6.

Multiple Variable Assignment

- Example: `x = a + b;`
  Values from a and b are added and stored in x.

Program Example: Average Age Calculator

```cpp
#include <iostream.h>
main ( )
{
    int age1, age2, age3, age4, age5, age6, age7, age8, age9, age10;
    int TotalAge;
    int AverageAge;

    cout << "Please enter the age of student 1: ";
    cin >> age1;
    cout << "Please enter the age of student 2: ";
    cin >> age2;
    // ... continues for all 10 students

    TotalAge = age1 + age2 + age3 + age4 + age5 + age6 + age7 + age8 + age9 + age10;
    AverageAge = TotalAge / 10;

    cout << "The average age of the class is: "<< AverageAge;
}
```

Quadratic Equation in C

- Algebra form: `y = ax² + bx + c`
- C form: `y = a * x * x + b * x + c;`

Operator Precedence Examples

- Expression: `a * b % c + d`
- Equivalent to: `a * (b % c) // Not correct - actual precedence: * and % have same precedence`
- Actually equals: `(a * b) % c + d`

Common Mistake: Discriminant Calculation

- Incorrect: `b*b - 4*a*c / 2*a`
- Correct: `(b*b - 4*a*c) / (2*a)`

Important Notes

1. No expression on the left-hand side of assignment
2. Integer division truncates fractional part
3. Use parentheses liberally for clarity and correctness

Interesting Problem: Separate Digits of a 4-Digit Integer

Analysis

Number = 1234

1. Get last digit: `1234 % 10 = 4`
2. Remove last digit: `1234 / 10 = 123` (integer division truncates)
3. Get next digit: `123 % 10 = 3`
4. Remove last digit: `123 / 10 = 12`
5. Get next digit: `12 % 10 = 2`
6. Remove last digit: `12 / 10 = 1`

CODE

```cpp
#include <iostream.h>

main ( )
{
    int number;
    int digit;

    cout << "Please enter a 4-digit integer: ";
    cin >> number;

    digit = number % 10;
    cout << "The digit is: " << digit << '\n';
    number = number / 10;

    digit = number % 10;
```

```
    cout << "The digit is: " << digit << '\n';
    number = number / 10;

    digit = number % 10;
    cout << "The digit is: " << digit << '\n';
    number = number / 10;

    digit = number % 10;
    cout << "The digit is: " << digit;
}
```

Special Character: Newline

- `\n` - Used to move to the next line in output.

MCQs

✅ Multiple Choice Questions (MCQs) – Lecture 4 (with Answers)

1. Which of these is NOT a C keyword?

   a) `main`

   b) `if`

   c) `cout`

   d) `while`

   Answer: c) cout

2. What is stored in x after `x = 2 + 4;`?

   a) `2`

   b) `4`

   c) `6`

   d) `8`

   Answer: c) 6

3. How is `y = ax² + bx + c` written in C?

   a) `y = a*x^2 + b*x + c`

   b) `y = a*x*x + b*x + c`

   c) `y = a*x² + b*x + c`

   d) `y = a*x*2 + b*x + c`

   Answer: b) y = a*x*x + b*x + c

4. What is the result of `7 / 2` in integer division?

   a) `3.5`

   b) `3`

   c) `4`

   d) `3.0`

   Answer: b) 3

5. Which operator gives the remainder?

   a) `/`

   b) `%`

   c) `*`

   d) `+`

   Answer: b) %

6. What is `1234 % 10`?

   a) `123`

   b) `4`

   c) `1234`

   d) `1`

   Answer: b) 4

7. What is `1234 / 10` in integer division?

   a) `123.4`

   b) `123`

   c) `124`

   d) `12`

   Answer: b) 123

8. What does `\n` do?

   a) Tab space

   b) New line

   c) Backspace

   d) Quote

   Answer: b) New line

9. Which is the correct discriminant formula?

   a) `b*b - 4*a*c / 2*a`

   b) `(b*b - 4*a*c) / 2*a`

   c) `(b*b - 4*a*c) / (2*a)`

   d) `b*b - (4*a*c) / (2*a)`

   Answer: c) (b*b - 4*a*c) / (2*a)

10. How many variables are declared in the average age program?

    a) `10`

    b) `11`

    c) `12`

    d) `13`

    Answer: d) 13 (10 age variables + TotalAge + AverageAge)

Answer Questions

📝 Short Answer Questions – Lecture 4 (with Answers)

Q1. List at least 4 keywords in C mentioned in this lecture.

A: `main`, `if`, `else`, `while`, `do`, `for` (any 4).

Q2. What is the result of `x = 2 + 4;`?

A: `x` becomes `6`.

Q3. In the expression `x = a + b;`, where do the values come from?

A: From variables `a` and `b`.

Q4. What is the purpose of the average age program example?

A: To calculate and display the average age of 10 students.

Q5. How do you write the quadratic equation `y = ax² + bx + c` in C?

A: `y = a * x * x + b * x + c;`

Q6. Why is `b*b - 4*a*c / 2*a` incorrect for calculating the discriminant?

A: Because division (`/`) and multiplication (`*`) have higher precedence than subtraction, so it's evaluated as `b*b - ((4*a*c)/(2*a))`.

Q7. What is the correct way to write the discriminant formula?

A: `(b*b - 4*a*c) / (2*a)`

Q8. What happens in integer division?

A: The fractional part is truncated (discarded).

Q9. Why should you use parentheses liberally in expressions?

A: To ensure correct order of evaluation and improve readability.

Q10. What operator is used to get the remainder of a division?

A: The modulus operator `%`.

Q11. How do you extract the last digit of a number?

A: Using `number % 10`.

Q12. How do you remove the last digit of a number?

A: Using `number / 10` (integer division).

Q13. What special character is used for a new line in C output?

A: `\n`

Q14. In the digit separation problem, what is the first step?

A: Get the last digit using `digit = number % 10;`

Q15. How many times is the process repeated to separate all digits of a 4-digit number?

A: 4 times.

Programming Exercise

💻 Programming Exercises – Lecture 4 (with Solutions)

1. **Quadratic Equation Calculator**

```cpp
#include <iostream>
using namespace std;
int main() {
    float a, b, c, x, y;
    cout << "Enter a, b, c for ax^2 + bx + c: ";
    cin >> a >> b >> c;
    cout << "Enter x: ";
    cin >> x;
    y = a * x * x + b * x + c;
    cout << "y = " << y << endl;
    return 0;
}
```

2. **Digit Separator for Any Number**

```cpp
#include <iostream>
using namespace std;
int main() {
    int num, digit;
    cout << "Enter a number: ";
    cin >> num;
```

```
    while (num > 0) {
        digit = num % 10;
        cout << "Digit: " << digit << endl;
        num = num / 10;
    }
    return 0;
}
```

3. **Average Calculator (Improved)**

```cpp
#include <iostream>
using namespace std;
int main() {
    int n, age, total = 0;
    cout << "How many students? ";
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cout << "Enter age of student " << i << ": ";
        cin >> age;
        total += age;
    }
    float average = (float)total / n;
    cout << "Average age: " << average << endl;
    return 0;
}
```

4. **Discriminant Calculator**

```cpp
#include <iostream>
using namespace std;
int main() {
    float a, b, c, discriminant;
    cout << "Enter a, b, c for quadratic equation: ";
    cin >> a >> b >> c;
    discriminant = (b * b - 4 * a * c) / (2 * a);
    cout << "Discriminant/(2a) = " << discriminant << endl;
    return 0;
}
```

5. **Reverse a Number**

```cpp
#include <iostream>
using namespace std;
int main() {
    int num, reversed = 0;
    cout << "Enter a number: ";
    cin >> num;
    while (num > 0) {
        reversed = reversed * 10 + (num % 10);
        num = num / 10;
    }
    cout << "Reversed: " << reversed << endl;
    return 0;
}
```

Conceptual Question

🎹 Concept Practice Problems (with Answers)

1. **What is the output?**

```cpp
int x = 17;
cout << x % 10;
```

Answer: `7`

2. **What is the output?**

```cpp
int x = 123;
x = x / 10;
cout << x;
```

Answer: `12`

3. **Write the expression for `(a + b)²` in C.**

   Answer: `(a + b) * (a + b)` or `a*a + 2*a*b + b*b`

4. **Correct this expression: `a + b * c / d - e` to add a and b first.**

   Answer: `(a + b) * c / d - e`

5. **How would you extract the tens digit from a number?**

   Answer: `(number / 10) % 10`

   Conceptual Question:
   Explain why integer division truncates the fractional part rather than rounding. How might this behavior lead to programming errors, and what precautions should a programmer take when performing division with integers?
   A: Integer division truncates because integers cannot represent fractional values by definition—they are whole numbers. The truncation is a direct consequence of storing the result in an integer variable type. This can lead to errors in calculations where fractional precision matters, such as computing averages, percentages, or financial calculations.
   Precautions: 1) Use floating-point types (`float`, `double`) when fractional results are needed, 2) Cast one operand to float before division (e.g., `(float)total / count`), 3) Be aware of truncation in intermediate steps of complex calculations, and 4) Use integer division intentionally only when the truncation behavior is desired (e.g., splitting digits).

Introduction to Programming – Lecture 5

In the Previous Lecture

- Basic structure of C program
- Variables and Data types
- Operators
- `cout` and `cin` for output and input

- Braces

Decision Making – `if` Statement

- Example in plain English: If Ali's height is greater than 6 feet, then Ali can become a member of the Basketball team.

`if` Statement in C

Simple form:

```cpp
if (condition)
    statement;
```

Block form:

```cpp
if (condition) {
    statement1;
    statement2;
    // ...
}
```

Example:

```cpp
if (age1 > age2)
    cout << "Student 1 is older than student 2";
```

Relational Operators

Operator	Meaning
`<`	less than
`<=`	less than or equal to
`==`	equal to
`>=`	greater than or equal to

`>`	greater than
`!=`	not equal to

Examples

```cpp
a != b;
x = 0;
x == 0;
```

Complete Example Program

```cpp
#include <iostream.h>

main ( )
{
    int AmirAge, AmaraAge;
    AmirAge = 0;
    AmaraAge = 0;

    cout << "Please enter Amir's age";
    cin >> AmirAge;
    cout << "Please enter Amara's age";
    cin >> AmaraAge;

    if (AmirAge > AmaraAge)
    {
        cout << "\n" << "Amir's age is greater than Amara's age";
    }
}
```

Flowchart Symbols

- Oval: Start or stop
- Rectangle: Process
- Diamond: Decision

- Arrow: Flow line
- Circle: Continuation mark

Flowchart for `if` Statement

```text
[Start] → [Condition] → (True) → [Process] → [End]
        |
        (false)
        |
        [End]
```

- Entry point for `if` block
- Exit point for `if` block
- Note: Indentation from left to right in code

Logical Operators

- AND: `&&`
- OR: `||`

Examples:

```cpp
if (a > b && c > d)
if (age > 18 || height > 5)
```

`if-else` Statement

```cpp
if (condition) {
    // statements if true
} else {
    // statements if false
}
```

Flowchart for `if-else`

```text
```

```
[Start] → [Condition] → (True) → [Process 1] → [End]
        |
        (False)
        |
        [Process 2] → [End]
```

Code Examples Comparison

Using Two Separate `ifs`

```cpp
if (AmirAge > AmaraAge) {
    cout << "Amir is older than Amara";
}
if (AmirAge < AmaraAge) {
    cout << "Amir is younger than Amara";
}
```

Using `if-else`

```cpp
if (AmirAge > AmaraAge) {
    cout << "Amir is older than Amara";
} else {
    cout << "Amir is younger than Amara";
}
```

Including Equality

```cpp
if (AmirAge > AmaraAge) {
    cout << "Amir is older than Amara";
} else {
    cout << "Amir is younger than or of the same age as Amara";
}
```

Unary NOT Operator

- `!true = false`

- `!false = true`

  Examples:

```cpp
if (AmirAge != AmaraAge)
    cout << "Amir and Amara's Ages are not equal";

if (!(AmirAge > AmaraAge))
    // Means: if AmirAge is NOT greater than AmaraAge
```

Complex Condition Example

```cpp
if ((interMarks > 45) && (testMarks >= passMarks)) {
    cout << " Welcome to GC University";
}
```

Negating a Complex Condition

```cpp
if (!((interMarks > 45) && (testMarks >= passMarks)))
    // Means: if NOT (interMarks > 45 AND testMarks >= passMarks)
```

Nested `if`

```cpp
if (age > 18) {
    if (height > 5) {
        // statements
    }
}
```

Today's Lecture Summary

- Decision making
- `if` statement
- `else` statement
- Flowcharts

- Nested `if`

MCQs

Multiple Choice Questions (MCQs) – Lecture 5 (with Answers)

1. Which operator is used for "equal to" comparison?

   a) `=`

   b) `==`

   c) `!=`

   d) `<>`

   Answer: b) `==`

2. What is the correct syntax for a simple `if` statement?

   a) `if condition {}`

   b) `if (condition) {}`

   c) `if (condition) then {}`

   d) `if (condition)`

   Answer: b) if (condition) {}

3. Which logical operator means "AND"?

   a) `&&`

   b) `||`

   c) `!`

   d) `&`

   Answer: a) &&

4. What does `if (!condition)` mean?

   a) If condition is true

   b) If condition is false

   c) If condition is zero

d) If condition is positive

Answer: b) If condition is false

5. Which is a valid `if-else` structure?

   a) `if (x>y) {} otherwise {}`

   b) `if (x>y) {} else {}`

   c) `if (x>y) {} or {}`

   d) `if (x>y) {} then else {}`

   Answer: b) if (x>y) {} else {}

6. What is the result of `(5 > 3) && (2 < 1)`?

   a) True

   b) False

   c) 1

   d) 0

   Answer: b) False

7. What is the result of `(5 > 3) || (2 < 1)`?

   a) True

   b) False

   c) 1

   d) 0

   Answer: a) True

8. Which flowchart symbol represents a decision?

   a) Rectangle

   b) Oval

   c) Diamond

   d) Circle

   Answer: c) Diamond

9. What does `if (x != 0)` mean?

   a) If x is zero

   b) If x is not zero

   c) If x is positive

d) If x is negative

Answer: b) If x is not zero

10. In nested `if` statements, where is the inner `if` placed?

    a) Before the outer `if`

    b) Inside the outer `if`

    c) After the outer `if`

    d) Parallel to the outer `if`

    Answer: b) Inside the outer if

Answer Questions

Short Answer Questions – Lecture 5 (with Answers)

Q1. What control structure is used for decision-making in C?

A: The `if` statement.

Q2. What are the two forms of the `if` statement shown in this lecture?

A: Simple form (single statement) and block form (multiple statements in braces).

Q3. List all six relational operators in C.

A: `<`, `<=`, `==`, `>=`, `>`, `!=`.

Q4. What is the difference between `=` and `==`?

A: `=` is the assignment operator, `==` is the equality comparison operator.

Q5. What are the two logical operators discussed?

A: `&&` (AND) and `||` (OR).

Q6. What does the `&&` operator do?

A: It returns true only if both conditions are true.

Q7. What does the `||` operator do?

A: It returns true if at least one condition is true.

Q8. What is the purpose of the `else` clause?

A: To execute code when the `if` condition is false.

Q9. What does the `!` operator do?

A: It negates (reverses) a boolean value (NOT operator).

Q10. What is a nested `if` statement?

A: An `if` statement inside another `if` statement.

Q11. In flowcharting, what shape represents a decision?

A: A diamond.

Q12. What shape represents a process/action in a flowchart?

A: A rectangle.

Q13. What shape represents start/stop in a flowchart?

A: An oval.

Q14. What does `if (age > 18 || height > 5)` mean?

A: If age is greater than 18 OR height is greater than 5.

Q15. How would you write "if a is not equal to b" in C?

A: `if (a != b)`

Programming Exercise

Programming Exercises – Lecture 5 (with Solutions)

1. **Simple Age Checker**

```cpp
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;
    if (age >= 18) {
        cout << "You are an adult." << endl;
    }
    return 0;
}
```

2. **Even or Odd Checker**

```cpp
#include <iostream>
using namespace std;
int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    if (num % 2 == 0) {
        cout << "Even" << endl;
    } else {
        cout << "Odd" << endl;
    }
    return 0;
}
```

3. **Largest of Two Numbers**

```cpp
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    if (a > b) {
        cout << a << " is larger" << endl;
```

```
    } else if (b > a) {
        cout << b << " is larger" << endl;
    } else {
        cout << "Both are equal" << endl;
    }
    return 0;
}
```

4. **Grade Calculator**

```cpp
#include <iostream>
using namespace std;
int main() {
    int marks;
    cout << "Enter marks (0-100): ";
    cin >> marks;
    if (marks >= 90) {
        cout << "Grade: A" << endl;
    } else if (marks >= 80) {
        cout << "Grade: B" << endl;
    } else if (marks >= 70) {
        cout << "Grade: C" << endl;
    } else if (marks >= 60) {
        cout << "Grade: D" << endl;
    } else {
        cout << "Grade: F" << endl;
    }
    return 0;
}
```

5. **Login System with Logical Operators**

```cpp
#include <iostream>
#include <string>
using namespace std;
int main() {
    string username, password;
    cout << "Enter username: ";
```

```
    cin >> username;
    cout << "Enter password: ";
    cin >> password;
    if (username == "admin" && password == "1234") {
        cout << "Login successful!" << endl;
    } else {
        cout << "Invalid credentials!" << endl;
    }
    return 0;
}
```

Conceptual Question

Flowchart Practice

1. **Draw a flowchart for: "If student's marks are greater than 50, print 'Pass', otherwise print 'Fail'."**

```text
[Start] → [Input marks] → [marks > 50?] → (Yes) → [Print "Pass"] → [End]
    |
    (No)
    |
    [Print "Fail"] → [End]
```

2. **Draw a flowchart for: "Check if a number is positive, negative, or zero."**

```text
[Start] → [Input num] → [num > 0?] → (Yes) → [Print "Positive"] → [End]
    |
    (No)
    ↓
    [num == 0?] → (Yes) → [Print "Zero"] → [End]
    |
    (No)
    ↓
```

```
    [Print "Negative"] → [End]
```

Conceptual Question:
Explain the importance of proper indentation and use of braces `{}` in `if-else` statements, especially when dealing with nested conditions. What problems can arise from improperly structured code?
A: Proper indentation and braces are crucial for code readability and preventing logical errors. Indentation visually represents the structure, making it easier to understand which statements belong to which condition. Braces explicitly define the scope of each `if` or `else` block.
Problems from improper structure: 1) **Dangling else problem** - an `else` may pair with the wrong `if`, 2) **Incorrect execution** - statements intended to be conditional may execute unconditionally, 3) **Maintenance errors** - adding statements without braces can introduce bugs, 4) **Misunderstanding** - other programmers (or yourself later) may misinterpret the logic.
Best practice: Always use braces `{}` for multi-statement blocks, even if there's only one statement, and maintain consistent indentation.

Conditional Construct – Lecture 6

In the Last Lecture

- `if`
- `if-else`

Loop – Repetition Structure

- Problem: Sum integers from 1 to 10

```cpp
int sum;
```

```
sum = 1 + 2 + 3 + 4 + 5 + ... + 10;
cout << sum;
```

Example Problem

Find the sum of the first 100 integers starting from 1.

`while` Loop Syntax

```cpp
while (logicalExpression) {
    statements;
    // ...
}
```

Example: Sum of First 1000 Integers

```cpp
int sum;
sum = 0;
int number;
number = 1;
while (number <= 1000) {
    sum = sum + number;
    number = number + 1;
}
cout << "The sum of the first 1000 integers starting from 1 is " << sum;
```

Generalized Sum Program with User Input

```cpp
int sum, number, UpperLimit;
sum = 0;
number = 1;
cout << "Please enter the upper limit for which you want the sum ";
cin >> UpperLimit;
while (number <= UpperLimit) {
    sum = sum + number;
    number = number + 1;
}
```

```
cout << "The sum of the first " << UpperLimit << " integers is " << sum;
```

Example: Sum of Even Numbers

```cpp
sum = 0;
number = 1;
cout << "Please enter the upper limit for which you want the sum ";
cin >> UpperLimit;
while (number <= UpperLimit) {
    if (number % 2 == 0) {
        sum = sum + number;
        number = number + 1;
    }
}
cout << "The sum of all even integers between 1 and " << UpperLimit << " is " << sum;
```

Note: The above code has an issue: `number` only increments inside the `if`, which may cause an infinite loop if the condition is false.

Potential Infinite Loop Example

```cpp
int Junk;
Junk = 1;
while (Junk <= UpperLimit) { // Infinite Loop if Junk never changes
    sum = sum + number;
    number = number + 1;
    // Missing: Junk = Junk + 1;
}
```

Flowchart for `while` Loop

```text
[Start] → [Initialize] → [Condition true?] → (Yes) → [Process] → [Update] → [Loop back]
    |
    (No)
    ↓
```

```
    [Exit] ←---+
```

- Entry point for `while` block
- Exit point for `while` block

Factorial Definition

```text
n! = n × (n-1) × (n-2) × ... × 3 × 2 × 1
```

Factorial Program

```cpp
#include <iostream.h>
main() {
    int number;
    int factorial;
    factorial = 1;
    cout << "Enter the number for Factorial: ";
    cin >> number;
    while (number >= 1) {
        factorial = factorial * number;
        number = number - 1;
    }
    cout << "Factorial is " << factorial;
}
```

Property of `while` Statement

- It executes zero or more times (if condition is false initially, it doesn't execute at all).

MCQs

✅ Multiple Choice Questions (MCQs) – Lecture 6 (with Answers)

1. Which loop executes at least once?

   a) `while`

   b) `do-while`

   c) Both

   d) Neither

   Answer: b) do-while

2. What is the sum of first 100 integers?

   a) 5000

   b) 5050

   c) 5100

   d) 4950

   Answer: b) 5050

3. Which part prevents infinite loops?

   a) Initialization

   b) Condition

   c) Update

   d) All of the above

   Answer: c) Update

4. What is `6!` (factorial)?

   a) 720

   b) 120

   c) 5040

   d) 360

   Answer: a) 720

5. Which loop is best when iterations are unknown?

   a) `for`

   b) `while`

   c) `do-while`

   d) All are equal

   Answer: b) while

6. What does `while(1)` create?

   a) No loop

   b) Finite loop

   c) Infinite loop

   d) Error

   Answer: c) Infinite loop

7. In the flowchart, what shape is used for condition?

   a) Rectangle

   b) Diamond

   c) Oval

   d) Circle

   Answer: b) Diamond

8. What is missing in the infinite loop example?

   a) Condition

   b) `Junk++`

   c) sum update

   d) Nothing

   Answer: b) Junk++

9. What is the value of sum after this loop?

```cpp
int sum=0, i=1;
while(i<=3) {
    sum += i;
    i++;
}
```

   a) 0

   b) 3

   c) 6

   d) 10

   Answer: c) 6

10. Which is NOT a loop structure?

    a) `while`

    b) `for`

    c) `if`

    d) `do-while`

    Answer: c) if

Answer Questions

📝 Short Answer Questions – Lecture 6 (with Answers)

Q1. What are the two control structures covered in the previous lecture?

A: `if` and `if-else`.

Q2. What is a loop in programming?

A: A control structure that repeats a block of code multiple times.

Q3. What is the purpose of the `while` loop?

A: To execute a block of code repeatedly while a condition remains true.

Q4. What are the three main parts of a loop?

A: Initialization, condition check, and update (increment/decrement).

Q5. What is the output of the sum of first 1000 integers program?

A: 500500 (sum from 1 to 1000).

Q6. What is the common mistake in the "sum of even numbers" example?

A: The variable `number` only increments inside the `if` statement, which may cause an infinite loop for odd numbers.

Q7. What causes an infinite loop?

A: When the loop condition never becomes false (usually due to missing or incorrect update statement).

Q8. What is the factorial of 5?

A: 120 (5×4×3×2×1).

Q9. How many times does a `while` loop execute if the condition is false initially?

A: Zero times.

Q10. What is the difference between `while` and `do-while` loops?

A: `while` checks condition first (0+ executions), `do-while` executes once first (1+ executions).

Programming Exercise

💻 Programming Exercises – Lecture 6 (with Solutions)

1. **Sum of First N Numbers**

```cpp
#include <iostream>
using namespace std;
int main() {
    int n, sum = 0, i = 1;
    cout << "Enter N: ";
    cin >> n;
    while (i <= n) {
        sum += i;
        i++;
    }
    cout << "Sum = " << sum << endl;
    return 0;
}
```

2. **Factorial Calculator**

```cpp
#include <iostream>
using namespace std;
int main() {
    int n, fact = 1;
    cout << "Enter number: ";
    cin >> n;
    int temp = n;
    while (n > 0) {
        fact *= n;
        n--;
    }
    cout << temp << "! = " << fact << endl;
    return 0;
}
```

3. **Print Numbers 1 to N**

```cpp
#include <iostream>
using namespace std;
int main() {
    int n, i = 1;
    cout << "Enter N: ";
    cin >> n;
    while (i <= n) {
        cout << i << " ";
        i++;
    }
    return 0;
}
```

4. **Sum of Even Numbers (Fixed Version)**

```cpp
#include <iostream>
using namespace std;
int main() {
    int limit, sum = 0, num = 1;
```

```cpp
    cout << "Enter upper limit: ";
    cin >> limit;
    while (num <= limit) {
        if (num % 2 == 0) {
            sum += num;
        }
        num++;  // This was missing in the lecture example!
    }
    cout << "Sum of evens = " << sum << endl;
    return 0;
}
```

5. **Reverse Counting**

```cpp
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter N: ";
    cin >> n;
    while (n > 0) {
        cout << n << " ";
        n--;
    }
    return 0;
}
```

Conceptual Question

📊 Flowchart Practice

1. **Draw a flowchart for the factorial program.**

```text
[Start] → [Input n] → [fact=1] → [n>=1?] → (Yes) → [fact=fact*n] → [n=n-1] → [Loop back]
```

```
                  |                |
                 (No)              |
                  ↓                |
          [Print fact] → [End] ←--+
```

2. **Draw a flowchart for checking if a number is prime.**

```text
[Start] → [Input n] → [i=2, isPrime=1] → [i<=n/2?] → (Yes) → [n%i==0?] → (Yes) → [isPrime=0] → [Break]
                |                 |                |                |
               (No)              (No)             (No)             |
                ↓                 ↓                ↓                |
        [Print result] ←--- [Check isPrime] ←---+
```

🎯 Debugging Exercises (with Answers)

1. **Fix this infinite loop:**

```cpp
int x = 1;
while (x <= 10) {
    cout << x;
    // Missing: x++;
}
```

Answer: Add `x++;` inside the loop.

2. **What's wrong here?**

```cpp
int sum = 0, i = 1;
while (i <= 5)
    sum += i;
    i++;  // Not in loop braces!
```

Answer: Missing braces - `i++` is outside the loop.

3. **Correct this factorial code:**

```cpp
int n = 5, fact = 0;  // Should be 1
```

```
while (n > 0) {
    fact *= n;
    n--;
}
```

Answer: Initialize `fact = 1` instead of 0.

4. **Fix this sum program:**

```cpp
int sum, i = 1;
while (i <= 10) {
    sum += i;  // sum not initialized
    i++;
}
```

Answer: Initialize `sum = 0`.

5. **What will this print?**

```cpp
int i = 10;
while (i > 0) {
    cout << i << " ";
    i -= 2;
}
```

Answer: `10 8 6 4 2`

Conceptual Question:

Explain the concept of an "infinite loop" and why it's considered a programming error. What are the common causes of infinite loops in `while` statements, and how can they be prevented?

A: An **infinite loop** is a loop that never terminates because its condition always evaluates to true. It's a programming error because it causes the program to hang, consume CPU resources indefinitely, and fail to produce the desired output or exit normally.

Common causes in `while` loops: 1) Missing or incorrect update statement (e.g.,

forgetting to increment the counter), 2) Condition that never becomes false (e.g., `while(1)`), 3) Update that doesn't affect the condition (e.g., incrementing wrong variable), 4) Logical error in condition expression.

Prevention: 1) Ensure the loop variable is properly initialized, 2) Verify the update statement modifies the condition variable, 3) Test boundary conditions, 4) Use debugging tools to trace loop execution, 5) Consider adding a safety counter for loops with complex conditions.

Introduction to Programming-Lecture 7

while Loop Review

```cpp
while (condition) {
    statements;
}
```

• Executes zero or more times.
• What if we want the loop to execute at least once?

do-while Loop

• Executes one or more times.

Syntax:

```cpp
do {
    statements;
} while (condition);
```

Example: Guessing Game

```cpp
char c;
int tryNum = 1;
do {
```

```cpp
    cout << "Please enter your guess by pressing a character key from a to z ";
    cin >> c;
    if (c == 'z') {
        cout << "Congratulations! you guessed the right answer";
        tryNum = 6;
    } else {
        tryNum = tryNum + 1;
    }
} while (tryNum <= 5);
```

Flowchart for do-while Loop

```text
[Start] → [Process] → [Condition] → (True) → [Loop back]
                      |           |
                    (False)       |
                      ↓           |
                   [Exit] ←------+
```

while Loop Version of Guessing Game

```cpp
char c;
int tryNum, maxTries;
tryNum = 1;
maxTries = 5;
cout << "Guess the alphabet between a to z ";
cin >> c;
while ((tryNum <= maxTries) && (c != 'z')) {
    cout << "Guess the alphabet between a to z ";
    cin >> c;
    tryNum = tryNum + 1;
}
```

for Loop

Syntax:

```cpp
for (initialization; termination condition; increment) {
    statement(s);
}
```

Example:

```cpp
int counter;
for (counter = 0; counter < 10; counter = counter + 1)
    cout << counter;
```

Output: 0123456789

Example: Multiplication Table for 2

```cpp
#include <iostream.h>
main() {
    int counter;
    for (counter = 1; counter <= 10; counter = counter + 1) {
        cout << "2 x " << counter << " = " << 2 * counter << "\n";
    }
}
```

Output:

```text
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
...
2 x 10 = 20
```

Flowchart for Table Example

```text
[Start] → [counter = 1] → [counter <= 10?] → (Yes) → [Print 2*counter] → [counter = c
ounter + 1] → [Loop back]
                    |                      |
                   (No)                    |
                    ↓                      |
                  [Stop] ←-----------------+
```

Enhanced Table Program (User Input)

```cpp
#include <iostream.h>
main() {
```

```
    int number;
    int maxMultiplier;
    int counter;
    maxMultiplier = 10;
    cout << "Please enter the number for which you wish to construct the table ";
    cin >> number;
    for (counter = 1; counter <= maxMultiplier; counter = counter + 1) {
        cout << number << " x " << counter << " = " << number * counter << "\n";
    }
}
```

Tip: Always think re-use. Don't use explicit constants.

Increment/Decrement Operators

- **Increment:** ++

```cpp
counter++;  // same as counter = counter + 1;
```

- **Decrement:** --

```cpp
counter--;  // same as counter = counter - 1;
```

Compound Assignment Operators

Operator	Example	Equivalent
+=	counter += 3	counter = counter + 3
-=	counter -= 5	counter = counter - 5
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2
%=	x %= 2	x = x % 2

Comments

- Write comments at the top of the program to explain what it does.
- Write meaningful comments.

Today's Lecture Summary

- do-while – Executes at least once.
- for loop – Executes zero or more times.
- Shorthand operators – Incrementing, decrementing.
- Compound assignment operators – +=, -=, *=, /=, %=.

✅ Multiple Choice Questions (MCQs)

Which loop always executes at least once?

a) `while`

b) `do-while`

c) `for`

d) All of the above

Answer: b) `do-while`

1. **What is the output of this code?**

```cpp
for(int i=1; i<=3; i++)
    cout << i;
```

a) `123`
b) `012`
c) `0123`
d) `321`
Answer: a) `123`

2. **Which operator increments a variable by 1?**
a) `+`
b) `+=`
c) `++`
d) `--`
Answer: c) `++`
3. **What does** `x -= 3` **mean?**
a) `x = 3`
b) `x = x - 3`

c) `x = x + 3`
d) `x = 3 - x`
Answer: b) `x = x - 3`

4. **How many times does this loop run?**

```cpp
int i = 5;
do {
    cout << i;
    i--;
} while(i > 0);
```

a) 4
b) 5
c) 6
d) Infinite
Answer: b) 5

5. **Which part of a `for` loop is optional?**
a) Initialization
b) Condition
c) Increment
d) All are optional
Answer: d) All are optional
6. **What is the shorthand for `x = x * 4`?**
a) `x *= 4`
b) `x =* 4`
c) `x ** 4`
d) `x * 4=`
Answer: a) `x *= 4`
7. **Which loop is best for input validation?**
a) `while`
b) `do-while`
c) `for`
d) `if`
Answer: b) `do-while`
8. **What does `i++` return?**
a) The new value of `i`
b) The old value of `i`
c) `i + 1`

d) `1`
Answer: b) The old value of `i` (post-increment)

9. **What is the final value of `count`?**

```cpp
int count = 10;
count /= 2;
```

a) `5`
b) `10`
c) `2`
d) `20`
Answer: a) `5`

📝 Short Answer Questions

1. **What is the main difference between `while` and `do-while` loops?**
   Answer: `while` executes zero or more times; `do-while` executes one or more times.
2. **When would you use a `do-while` loop instead of a `while` loop?**
   Answer: When you need to execute the loop body at least once (e.g., menu-driven programs, input validation).
3. **What are the three parts of a `for` loop header?**
   Answer: Initialization, condition, and increment/decrement.
4. **What does `counter++` do?**
   Answer: Increments `counter` by 1.
5. **What is the shorthand for `x = x + 5`?**
   Answer: `x += 5`
6. **What does the `--` operator do?**
   Answer: Decrements a variable by 1.
7. **What is the purpose of the guessing game example?**
   Answer: To demonstrate a `do-while` loop with user input and conditional exit.
8. **How many times does a `for` loop execute if the condition is false initially?**
   Answer: Zero times.
9. **What is the advantage of using compound assignment operators?**
   Answer: They make code shorter and more readable.
10. **Why should you avoid explicit constants in loops?**
    Answer: To make code more reusable and easier to maintain (use variables instead).

11. **What is the output of the `for` loop example with `counter` from 0 to 9?**
    Answer: `0123456789`
12. **What does `x %= 2` mean?**
    Answer: `x = x % 2` (assigns the remainder of `x / 2` to `x`).
13. **What should you write at the top of a program in comments?**
    Answer: A description of what the program does.
14. **What is the purpose of the `maxMultiplier` variable in the enhanced table example?**
    Answer: To make the program flexible—allows changing the table length easily.
15. **Which loop is best when the number of iterations is known in advance?**
    Answer: `for` loop.

💻 Programming Exercises

1. `do-while`: Number Guessing Game

```cpp
#include <iostream>
using namespace std;
int main() {
    int secret = 42, guess, tries = 0;
    do {
        cout << "Enter guess (1-100): ";
        cin >> guess;
        tries++;
        if (guess > secret) cout << "Too high!\n";
        else if (guess < secret) cout << "Too low!\n";
    } while (guess != secret && tries < 5);
    if (guess == secret)
        cout << "Correct! Tries: " << tries;
    else
        cout << "Out of tries! Number was " << secret;
    return 0;
}
```

2. `for` Loop: Multiplication Table

```cpp
#include <iostream>
using namespace std;
int main() {
    int n, limit = 10;
    cout << "Enter number: ";
    cin >> n;
    for (int i = 1; i <= limit; i++) {
        cout << n << " x " << i << " = " << n * i << endl;
    }
    return 0;
}
```

3. Compound Assignment Operators

```cpp
#include <iostream>
using namespace std;
int main() {
    int x = 10;
    x += 5;   // x = 15
    x -= 3;   // x = 12
    x *= 2;   // x = 24
    x /= 4;   // x = 6
    x %= 4;   // x = 2
    cout << "Final x: " << x << endl;
    return 0;
}
```

4. Sum of First N Natural Numbers (while)

```cpp
#include <iostream>
using namespace std;
int main() {
    int n, sum = 0, i = 1;
    cout << "Enter N: ";
    cin >> n;
    while (i <= n) {
```

```
        sum += i;
        i++;
    }
    cout << "Sum = " << sum << endl;
    return 0;
}
```

5. Factorial Using `for` Loop

```
cpp
#include <iostream>
using namespace std;
int main() {
    int n, fact = 1;
    cout << "Enter number: ";
    cin >> n;
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    cout << n << "! = " << fact << endl;
    return 0;
}
```

6. Print Even Numbers Using `do-while`

```
cpp
#include <iostream>
using namespace std;
int main() {
    int i = 1, limit;
    cout << "Enter limit: ";
    cin >> limit;
    do {
        if (i % 2 == 0)
            cout << i << " ";
        i++;
    } while (i <= limit);
    return 0;
```

```
}
```

7. Reverse Counting with `for`

```cpp
#include <iostream>
using namespace std;
int main() {
    for (int i = 10; i >= 1; i--) {
        cout << i << " ";
    }
    cout << "Blast off!" << endl;
    return 0;
}
```

8. Average Calculator with `do-while`

```cpp
#include <iostream>
using namespace std;
int main() {
    int n, count = 0;
    float num, sum = 0;
    char choice;
    do {
        cout << "Enter number: ";
        cin >> num;
        sum += num;
        count++;
        cout << "Add more? (y/n): ";
        cin >> choice;
    } while (choice == 'y' || choice == 'Y');
    cout << "Average = " << sum / count << endl;
    return 0;
}
```

9. Power Calculation Using `for`

```cpp
#include <iostream>
using namespace std;
int main() {
    int base, exp, result = 1;
    cout << "Enter base and exponent: ";
    cin >> base >> exp;
    for (int i = 1; i <= exp; i++) {
        result *= base;
    }
    cout << base << "^" << exp << " = " << result << endl;
    return 0;
}
```

10. Pattern Printing with Nested `for` Loops

```cpp
#include <iostream>
using namespace std;
int main() {
    int rows;
    cout << "Enter rows: ";
    cin >> rows;
    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Flowchart Practice

1. Draw a flowchart for a `do-while` loop that asks for a password until correct.

```text
[Start] → [Initialize tries=0] → [Input password] → [Check password] → (Wrong & tries
<3) → [Increment tries] → [Loop back]
                                    |                                |
                                 (Correct)                       (Wrong & tries>=
3)
                                    ↓                                ↓
                              [Access granted]                 [Access denied] → [E
nd]
```

2. Draw a flowchart for printing numbers 1 to N using a `for` loop.

```text
[Start] → [Input N] → [i=1] → [i<=N?] → (Yes) → [Print i] → [i++] → [Loop back]
                        |                |
                       (No)              |
                        ↓                |
                      [End] ←------------+
```

🎯 Concept Practice Questions (with Answers)

1. **Convert this `while` loop to a `for` loop:**

```cpp
int i=1;
while(i<=5) {
    cout << i;
    i++;
}
```

Answer:

```cpp
```

```
for(int i=1; i<=5; i++)
    cout << i;
```

2. **What's wrong with this loop?**

```cpp
do {
    cout << "Hello";
} while(5 > 10);
```

Answer: Nothing—it will print "Hello" once because `do-while` executes once before checking the false condition.

3. **Write the shorthand for `x = x % 3`.**
Answer: `x %= 3`
4. **How many times is "Hello" printed?**

```cpp
for(int i=0; i<5; i+=2)
    cout << "Hello";
```

Answer: 3 times (i=0,2,4).

5. **Write a `do-while` loop that asks for a positive number.**
Answer:

```cpp
int num;
do {
    cout << "Enter positive number: ";
    cin >> num;
} while(num <= 0);
```

LECTURE 8: LOOP CONSTRUCTS, DECISION STRUCTURES, CONTROL FLOW

- **Objective:**
  Extract key programming concepts, constructs, and examples from the lecture slides.

- **Categories & Extracted Content:**

1. Loop Constructs

- while
- do while
- for

2. Operators

- Increment / Decrement (++, --)
- Compound Assignment Operators (+=, -=, etc.)

3. Example: Average Calculation (Flawed)

```cpp
do {
    cin >> grade;
    sum += grade;
    students++;
} while (grade >= 0);
average = sum / students;
cout << average;
```

Note: Flaw — division when grade < 0 breaks logic.

4. Decision Structures

- Multi-way if (inefficient)
- if-else ladder
- switch statement

5. Switch Statement Details

```cpp
switch(grade) {
    case 'A': cout << "Excellent"; break;
```

```
    case 'B': cout << "Very Good"; break;
    case 'C': cout << "Good"; break;
    case 'D': cout << "Poor"; break;
    case 'F': cout << "Fail"; break;
    default: cout << "Please Enter Grade from 'A' to 'D' or 'F'";
}
```

Note: break prevents fall-through; default handles invalid cases.

6. Control Flow Keywords

- break — exits loop/switch
- continue — skips to next iteration
- goto — unstructured jump (discouraged)

7. Structured Programming Principles

- Minimize break and continue
- Never use goto
- Single entry, single exit
- Modular design
- Rules for structured flowcharting (replace rectangles with structured constructs)

8. Upcoming Topics (Milestones)

- Data Structures
- Arrays
- Character Strings
- Pointers

Result:

Key concepts extracted: loops, operators, decision-making (if, switch), control flow (break, continue, goto), structured programming guidelines, and future topics.

✅ Multiple Choice Questions (MCQs)

1. Which loop executes its body at least once regardless of the condition?

   a) while

   b) for

   c) do while

   d) if-else

   Key: c) do while – Condition is checked at the end of the loop body.
2. In a switch statement, what happens if a `break` statement is omitted?

   a) Compilation error occurs

   b) Execution "falls through" to the next case

   c) The program terminates

   d) The default case executes

   Key: b) Fall-through to next case – Without break, execution continues to subsequent cases.
3. Which of the following is NOT a compound assignment operator?

   a) `+=`

   b) `-=`

   c) `++`

   d) `*=`

   Key: c) `++` – This is an increment operator, not a compound assignment operator.
4. The `continue` statement in a loop causes:

   a) Immediate termination of the loop

   b) Skipping the rest of the current iteration

   c) Restarting the program

   d) Exiting the program

   Key: b) Skip the current iteration – Jumps to the loop condition/test expression.
5. Which control flow keyword is strongly discouraged in structured programming?

   a) break

   b) continue

   c) goto

d) switch

Key: c) goto – Leads to unstructured "spaghetti code."

6. What is the purpose of the `default` case in a switch statement?

   a) It must always be present

   b) It handles unlisted cases/values

   c) It executes first

   d) It prevents fall-through

   Key: b) It handles unlisted cases – Acts as a catch-all for unmatched values.

7. Which loop is best when the number of iterations is known beforehand?

   a) while

   b) do-while

   c) for

   d) All are equal

   Key: c) for – Designed for known iteration counts with initialization, condition, and increment in one line.

8. If `int x = 5;` and you write `x += 3;`, what is the new value of x?

   a) 3

   b) 8

   c) 15

   d) 5

   Key: b) 8 – `x += 3` is equivalent to `x = x + 3`.

❓ Short Answer Questions

Q1: Differentiate between `break` and `continue`.

A1: `break` immediately exits the current loop or switch statement. `continue` skips the remaining code in the current loop iteration and jumps to the next iteration.

Q2: Write the syntax of a `do-while` loop and give one scenario where it is preferred.
A2:

```cpp
do {
    // statements
} while(condition);
```

Scenario: Menu-driven programs where the menu must be displayed at least once before checking the user's exit choice.

Q3: Explain "fall-through" in a switch statement.
A3: Fall-through occurs when a `case` lacks a `break` statement, causing execution to continue into the next `case` block instead of exiting the switch.

Q4: Name three compound assignment operators.
A4: `+=` (add and assign), `-=` (subtract and assign), `*=` (multiply and assign).

Q5: Why is the `goto` statement discouraged?
A5: It makes code difficult to read, debug, and maintain (spaghetti code). It violates structured programming principles (single entry/single exit).

Q6: Write a switch statement that prints remarks for grades 'A' through 'F'.
A6:

```cpp
switch(grade) {
    case 'A': cout << "Excellent"; break;
    case 'B': cout << "Very Good"; break;
    case 'C': cout << "Good"; break;
    case 'D': cout << "Poor"; break;
    case 'F': cout << "Fail"; break;
    default: cout << "Invalid grade";
}
```

Q7: Fix the flawed average calculation code that incorrectly includes the sentinel value -1.

A7:

```cpp
int sum = 0, students = 0, grade;
do {
    cin >> grade;
    if(grade >= 0) {
        sum += grade;
        students++;
    }
} while(grade >= 0);
if(students > 0) {
    average = sum / students;
    cout << average;
}
```

Q8: What is the purpose of the `default` case in a switch statement?

A8: To handle any input value that does not match any of the explicit `case` labels. It ensures no input is silently ignored.

💻 Programming Exercises

1. **Skip Number with Continue:** Write a program that prints numbers 1 to 10 but skips number 5 using `continue`.

```cpp

for(int i = 1; i <= 10; i++) {
    if(i == 5) continue;
    cout << i << " ";
}
```

2. **If-else to Switch Conversion:** Convert the given if-else ladder to a switch statement.

```cpp
switch(grade) {
    case 'A': cout << "Excellent"; break;
    case 'B': cout << "Very Good"; break;
    case 'C': cout << "Good"; break;
    case 'D': cout << "Poor"; break;
    default: cout << "Fail"; break;
}
```

3. **Correct Sentinel Logic:** Correct the flawed average calculation.

```cpp
int sum=0, count=0, grade;
while(true) {
    cin >> grade;
    if(grade < 0) break;
    sum += grade;
    count++;
}
if(count > 0) cout << "Average: " << (double)sum/count;
```

□ Conceptual/Long Questions

Q1: Compare `while`, `do-while`, and `for` loops. Provide a use case for each.
A1:

- **while:** Condition checked first. Use when iterations are unknown (e.g., reading input until valid).
- **do-while:** Body executes once, then condition checked. Use for menu-driven programs.

- **for:** Initialization,

Lecture 8: Loop Constructs & Decision Structures

Objective:

Extract key programming concepts, constructs, and examples from the lecture slides.

Categories & Extracted Content:

1. **Loop Constructs**
   - while
   - do while
   - for
2. **Operators**
   - Increment / Decrement (++, --)
   - Compound Assignment Operators (+=, -=, etc.)
3. **Example: Average Calculation (Flawed)**

```cpp

do {
    cin >> grade;
    sum += grade;
    students++;
} while (grade >= 0);
average = sum / students;
cout << average;
```

Note: Flaw — division when grade < 0 breaks logic.

4. **Decision Structures**
   - Multi-way if (inefficient)

- if-else ladder
- switch statement

5. **Switch Statement Details**

```cpp
switch(grade) {
    case 'A': cout << "Excellent"; break;
    case 'B': cout << "Very Good"; break;
    case 'C': cout << "Good"; break;
    case 'D': cout << "Poor"; break;
    case 'F': cout << "Fail"; break;
    default: cout << "Please Enter Grade from 'A' to 'D' or 'F'";
}
```

Note: break prevents fall-through; default handles invalid cases.

6. **Control Flow Keywords**
   - break — exits loop/switch
   - continue — skips to next iteration
   - goto — unstructured jump (discouraged)
7. **Structured Programming Principles**
   - Minimize break and continue
   - Never use goto
   - Single entry, single exit
   - Modular design
   - Rules for structured flowcharting (replace rectangles with structured constructs)
8. **Upcoming Topics (Milestones)**
   - Data Structures
   - Arrays
   - Character Strings
   - Pointers

Result:
Key concepts extracted: loops, operators, decision-making (if, switch), control flow (break, continue, goto), structured programming guidelines, and future topics.

✅ Multiple Choice Questions (MCQs)

1. Which loop executes its body at least once regardless of condition?
   a) while
   b) for
   c) do while
   d) if-else

Key: c) do while – Executes body first, then checks condition at end.

2. What happens if you omit `break` in a switch case?
   a) Compilation error
   b) Fall-through to next case
   c) Loop termination
   d) Nothing

Key: b) Fall-through to next case – Without break, execution continues to next case.

3. Which is NOT a compound assignment operator?
   a) +=
   b) -=
   c) ++
   d) *=

Key: c) ++ – This is an increment operator, not compound assignment.

4. What does `continue` do in a loop?
   a) Exits the loop
   b) Skips the current iteration
   c) Restarts the program
   d) Pauses execution

Key: b) Skip the current iteration – Jumps to next iteration.

5. Which keyword is discouraged in structured programming?
   a) break
   b) continue
   c) goto
   d) switch

Key: c) goto – Leads to unstructured "spaghetti code."

6. What is the purpose of `default` in switch statement?
   a) It must always be present
   b) It handles unlisted cases
   c) It executes first
   d) It prevents fall-through

Key: b) It handles unlisted cases – Catches unmatched values.

7. Which loop is best when iterations are known beforehand?
   a) while
   b) do-while
   c) for
   d) All are equal

Key: c) for – Designed for known iterations with init, condition, update.

8. If `int x = 5;` and `x += 3;`, what is new value?
   a) 3
   b) 8

c) 15

d) 5

Key: b) 8 – `x += 3` means `x = x + 3`.

❓ Short Answer Questions

Q1: Differentiate between `break` and `continue`.
A1: `break` exits loop/switch immediately. `continue` skips rest of current iteration and jumps to next iteration.

Q2: Write syntax of `do-while` loop and give one use case.
A2:

```cpp
do {
    // statements
} while(condition);
```

Use case: Menu-driven programs where menu must show at least once.

Q3: Explain "fall-through" in switch statement.
A3: When no `break` after a case, execution continues to next case instead of exiting switch.

Q4: Name three compound assignment operators.
A4: `+=` (add and assign), `-=` (subtract and assign), `*=` (multiply and assign).

Q5: Why is `goto` discouraged?
A5: Makes code hard to read/debug (spaghetti code), violates structured programming principles.

Q6: Write switch statement for grades A-F with remarks.

A6:

```cpp
switch(grade) {
    case 'A': cout << "Excellent"; break;
    case 'B': cout << "Very Good"; break;
    case 'C': cout << "Good"; break;
    case 'D': cout << "Poor"; break;
    case 'F': cout << "Fail"; break;
    default: cout << "Invalid grade";
}
```

Q7: Fix the flawed average calculation code.

A7:

```cpp
int sum=0, students=0, grade;
do {
    cin >> grade;
    if(grade >= 0) {
        sum += grade;
        students++;
    }
} while(grade >= 0);
if(students > 0) {
    average = sum / students;
    cout << average;
}
```

Q8: What is purpose of `default` case in switch?

A8: Handles values that don't match any case, ensuring no silent failure.

💻 Programming Exercises

1. **Skip Number 5:**

```cpp
for(int i=1; i<=10; i++) {
    if(i == 5) continue;
    cout << i << " ";
}
```

2. **If-else to Switch Conversion:**

```cpp
switch(grade) {
    case 'A': cout << "Excellent"; break;
    case 'B': cout << "Very Good"; break;
    case 'C': cout << "Good"; break;
    case 'D': cout << "Poor"; break;
    default: cout << "Fail"; break;
}
```

3. **Correct Average Calculation:**

```cpp
int sum=0, count=0, grade;
while(true) {
    cin >> grade;
    if(grade < 0) break;
    sum += grade;
    count++;
}
if(count > 0) cout << "Average: " << (double)sum/count;
```

□ Conceptual/Long Questions

Q1: Compare `while`, `do-while`, and `for` loops with examples.

A1:

- **while:** Condition checked first. Use when iterations unknown.

```cpp
while(condition) { ... }
```

- **do-while:** Executes once, then checks. Use for menus.

```cpp
do { ... } while(condition);
```

- **for:** Known iterations. Use for counting.

```cpp
for(int i=0; i<n; i++) { ... }
```

Q2: Explain structured programming principles from slides.

A2:

1. Minimize use of `break` (except switch)
2. Minimize `continue`
3. Never use `goto`
4. Single entry, single exit for structures
5. Modular design (functions)

Q3: Write program with `for` loop and `continue` to skip number 5.

A3:

```cpp
for(int i=1; i<=10; i++) {
    if(i == 5) continue;
    cout << i << " ";
}
// Output: 1 2 3 4 6 7 8 9 10
```

Q4: Convert if-else ladder to switch for grades.

A4:

```cpp
switch(grade) {
    case 'A': cout << "Excellent"; break;
    case 'B': cout << "Very Good"; break;
    case 'C': cout << "Good"; break;
    case 'D': cout << "Poor"; break;
    default: cout << "Fail"; break;
}
```

Q5: Identify and fix logical error in flawed average code.

A5: Error: Negative sentinel added to sum and counted.

Fix:

```cpp
do {
    cin >> grade;
    if(grade >= 0) {  // Only add non-negative
        sum += grade;
        students++;
    }
} while(grade >= 0);
```

Q6: Explain "single entry-single exit" principle.

A6: Each control structure should have one entry point and one exit point. Makes code easier to trace and debug.

Q7: Draw flowchart for switch statement handling grades.

A7:

```text
Start
  ↓
Input grade
  ↓
switch(grade)
  ├─ case 'A': → Print "Excellent" → break → End
```

```
├─ case 'B': → Print "Very Good" → break → End
├─ case 'C': → Print "Good" → break → End
├─ case 'D': → Print "Poor" → break → End
├─ case 'F': → Print "Fail" → break → End
└─ default: → Print "Invalid" → End
```

Q8: How does `break` work in loops vs switches?
A8: In loops: terminates loop immediately. In switches: exits switch block, prevents fall-through.

Lecture 9: Functions in C++

Objective:
Extract key concepts, syntax, and principles related to functions in C++ from the lecture slides.

Categories & Extracted Content:

1. **Functions – Definition & Purpose**
   • Functions break complex problems into smaller pieces (top-down structured approach)
   • Functions are used like subtasks when constructing something (e.g., making a stool)
   o Make a seat
   o Make legs
   o Assemble them
2. **Types of Functions**
   • Functions that return a value
   • Functions that do not return a value (void functions)
3. **Function Syntax**

```cpp
return-value-type function-name(argument-list) {
    // declarations and statements
}
```

4. **Function Declaration (Prototype)**

```cpp
return-value-type function-name(argument-type-list);
```

Example:

```cpp
int function-name(int, int, double);
```

5. **Function Definition**

```cpp
int function-name(int i, double j) {
    // body
}
```

6. **Function Call**

```cpp
int x;
x = square(i);
```

7. **Example Functions**
   - `square()` – returns square of an integer
   - `raiseToPow(x, power)` – calculates integer power (x^n)
   - `circleArea(radius)` – calculates area of a circle
8. **Passing Arguments – Call by Value**
   - Arguments are passed by value (copied into function parameters)

9. **Practical Examples**
   - Area of a ring = Area(outer circle) – Area(inner circle)
   - Power function with loop
10. **Good Programming Practices**
    - Each function should be a small, self-contained module
    - Variable and function names should be self-explanatory
    - Always comment your code
    - Functions should solve well-defined problems
11. **Exercises Mentioned**

1. Modify `raiseToPow` to handle negative, zero, and positive powers
2. Add error checking to area of ring function

12. **Recall of Previous Constructs**
    - Decisions
    - Loops
    - Sequences

Result:
Key concepts extracted include: function declaration, definition, and calling; types of functions (value-returning vs void); call by value mechanism; examples of mathematical functions; structured programming with functions (modular, top-down); exercises for extension and error handling.

Takeaways for Exam:
- Understand function syntax and prototype
- Know difference between declaration and definition
- Be able to write simple functions (e.g., power, area)
- Recognize call by value
- Apply good naming and commenting practices

✅ Multiple Choice Questions (MCQs)

1. What is a function prototype?
   a) The body of the function
   b) The declaration of the function before its use
   c) The call to the function
   d) The return statement

Key: b) The declaration of the function before its use – Tells compiler about function signature.

2. Which type of function does NOT return a value?
   a) int
   b) double
   c) void
   d) char

Key: c) void – Specifies no return value.

3. In C++, arguments are passed to functions by default using:
   a) Call by reference
   b) Call by value
   c) Call by address
   d) Call by name

Key: b) Call by value – Copies of arguments are passed to function.

4. What does `return (i * i);` do in a function?
   a) Prints the square of i
   b) Returns the square of i to the calling function
   c) Stores the square in a variable
   d) None of these

Key: b) Returns the square of i to the calling function – Return statement sends value back.

5. The `void` keyword in a function return type indicates:
   a) Function returns an integer
   b) Function returns no value
   c) Function returns a character
   d) Function returns a float

Key: b) Function returns no value – Void means nothing is returned.

6. Which of these is a correct function declaration?
   a) int sum(a, b);
   b) int sum(int, int);
   c) sum(int a, int b);
   d) int sum(int a; int b);

Key: b) int sum(int, int); – Valid prototype with parameter types.

7. What is the purpose of breaking a program into functions?
   a) To make the program longer
   b) To follow top-down structured approach
   c) To avoid using loops
   d) To increase compilation time

Key: b) To follow top-down structured approach – Modular design principle.

8. In the stool-making analogy, "Make legs" and "Make seat" represent:
   a) Variables
   b) Functions
   c) Loops
   d) Comments

Key: b) Functions – Subtasks in modular programming.

❓ Short Answer Questions

Q1: Differentiate between function declaration and function definition.
A1: Declaration (prototype): Tells compiler about function name, return type, and parameters. **Definition:** Contains the actual implementation/code of the function.

Q2: What is "call by value"? Explain with an example.
A2: Call by value passes copies of arguments to function. Changes inside function don't affect original variables.

```cpp
void change(int a) { a = 20; }
int main() {
    int x = 10;
    change(x);  // x remains 10
}
```

Q3: Write a function prototype for a function named `calculateArea` that takes a double radius and returns a double.
A3: `double calculateArea(double radius);`

Q4: Why should functions be "small, self-contained modules"?
A4: Easier to test, debug, reuse, maintain, and understand. Promotes modularity and code organization.

Q5: What is the return type of a function that doesn't return any value?
A5: `void`

Q6: Write a function call for a function `square` that takes an integer and returns its square.
A6: `int result = square(5);`

Q7: What will happen if you forget to put a return statement in a non-void function?
A7: Compiler warning/error; undefined behavior if executed (garbage value returned).

Q8: Explain the stool-making analogy in the context of functions.
A8: Complex task (making stool) is broken into smaller functions: `makeSeat()`, `makeLegs()`, `assembleStool()`. Each function handles one specific subtask.

💻 Programming Exercises

1. **isEven Function:**

```cpp
bool isEven(int num) {
    return (num % 2 == 0);
}
```

2. **maxOfTwo Function:**

```cpp
int maxOfTwo(int a, int b) {
    return (a > b) ? a : b;
}
```

3. **Modified raiseToPow for Negative Exponents:**

```cpp
double raiseToPow(double x, int power) {
    double result = 1.0;
    bool negative = false;

    if (power < 0) {
        negative = true;
```

```
        power = -power;
    }

    for (int i = 1; i <= power; i++) {
        result *= x;
    }

    if (negative) {
        return 1.0 / result;
    }
    return result;
}
```

4. **circleArea Function:**

```cpp
double circleArea(double radius) {
    return 3.1415926 * radius * radius;
}
```

5. **printMenu Function:**

```cpp
int printMenu() {
    int choice;
    cout << "1. Add\n2. Subtract\n3. Multiply\n4. Divide\nEnter choice: ";
    cin >> choice;
    return choice;
}
```

6. **isPrime Function:**

```cpp
bool isPrime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
```

```
}
```

7. **Area of Ring Program:**

```cpp
#include <iostream>
using namespace std;

double circleArea(double radius) {
    return 3.1415926 * radius * radius;
}

double ringArea(double outer, double inner) {
    if (inner >= outer) {
        cout << "Error: Inner radius must be smaller than outer radius\n";
        return 0;
    }
    return circleArea(outer) - circleArea(inner);
}

int main() {
    double r1, r2;
    cout << "Enter outer radius: "; cin >> r1;
    cout << "Enter inner radius: "; cin >> r2;
    cout << "Area of ring: " << ringArea(r1, r2);
    return 0;
}
```

8. **Factorial Function:**

```cpp
int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

□ Conceptual/Long Questions

Q1: Compare value-returning functions and void functions. When would you use each?
A1: Value-returning functions: Return a result to caller (e.g., `calculateArea()`). Use when you need a computed result. **Void functions:** Perform actions but don't return values (e.g., `printMenu()`). Use for operations like display, input, or modifying parameters by reference.

Q2: Explain the concept of modular programming with functions. How does it improve code quality?
A2: Modular programming divides program into independent, reusable functions. Improves: readability, maintainability, testing (unit testing), debugging (isolate issues), team collaboration, code reuse.

Q3: Describe the complete process: function declaration, definition, and call with an example.
A3:

1. **Declaration:** `int multiply(int a, int b);`
2. **Definition:**

```cpp
int multiply(int a, int b) {
    return a * b;
}
```

3. **Call:** `int result = multiply(5, 3);`

Q4: How does call by value protect original data? When might this be a disadvantage?
A4: Protection: Changes inside function only affect copies, not originals. **Disadvantage:** Overhead for large data (structs, arrays); cannot modify originals when needed; extra memory usage.

Q5: What are function signatures? Why must they match between prototype and definition?
A5: Signature: function name + parameter types (excluding return type). Must match for compiler to link function calls correctly.

Q6: Explain the concept of "scope" as it relates to function parameters and local variables.
A6: Parameters and local variables have **function scope** - exist only within their function, destroyed when function returns. Cannot be accessed outside function.

Q7: Write a function that validates input (age between 0-120) and returns a boolean.
A7:

```cpp
bool isValidAge(int age) {
    return (age >= 0 && age <= 120);
}
```

Q8: How would you design a calculator program using functions? List the functions needed.
A8:

1. `add(double a, double b)`
2. `subtract(double a, double b)`
3. `multiply(double a, double b)`
4. `divide(double a, double b)`
5. `getInput()` - gets two numbers
6. `displayMenu()` - shows operations
7. `calculate()` - main logic calling other functions
8. `displayResult(double result)` - shows output

Lecture 10: Header Files, Scope, Reference, Recursion

Objective:

Extract key concepts related to header files, variable scope, function parameter passing (call by value vs reference), and recursion from Lecture 10 slides.

Categories & Extracted Content:

1. **Header Files**
   - Purpose: Store prototypes, constants, and definitions
   - Syntax: `#include <iostream.h>`
   - Using `#define` for constants:

```cpp
#define pi 3.1415926
```

- Benefits: Meaningful names, reusable across programs

2. **Scope of Variables/Identifiers**
   - Scope = Visibility of an identifier
   - Three levels of scope:

1. **File/Global Scope:** Declared outside functions, visible everywhere
2. **Function Scope:** Declared inside a function, visible throughout function
3. **Block Scope:** Declared inside {}, visible only within block
   - Example: `for(int i=0; ...)` → i has block scope
   - Best Practice: Minimize global variables, prefer local variables

3. **Functions – Parameter Passing**
   - **Call by Value:**

   o Passes a copy of the variable

   o Original remains unchanged

o Example: `square(number)` leaves number unchanged

• **Call by Reference:**

o Passes the memory address (`&variable`)

o Original can be modified

o Requires pointers (`*x`)

o Example: `square(&x)` changes x

4. **Pointers in Call by Reference**
   • `&x` → Address of x
   • `*x` → Value at address x
   • Used to receive addresses in functions
5. **Math Functions (math.h)**
   • Include: `#include <math.h>`
   • Functions: `sqrt()`, `log10()`, `pow()`, `sin()`, `cos()`, `tan()`
6. **Recursive Functions**
   • Function that calls itself
   • Base case stops recursion
   • Example: Factorial

```cpp
long factorial(long n) {
    if(n == 1) return n;
    else return n * factorial(n-1);
}
```

• Mathematical basis:

o `n! = n * (n-1)!`

o `0! = 1`

7. **Computer Management Issues**
   • Memory overhead: Extra memory for recursion
   • Stack overhead: Function call stack growth
   • Trade-off: Elegant code vs. efficient code

8. **Exercises**
   • Write recursive Fibonacci series
   • Find power of a number using recursion
9. **Good Programming Practices**
   • Use header files for constants and prototypes
   • Avoid global variables when possible
   • Use meaningful names
   • Understand scope to avoid name conflicts

Result:
Key concepts extracted: header files for organization and constants; variable scope (global, function, block); call by value vs. call by reference; pointers for reference passing; recursion with factorial example; memory/stack overhead considerations; exercises for recursive programming.

Takeaways for Exam:
• Know scope rules and variable visibility
• Understand parameter passing mechanisms
• Write recursive functions (factorial, Fibonacci)
• Use `#define` for constants
• Differentiate between local and global variables
• Recognize pointer syntax in call by reference

✅ Multiple Choice Questions (MCQs)

1. Which scope does a variable declared inside a for loop have?
   a) Global scope
   b) Function scope

c) Block scope

d) File scope

Key: c) Block scope – Limited to the loop's {} block.

2. What does `#define pi 3.1415926` do?

   a) Declares a variable named pi

   b) Creates a constant named pi

   c) Defines a function named pi

   d) Includes a header file

Key: b) Creates a constant named pi – Text substitution macro.

3. In call by value:

   a) Original variable can be modified

   b) Address of variable is passed

   c) Copy of variable is passed

   d) Both a and b

Key: c) Copy of variable is passed – Original remains unchanged.

4. Which operator is used to get the address of a variable?

   a) *

   b) &

   c) #

   d) @

Key: b) & – Address-of operator.

5. What is the base case in the factorial recursive function?

   a) if(n == 0)

   b) if(n == 1)

   c) if(n <= 1)

   d) if(n > 1)

Key: b) if(n == 1) – From slides: `if(n == 1) return n;`

6. Which header file contains mathematical functions?
   a) iostream.h
   b) math.h
   c) stdlib.h
   d) stdio.h

Key: b) math.h – Contains sqrt(), pow(), sin(), cos(), etc.

7. A global variable is declared:
   a) Inside a function
   b) Inside a block
   c) Outside all functions
   d) Inside main()

Key: c) Outside all functions – File/global scope.

8. Recursion has which overhead?
   a) Memory overhead
   b) Stack overhead
   c) Both a and b
   d) No overhead

Key: c) Both a and b – Each recursive call uses stack memory.

❓ Short Answer Questions

Q1: What are the three levels of variable scope in C++?

A1: 1. **Global/File scope** – outside all functions

2. **Function scope** – inside a function

3. **Block scope** – inside {} (e.g., loops, if blocks)

Q2: Differentiate between call by value and call by reference.
A2: Call by value: Copy is passed, original unchanged. **Call by reference:** Address is passed, original can be modified.

Q3: Why should global variables be minimized?
A3: Can cause naming conflicts, harder to debug and maintain, violates modularity, can lead to unintended side effects.

Q4: Write the recursive formula for factorial.
A4: `n! = n × (n-1)!` for n > 1, with base cases: `0! = 1` and `1! = 1`

Q5: What is the purpose of `#include <math.h>`?
A5: Includes mathematical functions like sqrt(), pow(), sin(), cos(), tan(), etc.

Q6: How do you pass a variable by reference to a function?
A6: Use `&` in parameter list: `void func(int &x)` and call `func(num);`

Q7: What happens to a variable with block scope when the block ends?
A7: It is destroyed/no longer accessible.

Q8: Explain the trade-off between elegant and efficient code in recursion.
A8: Elegant: Clean, readable, mathematical (recursion). **Efficient:** Less memory, faster (iteration). Recursion has stack/memory overhead but is often more elegant.

💻 Programming Exercises

1. **Circle area using #define:**

```cpp
#include <iostream>
#define PI 3.1415926
using namespace std;
```

```cpp
int main() {
    double radius;
    cout << "Enter radius: ";
    cin >> radius;
    double area = PI * radius * radius;
    cout << "Area: " << area;
    return 0;
}
```

2. **Swap using call by reference:**

```cpp
void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
```

3. **Recursive Fibonacci:**

```cpp
int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n-1) + fibonacci(n-2);
}
```

4. **Power using recursion:**

```cpp
double power(double x, int n) {
    if (n == 0) return 1;
    if (n > 0) return x * power(x, n-1);
    return 1 / power(x, -n);  // Handle negative exponents
}
```

5. **Variable scope demonstration:**

```cpp
```

```cpp
#include <iostream>
using namespace std;

int global = 10;  // Global scope

void demo() {
    int local = 20;  // Function scope
    {
        int block = 30;  // Block scope
        cout << "Block: " << block << endl;
    }
    // cout << block;  // Error: block out of scope
    cout << "Local: " << local << endl;
    cout << "Global: " << global << endl;
}

int main() {
    demo();
    return 0;
}
```

6. **Math.h functions:**

```cpp
cpp

#include <iostream>
#include <math.h>
using namespace std;

int main() {
    double num = 16.0;
    cout << "Square root: " << sqrt(num) << endl;
    cout << "2^3: " << pow(2, 3) << endl;
    return 0;
}
```

7. **Double array elements by reference:**

```cpp
cpp

void doubleArray(int arr[], int &size) {
```

```
    for (int i = 0; i < size; i++) {
        arr[i] *= 2;
    }
}
```

8. **Sum of digits recursively:**

```cpp
int sumOfDigits(int n) {
    if (n == 0) return 0;
    return (n % 10) + sumOfDigits(n / 10);
}
```

□ Conceptual/Long Questions

Q1: Explain the memory layout for global, local, and block variables with examples.

A1: Global: In data segment, persists whole program. **Local:** On stack, created/destroyed with function. **Block:** On stack, limited to {} scope.

Example:

```cpp
int global = 1;           // Data segment
void func() {
    int local = 2;        // Stack - function scope
    { int block = 3; }    // Stack - block scope, destroyed after }
}
```

Q2: Compare iterative and recursive solutions for factorial. Which is better and why?

A2:

Iterative:

```cpp
int fact(int n) {
    int result = 1;
    for(int i=1; i<=n; i++) result *= i;
```

```
    return result;
}
```

Recursive:

```cpp
int fact(int n) {
    if(n <= 1) return 1;
    return n * fact(n-1);
}
```

Better: Iterative - less memory, no stack overflow risk, faster.

Q3: Write a recursive function to find GCD of two numbers.

A3:

```cpp
int gcd(int a, int b) {
    if(b == 0) return a;
    return gcd(b, a % b);
}
```

Q4: Explain how header files help in code organization and reusability.

A4: Header files (.h) store function prototypes, constants, macros. Allow: code organization, separate compilation, reusability across multiple files, abstraction (hide implementation).

Q5: What happens when a recursive function lacks a base case?

A5: Infinite recursion → stack overflow → program crashes.

Q6: Write a program that demonstrates all three scopes and shows variable visibility.

A6:

```cpp
#include <iostream>
using namespace std;

int global = 100;  // Global
```

```cpp
void test() {
    int functionVar = 200;  // Function scope
    cout << "Global: " << global << endl;
    cout << "Function: " << functionVar << endl;

    {
        int blockVar = 300;  // Block scope
        cout << "Block: " << blockVar << endl;
    }
    // cout << blockVar; // ERROR: not visible
}

int main() {
    test();
    // cout << functionVar; // ERROR: not visible
    return 0;
}
```

Q7: Explain how `#define` differs from `const` for creating constants.

A7: `#define` is preprocessor text substitution (no type checking). `const` creates typed constant (type safe, checked by compiler). Example:

```cpp
#define PI 3.14159  // Text substitution
const double PI = 3.14159;  // Typed constant
```

Q8: Design a program that uses both recursion and iteration to solve the same problem (e.g., Fibonacci).

A8:

```cpp
// Recursive Fibonacci
int fibRecursive(int n) {
    if(n <= 1) return n;
    return fibRecursive(n-1) + fibRecursive(n-2);
}

// Iterative Fibonacci
int fibIterative(int n) {
```

```
    if(n <= 1) return n;
    int a = 0, b = 1, c;
    for(int i=2; i<=n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

Lecture 11: Arrays

Objective:

Extract key concepts related to arrays in C++, including declaration, initialization, usage, copying, and practical examples.

Categories & Extracted Content:

1. **Arrays – Basic Definition**
   - Special data type / data structure
   - Stores identical data types in contiguous memory
   - Three components:

1. Name
2. Data type
3. Size

2. **Array Declaration**

```cpp
arrayType arrayName[numberOfElements];
```

Examples:

```cpp
```

```cpp
int age[10];
int age[10], height[10], names[20];  // Multiple arrays
int i, j, age[10];  // Mixed variables and arrays
```

3. **Array Indexing & Access**
   - Index starts at 0
   - Access using: `arrayName[index]`
   - Example: `age[4]` (5th element)
4. **Array Initialization**

   Methods:

1. Individual assignment in loop:

```cpp
for(i=0; i<10; i++) age[i] = 0;
```

2. At declaration:

```cpp
int age[10] = {0,0,0,0,0,0,0,0,0,0};
int age[10] = {0};  // All elements = 0
int age[] = {1,2,3,4,5,6,7,8,9,10};  // Size auto-detected
```

5. **Working with Arrays – Examples**
   - Example 1: Input 10 ages
   - Example 2: Calculate total age
   - Example 3: Read numbers until -1 (sentinel value)
   - Example 4: Sum of squares of array elements
   - Example 5: Linear search in array
6. **Copying Arrays**
   - Must be same data type and size
   - Cannot copy directly: `b = a;` ❌
   - Must copy element-by-element:

```cpp
```

```
for(i=0; i<10; i++)
    b[i] = a[i];
```

7. **Random Numbers (rand())**
   - Header: `#include <stdlib.h>`
   - Function: `rand()` returns 0 to 32767
   - Modulus for range: `rand() % n` → 0 to n-1
   - Examples:
   o Die roll: `1 + rand() % 6` → 1 to 6
   o Coin toss: `rand() % 2` → 0 or 1
   - Fair distribution expected over many trials
8. **Constants (const)**
   - Syntax: `const int arraySize = 100;`
   - Creates read-only identifier (not a variable)
   - Value cannot be changed
   - Useful for array sizes

Result:

Key concepts extracted: array declaration and initialization; indexing (0-based); array operations (input, output, search, sum); copying arrays element-wise; random number generation with rand(); constants using const; memory layout (contiguous storage).

Takeaways for Exam:

- Know array syntax and indexing rules
- Understand array initialization methods
- Be able to write loops for array processing
- Recognize that arrays cannot be directly copied/assigned
- Use rand() for random number generation
- Apply const for fixed sizes

✅ Multiple Choice Questions (MCQs)

1. What is the index range for an array declared as `int arr[10]`?
   a) 1 to 10
   b) 0 to 9
   c) 0 to 10
   d) 1 to 9

Key: b) 0 to 9 – Array indices start at 0, end at size-1.

2. Which statement about arrays is true?
   a) Arrays can store different data types
   b) Array elements are stored in random memory locations
   c) Arrays have a fixed size once declared
   d) Arrays can be copied using the = operator

Key: c) Arrays have a fixed size once declared – Size determined at declaration, cannot change.

3. How do you initialize all elements of an array to zero?
   a) `int arr[5] = {0};`
   b) `int arr[5] = 0;`
   c) `int arr[5] = {0,0,0,0,0};`
   d) Both a and c

Key: d) Both a and c – Both methods initialize all elements to zero.

4. What does `rand() % 6` generate?
   a) Numbers from 0 to 5
   b) Numbers from 1 to 6
   c) Numbers from 0 to 6
   d) Numbers from 1 to 5

Key: a) Numbers from 0 to 5 – `rand() % n` gives 0 to n-1.

5. Which header file is needed for `rand()`?
   a) iostream.h
   b) math.h
   c) stdlib.h
   d) stdio.h

Key: c) stdlib.h – Contains rand() and srand() functions.

6. What is the purpose of `const int size = 100;`?
   a) Declares a variable that can be changed later
   b) Declares a constant that cannot be modified
   c) Declares an array of size 100
   d) Declares a function

Key: b) Declares a constant that cannot be modified – Read-only identifier.

7. How do you access the 3rd element of array `arr`?
   a) arr(3)
   b) arr[2]
   c) arr[3]
   d) arr{3}

Key: b) arr[2] – Indices start at 0, so 3rd element is index 2.

8. What is the output of: `int arr[] = {1,2,3}; cout << arr[1];`?
   a) 1
   b) 2
   c) 3
   d) Error

Key: b) 2 – arr[1] accesses second element (value 2).

❓ Short Answer Questions

Q1: What are the three essential components of an array declaration?
A1: 1. Data type (e.g., int)
2. Array name (e.g., arr)
3. Size (e.g., [10])

Q2: Why can't we copy arrays using the assignment operator (=)?
A2: Arrays are stored as contiguous memory blocks; the array name is a pointer to the first element. Assignment would only copy addresses, not values.

Q3: Write the code to declare and initialize an array of 5 integers with values 10, 20, 30, 40, 50.
A3: `int arr[5] = {10, 20, 30, 40, 50};`

Q4: What is a sentinel value? Give an example from the slides.
A4: A special value that indicates the end of input/data. Example: -1 in Example 3 stops reading numbers.

Q5: How would you generate a random number between 10 and 20 inclusive?
A5: `int num = 10 + rand() % 11;` // 10 + (0 to 10)

Q6: What does `const` do, and why is it useful for array sizes?
A6: Creates a read-only identifier. Useful for array sizes to prevent accidental changes, improve code readability and safety.

Q7: Explain how array elements are stored in memory.
A7: Elements are stored in contiguous memory locations, one after another.

Q8: Write a loop to find the largest element in an array.
A8:

```cpp
int largest = arr[0];
for(int i=1; i<size; i++) {
```

```
    if(arr[i] > largest) largest = arr[i];
}
```

💻 Programming Exercises

1. **Average of array:**

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[10], sum = 0;
    cout << "Enter 10 numbers: ";
    for(int i=0; i<10; i++) {
        cin >> arr[i];
        sum += arr[i];
    }
    double avg = sum / 10.0;
    cout << "Average: " << avg;
    return 0;
}
```

2. **Reverse array function:**

```cpp
void reverseArray(int arr[], int size) {
    for(int i=0; i<size/2; i++) {
        int temp = arr[i];
        arr[i] = arr[size-1-i];
        arr[size-1-i] = temp;
    }
}
```

3. **Die roll frequency:**

```cpp
#include <iostream>
#include <stdlib.h>
#include <time.h>
using namespace std;

int main() {
    int freq[6] = {0};  // Index 0-5 for faces 1-6
    srand(time(0));  // Seed for randomness

    for(int i=0; i<100; i++) {
        int roll = 1 + rand() % 6;
        freq[roll-1]++;
    }

    for(int i=0; i<6; i++) {
        cout << "Face " << i+1 << ": " << freq[i] << " times\n";
    }
    return 0;
}
```

4. **Linear search function:**

```cpp
int linearSearch(int arr[], int size, int key) {
    for(int i=0; i<size; i++) {
        if(arr[i] == key) return i;  // Return index if found
    }
    return -1;  // Not found
}
```

5. **Copy array:**

```cpp
void copyArray(int source[], int dest[], int size) {
    for(int i=0; i<size; i++) {
        dest[i] = source[i];
    }
```

```
}
```

6. **Smallest in random array:**

```cpp
#include <iostream>
#include <stdlib.h>
#include <time.h>
using namespace std;

int main() {
    int arr[20];
    srand(time(0));

    // Fill with random numbers 1-100
    for(int i=0; i<20; i++) {
        arr[i] = 1 + rand() % 100;
    }

    int smallest = arr[0];
    for(int i=1; i<20; i++) {
        if(arr[i] < smallest) smallest = arr[i];
    }

    cout << "Smallest: " << smallest;
    return 0;
}
```

7. **Count even numbers:**

```cpp
int countEven(int arr[], int size) {
    int count = 0;
    for(int i=0; i<size; i++) {
        if(arr[i] % 2 == 0) count++;
    }
    return count;
}
```

8. **Merge two sorted arrays (Advanced):**

```cpp
void mergeSorted(int a[], int m, int b[], int n, int result[]) {
    int i=0, j=0, k=0;
    while(i < m && j < n) {
        if(a[i] < b[j]) result[k++] = a[i++];
        else result[k++] = b[j++];
    }
    while(i < m) result[k++] = a[i++];
    while(j < n) result[k++] = b[j++];
}
```

□ Conceptual/Long Questions

Q1: Explain the memory representation of arrays with a diagram.

A1: Arrays store elements in contiguous memory:

```text
int arr[5] = {10, 20, 30, 40, 50};
Memory: [10][20][30][40][50]
Address: 1000 1004 1008 1012 1016 (assuming 4-byte ints)
arr[0] at address 1000, arr[1] at 1004, etc.
```

Q2: Compare arrays with regular variables. What are the advantages of arrays?

A2: Regular variables: Single values, separate names. **Arrays:** Multiple values, single name, same type. **Advantages:** Efficient storage, easy looping, organized data, less code.

Q3: Write a complete program that reads n numbers, stores them in an array, and finds their average.

A3:

```cpp
#include <iostream>
```

```cpp
using namespace std;

int main() {
    const int MAX = 100;
    int arr[MAX], n, sum = 0;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter " << n << " numbers: ";
    for(int i=0; i<n; i++) {
        cin >> arr[i];
        sum += arr[i];
    }

    cout << "Average: " << (double)sum / n;
    return 0;
}
```

Q4: Why must array indices start at 0 instead of 1? Explain with memory addressing.

A4: Starting at 0 simplifies memory address calculation:

`address of arr[i] = base_address + (i * element_size)`

If indices started at 1: `address = base_address + ((i-1) * element_size)` - more complex.

Q5: What happens if you access `arr[10]` for an array of size 10?

A5: Array out-of-bounds access. May: 1) Access other variables' memory, 2) Cause segmentation fault, 3) Return garbage value. Undefined behavior.

Q6: Explain the concept of "sentinel-controlled" array input with an example.

A6: Use special value (sentinel) to mark end of input:

```cpp
int arr[100], i = 0, value;
cout << "Enter numbers (-1 to stop): ";
cin >> value;
while(value != -1 && i < 100) {
    arr[i++] = value;
    cin >> value;
```

```
}
```

Q7: How does `rand()` work? Why use `srand(time(0))`?
A7: `rand()` generates pseudo-random numbers. `srand(time(0))` seeds the random number generator with current time, making sequences different each run.

Q8: Design a student marks management system using arrays. What arrays would you need?
A8: Need arrays for:

1. `int rollNumbers[100]` – Student roll numbers
2. `string names[100]` – Student names
3. `float marks[100][5]` – Marks in 5 subjects
4. `float averages[100]` – Average marks
   Would also need functions for input, calculation, display.

Lecture 12: Strings, Searching, 2D Arrays

Objective:
Extract key concepts related to strings (character arrays), array algorithms, multi-dimensional arrays, and function parameter passing with arrays from Lecture 12.

Categories & Extracted Content:

1. **Strings (Character Arrays)**
   • Declaration: `char name[100];`
   • Null character `\0`: Terminates all C strings
   • Escape sequences:
   o `\n` → New line

- o `\t` → Tab
- o `\0` → Null character

2. **Initializing Character Arrays**

```cpp
char name[100] = {'a','b','c','0','1'};
char name[100] = "abc01";  // String literal
char name[] = "Hello World";  // Auto-size
```

3. **Input/Output of Strings**
   - Input: `cin >> name;`
   - Output: `cout << name;`
   - Loop output:

```cpp
for(i=0; i<100; i++) cout << name[i];
```

4. **Comparing Two Arrays**
   - Arrays must be equal in size
   - Compare element-by-element
   - Example code for integer array comparison
5. **Array Algorithms**
   - Exercise: Reverse a name
   - Length determination of character array
   - Sorting: Bubble sort, Quick sort
   - Searching: Linear search vs Binary search
   - Binary search is divide-and-conquer, O(log n)
6. **Swapping Elements**

```cpp
int temp = num[0];
num[0] = num[15];
num[15] = temp;
```

7. **Binary Search Efficiency**
   - For array size 2^n, max n tries needed
   - Example: 1000 numbers → max 10 tries (2^{10} = 1024)
   - Best for ordered arrays
8. **Functions and Arrays**
   - Default: Call by reference for arrays
   - Pass: Array name and size
   - Example:

```cpp
void reverse(char[], int);  // Prototype
reverse(name, 100);          // Call
```

- Note: `f(x[3])` passes a single element by value

9. **Two-Dimensional Arrays**
   - Declaration: `int x[2][3];`
   - Representation: Matrix with rows and columns
   - Input example with nested loops:

```cpp
for(row=0; row<maxRows; row++)
    for(col=0; col<maxCols; col++)
        cin >> matrix[row][col];
```

10. **Three-Dimensional Arrays**
    - Declaration: `int x[][][];`
    - Used for 3D data (e.g., vector products, 3D matrices)
11. **Key Concepts**
    - Brute-force technique for sorting
    - Divide and conquer for binary search
    - Arrays are always passed by reference to functions
    - 2D arrays → matrices; 3D arrays → cubes/higher dimensions

Result:
Key concepts extracted: character arrays as strings (null-terminated); array initialization and I/O; array comparison algorithm; searching algorithms (linear vs binary); sorting introduction; passing arrays to functions (always by reference); multi-dimensional arrays (2D, 3D); swapping and basic array manipulations.

Takeaways for Exam:
• Know string termination with `\0`
• Understand array comparison logic
• Differentiate linear vs binary search
• Recognize that arrays are passed by reference
• Write nested loops for 2D array processing
• Apply swapping in sorting algorithms

✅ Multiple Choice Questions (MCQs)

1. How are strings terminated in C/C++?
   a) With `\n`
   b) With `\0`
   c) With `\t`
   d) With `\\`

Key: b) With `\0` – Null character marks string end.

2. Which search algorithm uses the divide-and-conquer approach?
   a) Linear search
   b) Bubble sort
   c) Binary search
   d) Brute-force search

Key: c) Binary search – Repeatedly divides array in half.

3. How are arrays passed to functions in C++?
   a) Always by value
   b) Always by reference
   c) By value for integers, by reference for arrays
   d) Depends on the function prototype

Key: b) Always by reference – Array name is pointer to first element.

4. What is the maximum number of comparisons needed to search in a sorted array of 1024 elements using binary search?
   a) 10
   b) 1024
   c) 512
   d) 32

Key: a) 10 – 2^{10} = 1024, so max 10 comparisons.

5. Which of the following correctly initializes a character array?
   a) `char name[] = "Hello";`
   b) `char name[5] = "HelloWorld";`
   c) `char name = "Hello";`
   d) `char name[] = {'H','e','l','l','o'};` (no `\0`)

Key: a) `char name[] = "Hello";` – Auto-adds `\0`, correct size.

6. In a 2D array declared as `int arr[3][4]`, how many elements are there?
   a) 3
   b) 4
   c) 7
   d) 12

Key: d) 12 – 3 rows × 4 columns = 12 elements.

7. What is the escape sequence for a tab character?
   a) `\n`

b) `\t`

c) `\0`

d) `\\`

Key: b) `\t` – Horizontal tab.

8. When comparing two arrays for equality, what must be true?

   a) They must have the same name

   b) They must be of the same data type and size

   c) They must be sorted

   d) They must be passed to a function

Key: b) They must be of the same data type and size – Otherwise can't compare element-by-element.

❓ Short Answer Questions

Q1: Why must strings in C/C++ be null-terminated?
A1: The `\0` character marks the end of the string, so functions know where the string ends in memory.

Q2: Differentiate between linear search and binary search.
A2: Linear search: Checks each element sequentially, works on any array, O(n). **Binary search:** Divides sorted array in half each time, requires sorted array, O(log n).

Q3: Write code to swap two elements in an array.
A3:

```cpp
int temp = arr[i];
arr[i] = arr[j];
arr[j] = temp;
```

Q4: How do you pass a 2D array to a function?

A4:

```cpp
void func(int arr[][COLS], int rows);
// OR
void func(int (*arr)[COLS], int rows);
```

Q5: What is the advantage of binary search over linear search?

A5: Much faster for large arrays: O(log n) vs O(n) time complexity.

Q6: Write a statement to declare and initialize a 2D array with 2 rows and 3 columns.

A6: `int matrix[2][3] = {{1,2,3}, {4,5,6}};`

Q7: Why are arrays passed by reference by default?

A7: Array name is a pointer to the first element; passing the entire array by value would be inefficient (copying all elements).

Q8: Write a loop to print a 2D array in matrix form.

A8:

```cpp
for(int i=0; i<rows; i++) {
    for(int j=0; j<cols; j++) {
        cout << arr[i][j] << " ";
    }
    cout << endl;
}
```

💻 Programming Exercises

1. **String length function:**

```cpp
```

```cpp
int stringLength(char str[]) {
    int len = 0;
    while(str[len] != '\0') {
        len++;
    }
    return len;
}
```

2. **Reverse a string:**

```cpp
cpp

void reverseString(char str[]) {
    int len = 0;
    while(str[len] != '\0') len++;

    for(int i=0; i<len/2; i++) {
        char temp = str[i];
        str[i] = str[len-1-i];
        str[len-1-i] = temp;
    }
}
```

3. **Linear search:**

```cpp
cpp

int linearSearch(int arr[], int size, int key) {
    for(int i=0; i<size; i++) {
        if(arr[i] == key) return i;
    }
    return -1;
}
```

4. **Binary search:**

```cpp
cpp

int binarySearch(int arr[], int size, int key) {
    int left = 0, right = size-1;
    while(left <= right) {
        int mid = left + (right-left)/2;
```

```
        if(arr[mid] == key) return mid;
        if(arr[mid] < key) left = mid+1;
        else right = mid-1;
    }
    return -1;
}
```

5. **Transpose of matrix:**

```cpp
void transpose(int mat[][3], int rows, int cols) {
    for(int i=0; i<rows; i++) {
        for(int j=i+1; j<cols; j++) {
            // Swap mat[i][j] with mat[j][i]
            int temp = mat[i][j];
            mat[i][j] = mat[j][i];
            mat[j][i] = temp;
        }
    }
}
```

6. **Compare two arrays:**

```cpp
bool compareArrays(int arr1[], int arr2[], int size) {
    for(int i=0; i<size; i++) {
        if(arr1[i] != arr2[i]) return false;
    }
    return true;
}
```

7. **Bubble sort:**

```cpp
void bubbleSort(int arr[], int size) {
    for(int i=0; i<size-1; i++) {
        for(int j=0; j<size-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                // Swap
```

```
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

8. **3x3 matrix input/output:**

```cpp
#include <iostream>
using namespace std;

int main() {
    int matrix[3][3];

    // Input
    cout << "Enter 9 elements for 3x3 matrix:\n";
    for(int i=0; i<3; i++) {
        for(int j=0; j<3; j++) {
            cin >> matrix[i][j];
        }
    }

    // Output
    cout << "Matrix:\n";
    for(int i=0; i<3; i++) {
        for(int j=0; j<3; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

□ Conceptual/Long Questions

Q1: Explain how strings are stored in memory with the null terminator. Give an example.
A1: Strings stored as character arrays with `\0` at end. Example: `"Hello"` stored as: `['H']['e']['l']['l']['o']['\0']` in memory.

Q2: Compare linear search and binary search with time complexity and use cases.
A2:

- **Linear Search:** O(n), works on any array (sorted/unsorted), simple implementation.
- **Binary Search:** O(log n), requires sorted array, faster for large datasets.
  Use cases: Linear for small/unsorted data; Binary for large sorted data.

Q3: Write a complete program that reads a string, reverses it, and checks if it's a palindrome.
A3:

```cpp
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char str[100];
    cout << "Enter string: ";
    cin.getline(str, 100);

    int len = strlen(str);
    bool palindrome = true;

    for(int i=0; i<len/2; i++) {
        if(str[i] != str[len-1-i]) {
            palindrome = false;
            break;
        }
    }
```

```
    if(palindrome) cout << "Palindrome\n";
    else cout << "Not palindrome\n";

    return 0;
}
```

Q4: Explain why arrays are always passed by reference to functions. What are the implications?

A4: Arrays passed by reference because array name is pointer to first element. **Implications:**

1. Function can modify original array
2. No need to copy entire array (efficient)
3. Must pass size separately (function doesn't know array size)

Q5: Describe the binary search algorithm step-by-step with an example array.

A5:

Array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

Search for 23:

1. left=0, right=9, mid=4, arr[4]=16 < 23 → left=5
2. left=5, right=9, mid=7, arr[7]=56 > 23 → right=6
3. left=5, right=6, mid=5, arr[5]=23 == 23 → found at index 5

Q6: How would you store student marks for multiple subjects using arrays? Design the data structure.

A6: Use 2D array:

```cpp
const int STUDENTS = 50;
const int SUBJECTS = 5;
float marks[STUDENTS][SUBJECTS];
// marks[0][0] - student 0, subject 0
// marks[0][1] - student 0, subject 1, etc.
```

Q7: Write a function that takes a 2D array and returns the sum of all elements.

A7:

```cpp
int sum2DArray(int arr[][COLS], int rows, int cols) {
    int sum = 0;
    for(int i=0; i<rows; i++) {
        for(int j=0; j<cols; j++) {
            sum += arr[i][j];
        }
    }
    return sum;
}
```

Q8: What is a 3D array? Give a practical example where it might be used.

A8: 3D array has three dimensions: `int arr[x][y][z]`.

Example: Store RGB values for images: `int image[HEIGHT][WIDTH][3]` where [3] for Red, Green, Blue channels.

Lecture 13: 2D Arrays, Matrix Operations & Tax Problem

Objective:

Extract key concepts related to 2D array manipulation, matrix operations, transposition, practical problem-solving with arrays, and tax calculation logic from Lecture 13.

Categories & Extracted Content:

1. **2D Array Manipulation Basics**
   - Element addressing: `a[rowIndex][columnIndex]`
   - Declaration with constants:

```cpp
const maxRows = 3;
const maxCols = 3;
int a[maxRows][maxCols];
```

• Nested loops for input/output:

```cpp
for(row=0; row<maxRows; row++) {
    for(col=0; col<maxCols; col++) {
        cin >> a[row][col];
    }
}
```

2. **Formatted Output of Matrices**
   • Using `\t` (tab) for column spacing
   • Using `\n` for new rows
   • Example output loop:

```cpp
for(row=0; row<maxRows; row++) {
    for(col=0; col<maxCols; col++) {
        cout << a[row][col] << '\t';
    }
    cout << '\n';
}
```

3. **Reverse Traversal**
   • Reverse row order:

```cpp
for(row = maxRows-1; row >= 0; row--)
```

• Exercise: Print matrix in reverse column order

4. **Matrix Transposition**
   • Square matrix: rows == columns

• Condition for transpose: `a[i][j] = a[j][i]`

• Swapping mechanism:

```cpp
temp = a[row][col];
a[row][col] = a[col][row];
a[col][row] = temp;
```

5. **Practical Problem: Tax Calculation**

   • Problem: Find employees who get less take-home salary than others with lower initial income due to tax brackets.

   Tax Brackets:

1. 0 – 5,000: No tax
2. 5,001 – 10,000: 5% tax
3. 10,001 – 20,000: 10% tax
4. 20,001+: 15% tax

   Example Calculation:

   • Salary = 10,000 → Tax = 500 → Net = 9,500

   • Salary = 10,001 → Tax = 1,000.1 → Net = 9,000.9

   (Unlucky: Higher gross but lower net than 10,000)

6. **Program Design for Tax Problem**

   • Arrays used:

   o `sal[arraySize][2]` → [gross salary, net salary]

   o `lucky[arraySize]` → flags (0=unlucky, 1=lucky)

   • Modular functions:

1. `getInput()` → Input salaries
2. `calculateSalary()` → Compute net after tax
3. `locateUnluckyIndividual()` → Identify unlucky employees
4. `displayOutput()` → Show results

7. **Code Skeleton**

```cpp
```

```
#include<iostream.h>
void getInput(int[][2], int);
main() {
    const int arraySize = 100;
    int sal[arraySize][2];
    int lucky[arraySize] = {0};
    int numEmps;
    // Input and processing
}
```

8. **Exercise: Identity Matrix Check**
   - Given n x n square matrix
   - Check if it's an identity matrix (1s on diagonal, 0s elsewhere)
9. **Key Concepts**
   - 2D array traversal (forward/reverse)
   - Matrix operations (transposition)
   - Real-world problem modeling with arrays
   - Tax bracket logic and edge cases
   - Modular programming with functions

Result:

Key concepts extracted: 2D array input/output with formatting; matrix transposition algorithm; reverse traversal techniques; practical tax calculation problem; array-based program design; identity matrix verification exercise.

Takeaways for Exam:

- Know how to traverse 2D arrays in different orders
- Understand matrix transposition logic
- Be able to implement tax bracket calculations
- Design modular programs with arrays
- Check for identity matrix conditions
- Use constants for array sizes

✅ Multiple Choice Questions (MCQs)

1. How do you access the element in the 2nd row and 3rd column of array `arr`?
   a) arr[2][3]
   b) arr[1][2]
   c) arr[3][2]
   d) arr[2,3]

Key: b) arr[1][2] – Row indices start at 0, so 2nd row is index 1; 3rd column is index 2.

2. What does `const maxRows = 3;` do?
   a) Declares a variable that can be changed
   b) Declares a constant integer
   c) Declares an array
   d) Declares a function

Key: b) Declares a constant integer – Read-only value for array size.

3. In the tax problem, why might someone with a higher gross salary get lower net salary?
   a) Because of progressive tax brackets
   b) Because of errors in calculation
   c) Because of bonuses
   d) Because of deductions

Key: a) Because of progressive tax brackets – Jumping to higher bracket can reduce net income.

4. Which loop prints a matrix in reverse row order?
   a) `for(row=0; row<maxRows; row++)`
   b) `for(row=maxRows-1; row>=0; row--)`

c) `for(row=maxRows; row>0; row--)`

d) `for(row=0; row<=maxRows; row++)`

Key: b) `for(row=maxRows-1; row>=0; row--)` – Starts from last row to first.

5. What is the transpose of a matrix?

   a) Rows become columns and columns become rows

   b) All elements are reversed

   c) Matrix is sorted

   d) Diagonal elements are swapped

Key: a) Rows become columns and columns become rows – a[i][j] becomes a[j][i].

6. In the tax bracket system, what is the tax for salary = 15,000?

   a) 0%

   b) 5%

   c) 10%

   d) 15%

Key: c) 10% – Falls in 10,001–20,000 bracket (10% tax).

7. Which escape sequence adds a tab between columns when printing a matrix?

   a) `\n`

   b) `\t`

   c) `\0`

   d) `\\`

Key: b) `\t` – Horizontal tab character.

8. An identity matrix has:

   a) 1s on diagonal, 0s elsewhere

   b) 0s on diagonal, 1s elsewhere

   c) All 1s

   d) All 0s

Key: a) 1s on diagonal, 0s elsewhere – Definition of identity matrix.

❓ Short Answer Questions

Q1: Write code to declare a 4x4 2D integer array using constants.

A1:

```cpp
const int ROWS = 4;
const int COLS = 4;
int matrix[ROWS][COLS];
```

Q2: Explain the "unlucky employee" problem in your own words.

A2: Due to progressive tax brackets, an employee with slightly higher gross salary might fall into a higher tax bracket, resulting in a lower net salary than someone with slightly lower gross salary.

Q3: Write a nested loop to print a 2D array with tabs between columns and new lines between rows.

A3:

```cpp
for(int i=0; i<rows; i++) {
    for(int j=0; j<cols; j++) {
        cout << arr[i][j] << '\t';
    }
    cout << '\n';
}
```

Q4: How do you swap two elements in a 2D array?

A4:

```cpp
int temp = arr[r1][c1];
arr[r1][c1] = arr[r2][c2];
```

```
arr[r2][c2] = temp;
```

Q5: What are the four functions mentioned for the tax calculation program?

A5: 1. `getInput()` 2. `calculateSalary()` 3. `locateUnluckyIndividual()` 4. `displayOutput()`

Q6: Write the tax bracket conditions from the slides.

A6:

- 0–5,000: 0% tax
- 5,001–10,000: 5% tax
- 10,001–20,000: 10% tax
- 20,001+: 15% tax

Q7: What is the formula for net salary after tax deduction?

A7: `Net Salary = Gross Salary - (Gross Salary × Tax Rate)`

Q8: How would you check if a matrix is square?

A8:

```
cpp
bool isSquare(int rows, int cols) {
    return rows == cols;
}
```

💻 Programming Exercises

1. **Print matrix in reverse column order:**

```
cpp

#include <iostream>
using namespace std;

int main() {
```

```
    const int ROWS = 3, COLS = 3;
    int mat[ROWS][COLS];

    // Input
    cout << "Enter 3x3 matrix:\n";
    for(int i=0; i<ROWS; i++)
        for(int j=0; j<COLS; j++)
            cin >> mat[i][j];

    // Print in reverse column order
    cout << "Reverse column order:\n";
    for(int i=0; i<ROWS; i++) {
        for(int j=COLS-1; j>=0; j--) {
            cout << mat[i][j] << "\t";
        }
        cout << endl;
    }
    return 0;
}
```

2. **Transpose of square matrix:**

```
cpp

void transpose(int mat[][MAX], int n) {
    for(int i=0; i<n; i++) {
        for(int j=i+1; j<n; j++) {
            // Swap mat[i][j] with mat[j][i]
            int temp = mat[i][j];
            mat[i][j] = mat[j][i];
            mat[j][i] = temp;
        }
    }
}
```

3. **Tax calculation problem:**

```
cpp

#include <iostream>
using namespace std;
```

```cpp
double calculateNet(double gross) {
    double tax = 0;
    if(gross > 20000) tax = gross * 0.15;
    else if(gross > 10000) tax = gross * 0.10;
    else if(gross > 5000) tax = gross * 0.05;
    return gross - tax;
}

int main() {
    const int SIZE = 100;
    double gross[SIZE], net[SIZE];
    int n;

    cout << "Enter number of employees: ";
    cin >> n;

    // Input salaries
    for(int i=0; i<n; i++) {
        cout << "Enter gross salary for employee " << i+1 << ": ";
        cin >> gross[i];
        net[i] = calculateNet(gross[i]);
    }

    // Find unlucky employees
    cout << "\nUnlucky employees (higher gross but lower net than previous):\n";
    for(int i=1; i<n; i++) {
        if(net[i] < net[i-1] && gross[i] > gross[i-1]) {
            cout << "Employee " << i+1 << ": Gross=" << gross[i]
                 << ", Net=" << net[i] << endl;
        }
    }
    return 0;
}
```

4. **Identity matrix check:**

```cpp
bool isIdentity(int mat[][MAX], int n) {
```

```
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if(i == j) {
                if(mat[i][j] != 1) return false;
            } else {
                if(mat[i][j] != 0) return false;
            }
        }
    }
    return true;
}
```

5. **Add two 2x2 matrices:**

```
cpp

void addMatrices(int a[2][2], int b[2][2], int result[2][2]) {
    for(int i=0; i<2; i++) {
        for(int j=0; j<2; j++) {
            result[i][j] = a[i][j] + b[i][j];
        }
    }
}
```

6. **Multiply 3x3 matrices:**

```
cpp

void multiplyMatrices(int a[3][3], int b[3][3], int result[3][3]) {
    for(int i=0; i<3; i++) {
        for(int j=0; j<3; j++) {
            result[i][j] = 0;
            for(int k=0; k<3; k++) {
                result[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}
```

7. **Sum of each row:**

```
cpp
```

```cpp
void rowSums(int mat[][MAX], int rows, int cols, int sums[]) {
    for(int i=0; i<rows; i++) {
        sums[i] = 0;
        for(int j=0; j<cols; j++) {
            sums[i] += mat[i][j];
        }
    }
}
```

8. **Largest element in 2D array:**

```cpp
int findLargest(int mat[][MAX], int rows, int cols) {
    int largest = mat[0][0];
    for(int i=0; i<rows; i++) {
        for(int j=0; j<cols; j++) {
            if(mat[i][j] > largest) {
                largest = mat[i][j];
            }
        }
    }
    return largest;
}
```

□ Conceptual/Long Questions

Q1: Explain different ways to traverse a 2D array (row-major, column-major, reverse).

A1:

- **Row-major:** Outer loop rows, inner loop columns (standard)
- **Column-major:** Outer loop columns, inner loop rows
- **Reverse row:** `for(i=rows-1; i>=0; i--)`
- **Reverse column:** `for(j=cols-1; j>=0; j--)`

Q2: Write a complete program to check if a matrix is symmetric ($A = A^T$).

A2:

```cpp
bool isSymmetric(int mat[][MAX], int n) {
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if(mat[i][j] != mat[j][i]) return false;
        }
    }
    return true;
}
```

Q3: Design the complete tax calculation program with all four functions.

A3: (Full program as shown in programming exercise #3 with modular functions)

Q4: What is the time complexity of matrix transposition? Can it be done in-place?

A4: Time complexity: $O(n^2)$ for $n \times n$ matrix. **In-place:** Yes, by swapping elements across diagonal (only upper or lower triangular portion).

Q5: Explain the edge case in tax brackets where someone becomes "unlucky".

A5: When salary crosses bracket threshold (e.g., 10,000 to 10,001), tax rate jumps from 5% to 10%. Small increase in gross causes large increase in tax, reducing net income.

Q6: How would you modify the tax program to handle multiple tax brackets efficiently?

A6: Use arrays for bracket limits and tax rates:

```cpp
double brackets[] = {5000, 10000, 20000, INFINITY};
double rates[] = {0.00, 0.05, 0.10, 0.15};
// Loop through brackets to find correct rate
```

Q7: Write a function to find the trace (sum of diagonal elements) of a square matrix.

A7:

```cpp
int trace(int mat[][MAX], int n) {
    int sum = 0;
```

```
    for(int i=0; i<n; i++) {
        sum += mat[i][i];
    }
    return sum;
}
```

Q8: Design a student grade management system using 2D arrays for multiple subjects.

A8:

```cpp
const int STUDENTS = 50;
const int SUBJECTS = 5;
float grades[STUDENTS][SUBJECTS];
// Functions needed:
// 1. inputGrades()
// 2. calculateAverages()
// 3. findTopStudent()
// 4. displayReport()
```

Lecture 14: Pointers & Complete Tax Program

Objective:

Extract key concepts related to completing the tax calculation program, pointers, pointer syntax, dereferencing, and bubble sort with pointers from Lecture 14.

Categories & Extracted Content:

1. **Tax Calculation Program Completion**

   `calculateSalary()` Function:

   • Calculates net salary based on tax brackets using nested if-else

   • Tax brackets:

- o ≤ 5000: 0% tax
- o 5001–10000: 5% tax
- o 10001–20000: 10% tax
- o >20000: 15% tax
- • Logic issue: Multiple if-statements without else-if can cause overlap

`locateUnluckyIndividual()` Function:

• Compares each employee with all others

• Condition for unlucky: `grossSalary > sal[j][0] && netSalary < sal[j][1]`

• Flags unlucky employees in `lucky[]` array (1 = unlucky)

`displayOutput()` Function:

• Displays unlucky employees with their gross and net salaries

2. **Pointers**
   - Pointer: Variable that stores memory address
   - Declaration: `int *ptr;`
   - Address operator: `&` (e.g., `ptr = &x;`)
   - Dereference operator: `*` (e.g., `*ptr = value at address`)
3. **Pointer Initialization**
   - `ptr = &var;` → Points to variable
   - `ptr = 0;` or `ptr = NULL;` → Points to nothing (null pointer)
4. **Example with Function Parameter**

```cpp
void funct(int *numEmp) {
    cin >> *numEmp;  // Modifies original variable
}
main() {
    int numEmp;
    funct(&numEmp);  // Pass address
}
```

5. **Multiple Pointer Declaration**
   - `int *ptr1, *ptr2, *ptr3;` → All are pointers
   - `int *ptr, x, a[10];` → Mix of pointer, variable, array
6. **Bubble Sort with Pointers**
   - Swapping logic:

```cpp
temp = x;
x = y;
y = temp;
```

- Using pointers for swap function:

```cpp
swap(int *yptr, int *xptr) {
    // Swap using pointers
}
```

7. **const with Pointers**
   - Constant pointer: `int *const myptr = &x;` → Pointer cannot point elsewhere
   - Pointer to constant: `const int *myptr = &x;` → Cannot modify value via pointer
   - Constant integer: `const int x = 10;` → Value cannot change
8. **Arrays and Pointers**
   - Array name is a pointer to first element
   - `int a[10];` → `a` holds starting address of array
9. **Key Programming Issues**
   - Tax logic flaw: Using separate if-statements instead of else-if may cause incorrect calculations
   - Unlucky detection: Nested loop compares each employee with all others ($O(n^2)$)

Result:

Key concepts extracted: complete tax calculation program with all functions; pointer declaration, initialization, and dereferencing; using pointers as function parameters (call

by reference); bubble sort swap with pointers; const qualifier with pointers; relationship between arrays and pointers; common logic errors in conditionals.

Takeaways for Exam:

• Understand pointer syntax (`*` and `&`)
• Know how to pass pointers to functions
• Differentiate between constant pointer and pointer to constant
• Complete the tax calculation program logic
• Implement bubble sort with pointer parameters
• Recognize array-pointer relationship

✅ Multiple Choice Questions (MCQs)

1. What does `int *ptr;` declare?
   a) An integer variable
   b) A pointer to an integer
   c) An array of integers
   d) A constant integer

Key: b) A pointer to an integer – `*` indicates pointer type.

2. Which operator is used to get the address of a variable?
   a) `*`
   b) `&`
   c) `#`
   d) `@`

Key: b) `&` – Address-of operator.

3. In the tax program, what does `lucky[i] = 1` indicate?
   a) The employee is lucky

b) The employee is unlucky

c) The employee pays no tax

d) The employee's salary is highest

Key: b) The employee is unlucky – Flag set to 1 for unlucky employees.

4. What is the purpose of `*ptr`?

   a) Gets the address of ptr

   b) Gets the value at the address stored in ptr

   c) Declares a pointer

   d) Initializes a pointer

Key: b) Gets the value at the address stored in ptr – Dereference operator.

5. Which is a correct way to declare a null pointer?

   a) `ptr = 0;`

   b) `ptr = NULL;`

   c) `ptr = &null;`

   d) Both a and b

Key: d) Both a and b – Both `0` and `NULL` initialize null pointer.

6. In `const int *ptr`, what is constant?

   a) The pointer itself

   b) The value pointed to

   c) Both pointer and value

   d) Neither

Key: b) The value pointed to – Pointer to constant integer.

7. What is wrong with using multiple if statements instead of else-if in tax calculation?

   a) It's slower

   b) It may cause overlapping conditions

   c) It uses more memory

   d) It requires more variables

Key: b) It may cause overlapping conditions – Multiple ifs can execute multiple blocks.

8. Array name is a pointer to:
   a) The last element
   b) The first element
   c) The entire array
   d) A random element

Key: b) The first element – Array name points to base address.

❓ Short Answer Questions

Q1: Differentiate between `*` and `&` operators.
A1: `&` → Address operator: gets memory address of a variable. `*` → Dereference operator: gets value at a memory address.

Q2: Write the tax bracket conditions as an if-else ladder.
A2:

```cpp
if(salary <= 5000) tax = 0;
else if(salary <= 10000) tax = salary * 0.05;
else if(salary <= 20000) tax = salary * 0.10;
else tax = salary * 0.15;
```

Q3: Why is the nested loop used in `locateUnluckyIndividual()`?
A3: To compare each employee with every other employee to find if someone with higher gross has lower net.

Q4: What does `int *const ptr` mean?
A4: A constant pointer to an integer → pointer cannot point to another address, but the value can be changed.

Q5: Write a swap function using pointers.

A5:

```cpp
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Q6: How do you pass a variable by reference using pointers?

A6:

```cpp
void change(int *ptr) { *ptr = 20; }
int main() {
    int x = 10;
    change(&x);  // x becomes 20
}
```

Q7: What is the time complexity of the unlucky employee search?

A7: $O(n^2)$ because of nested loops comparing all pairs of employees.

Q8: Why might `grossSalary > sal[j][0] && netSalary < sal[j][1]` fail?

A8: If salaries are not sorted, someone with higher gross might have higher net, but we compare with everyone including those with lower gross but even lower net.

💻 Programming Exercises

1. **Complete corrected tax program:**

```cpp

#include <iostream>
using namespace std;
```

```
void getInput(int sal[][2], int numEmps) {
    for(int i=0; i<numEmps; i++) {
        cout << "Enter gross salary for employee " << i+1 << ": ";
        cin >> sal[i][0];
    }
}

void calculateSalary(int sal[][2], int numEmps) {
    for(int i=0; i<numEmps; i++) {
        double gross = sal[i][0];
        double net;

        if(gross <= 5000) net = gross;
        else if(gross <= 10000) net = gross - (gross * 0.05);
        else if(gross <= 20000) net = gross - (gross * 0.10);
        else net = gross - (gross * 0.15);

        sal[i][1] = net;
    }
}

void locateUnluckyIndividual(int sal[][2], int lucky[], int numEmps) {
    for(int i=0; i<numEmps; i++) {
        lucky[i] = 0; // Initialize as not unlucky
        for(int j=0; j<numEmps; j++) {
            if(i != j && sal[i][0] > sal[j][0] && sal[i][1] < sal[j][1]) {
                lucky[i] = 1; // Mark as unlucky
                break;
            }
        }
    }
}

void displayOutput(int sal[][2], int lucky[], int numEmps) {
    cout << "\nUnlucky Employees:\n";
    bool found = false;
    for(int i=0; i<numEmps; i++) {
        if(lucky[i] == 1) {
```

```
            cout << "Employee " << i+1
                 << ": Gross = " << sal[i][0]
                 << ", Net = " << sal[i][1] << endl;
            found = true;
        }
    }
    if(!found) cout << "No unlucky employees found.\n";
}

int main() {
    const int MAX = 100;
    int sal[MAX][2];
    int lucky[MAX] = {0};
    int numEmps;

    cout << "Enter number of employees: ";
    cin >> numEmps;

    getInput(sal, numEmps);
    calculateSalary(sal, numEmps);
    locateUnluckyIndividual(sal, lucky, numEmps);
    displayOutput(sal, lucky, numEmps);

    return 0;
}
```

2. **Swap function with pointers:**

```
cpp

#include <iostream>
using namespace std;

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
```

```cpp
    int x = 5, y = 10;
    cout << "Before: x=" << x << ", y=" << y << endl;
    swap(&x, &y);
    cout << "After: x=" << x << ", y=" << y << endl;
    return 0;
}
```

3. **Pointer arithmetic with arrays:**

```cpp
cpp

#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr; // Points to first element

    for(int i=0; i<5; i++) {
        cout << "arr[" << i << "] = " << *(ptr + i) << endl;
        // OR cout << "arr[" << i << "] = " << ptr[i] << endl;
    }
    return 0;
}
```

4. **Bubble sort with pointers:**

```cpp
cpp

void bubbleSort(int *arr, int size) {
    for(int i=0; i<size-1; i++) {
        for(int j=0; j<size-i-1; j++) {
            if(*(arr+j) > *(arr+j+1)) {
                // Swap
                int temp = *(arr+j);
                *(arr+j) = *(arr+j+1);
                *(arr+j+1) = temp;
            }
        }
    }
}
```

5. **Function taking pointer to array:**

```cpp
void sortArray(int *arr, int size) {
    // Simple selection sort
    for(int i=0; i<size-1; i++) {
        int minIndex = i;
        for(int j=i+1; j<size; j++) {
            if(*(arr+j) < *(arr+minIndex)) {
                minIndex = j;
            }
        }
        // Swap
        int temp = *(arr+i);
        *(arr+i) = *(arr+minIndex);
        *(arr+minIndex) = temp;
    }
}
```

6. **Find max using pointers:**

```cpp
#include <iostream>
using namespace std;

int findMax(int *arr, int size) {
    int max = *arr; // First element
    for(int i=1; i<size; i++) {
        if(*(arr+i) > max) {
            max = *(arr+i);
        }
    }
    return max;
}

int main() {
    int arr[] = {5, 2, 9, 1, 7};
    int size = 5;
    cout << "Maximum: " << findMax(arr, size);
```

```
    return 0;
}
```

7. **Reverse string using pointers:**

```cpp
#include <iostream>
#include <cstring>
using namespace std;

void reverseString(char *str) {
    char *start = str;
    char *end = str + strlen(str) - 1;

    while(start < end) {
        char temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }
}

int main() {
    char str[] = "Hello";
    cout << "Original: " << str << endl;
    reverseString(str);
    cout << "Reversed: " << str << endl;
    return 0;
}
```

8. **const with pointers demo:**

```cpp
#include <iostream>
using namespace std;

int main() {
    int x = 10, y = 20;
```

```
    // 1. Pointer to constant
    const int *ptr1 = &x;
    // *ptr1 = 30; // ERROR: Cannot modify through ptr1
    ptr1 = &y; // OK: Can point to different address

    // 2. Constant pointer
    int *const ptr2 = &x;
    *ptr2 = 30; // OK: Can modify value
    // ptr2 = &y; // ERROR: Cannot point elsewhere

    // 3. Constant pointer to constant
    const int *const ptr3 = &x;
    // *ptr3 = 40; // ERROR
    // ptr3 = &y; // ERROR

    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

□ Conceptual/Long Questions

Q1: Explain the four variations of `const` with pointers with examples.

A1:

1. **Pointer to constant:** `const int *p` – Can't modify value, can change pointer
2. **Constant pointer:** `int *const p` – Can modify value, can't change pointer
3. **Constant pointer to constant:** `const int *const p` – Can't modify value or pointer
4. **Constant variable:** `const int x = 10` – Value can't change

Q2: Trace through the tax calculation for salaries 4,000, 9,000, 10,001, and 25,000. Identify the "unlucky" case.

A2:

- 4,000 → Tax: 0, Net: 4,000
- 9,000 → Tax: 450, Net: 8,550
- 10,001 → Tax: 1,000.1, Net: 9,000.9 (unlucky: lower net than 9,000)
- 25,000 → Tax: 3,750, Net: 21,250

Q3: Write a function that uses pointers to sort an array of structures by a specific field.

A3:

```cpp
struct Student { char name[50]; int marks; };
void sortByMarks(Student *arr, int n) {
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if((arr+j)->marks > (arr+j+1)->marks) {
                Student temp = *(arr+j);
                *(arr+j) = *(arr+j+1);
                *(arr+j+1) = temp;
            }
        }
    }
}
```

Q4: How does passing arrays by reference differ from passing variables by reference?

A4: Arrays: Always passed by reference (array name is pointer). **Variables:** Can be passed by value (default) or reference (using `&`). Arrays don't need `&` in function call.

Q5: Design an improved version of the tax program that avoids $O(n^2)$ complexity.

A5: Sort employees by gross salary first, then check only adjacent pairs:

```cpp
// After sorting by gross salary
for(int i=1; i<n; i++) {
    if(sal[i].net < sal[i-1].net) {
        // Unlucky found
    }
}
// Complexity: O(n log n) for sorting + O(n) for check
```

Q6: Explain how pointers enable dynamic memory allocation. Why is this important?
A6: Pointers store addresses returned by `new` operator. Enables: variable-sized arrays, linked lists, trees. Important for flexible memory usage, runtime allocation.

Q7: Write a function that uses pointers to concatenate two strings without using library functions.
A7:

```cpp
void concat(char *dest, char *src) {
    // Move to end of dest
    while(*dest != '\0') dest++;
    // Copy src to dest
    while(*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0';
}
```

Q8: What are dangling pointers and how can they be avoided?
A8: Dangling pointers: Point to deallocated memory. **Avoid:** Set pointers to `NULL` after `delete`, don't return local variable addresses, use smart pointers.

Lecture 15: Pointers & Arrays Relationship

Objective:
Extract key concepts related to pointer-array relationships, pointer arithmetic, pointer expressions, string manipulation with pointers, and array traversal using pointers from Lecture 15.

Categories & Extracted Content:

1. **Pointers and Arrays**
   - Array name is a pointer to the first element
   - `yptr = y;` is equivalent to `yptr = &y[0];`
   - Declaration:

```cpp
int y[10];
int *yptr;
yptr = y;  // yptr points to y[0]
```

2. **Pointer Arithmetic**

   Increment/Decrement:
   - `yptr++` → Points to next element (increases by sizeof(int))
   - `yptr--` → Points to previous element

Addition/Subtraction:
- `yptr + 3` → Points to 3 elements ahead
- `y2 - y1` → Gives number of elements between pointers (must be same array)

Important Rules:
- `p1 + p2` → ERROR (cannot add two pointers)
- `*yptr++` → Increments pointer, not value (post-increment)
- `(*yptr)++` → Increments value pointed to

3. **Pointer Expressions**
   - `*yptr + 3` → Value at yptr + 3 (doesn't change pointer)
   - `*yptr += 3` → Adds 3 to value at yptr
   - `yptr += 3` → Moves pointer 3 elements ahead
4. **Pointer Comparison**
   - `if(y1 > y2)` → Compares memory addresses
   - `if(*y1 > *y2)` → Compares values at addresses
   - `if(y1 == y2)` → Checks if pointing to same location

5. **Accessing Array Elements with Pointers**
   - `y[5] = *(yptr + 5) = *(y + 5)`
   - `cout << *(yptr + 5);` → Prints value at index 5
6. **Strings as Character Arrays**
   - Strings are null-terminated: `\0`
   - Declaration:

```cpp
char name[20] = "Amir";  // Auto-adds \0
```

- Manual initialization:

```cpp
name[0] = 'A'; name[1] = 'm'; ... name[4] = '\0';
```

7. **String Copy Function with Pointers**

```cpp
void myStringCopy(char *destination, const char *source) {
    while(*source != '\0') {
        *destination++ = *source++;
    }
    *destination = '\0';
}
```

- Uses pointer increment to traverse strings
- `const char *source` → Cannot modify source string

8. **Examples**

   Example 1: Basic pointer-array assignment

   Example 2: Pointer increment and dereferencing

   Example 3: Value vs pointer increment confusion

   Example 4: String copying with pointers
9. **Key Points**
   - Array size for strings: One extra space for `\0`
   - `*ptr++` vs `(*ptr)++` → Critical difference

- Pointer arithmetic accounts for data type size
- Strings can be manipulated efficiently with pointers

Result:
Key concepts extracted: array-pointer equivalence (name as pointer to first element); pointer arithmetic (increment, addition, subtraction); pointer expressions and operator precedence; string manipulation with pointers; string copy implementation using pointer traversal; common pitfalls in pointer arithmetic.

Takeaways for Exam:
- Understand `array[index] = *(array + index)`
- Know pointer arithmetic rules (no pointer+pointer)
- Differentiate between pointer increment and value increment
- Implement string functions with pointers
- Remember null termination for strings
- Trace pointer arithmetic step-by-step

✅ Multiple Choice Questions (MCQs)

1. What is the relationship between an array name and a pointer?
   a) Array name is a pointer to the last element
   b) Array name is a pointer to the first element
   c) Array name is not a pointer
   d) Array name is a pointer to the entire array

Key: b) Array name is a pointer to the first element – Base address of array.

2. What does `yptr++` do if `yptr` is a pointer to integer?
   a) Increments the integer value
   b) Moves to next memory address for an integer

c) Decrements the pointer

d) Adds 1 to the value at yptr

Key: b) Moves to next memory address for an integer – Increases by sizeof(int).

3. Which is equivalent to `y[5]`?

   a) `*(y + 5)`

   b) `*y + 5`

   c) `&y[5]`

   d) `y + 5`

Key: a) `*(y + 5)` – Pointer arithmetic: base address + offset.

4. What is `*yptr++`?

   a) Increments value at yptr, then moves pointer

   b) Moves pointer, then gets value

   c) Increments pointer, then gets value at new location

   d) Gets value at yptr, then increments pointer

Key: d) Gets value at yptr, then increments pointer – Post-increment operator.

5. Which operation is NOT allowed with pointers?

   a) `p1 - p2`

   b) `p1 + p2`

   c) `p1++`

   d) `p1 < p2`

Key: b) `p1 + p2` – Cannot add two pointers (meaningless).

6. How are strings terminated in C/C++?

   a) With `\n`

   b) With `\t`

   c) With `\0`

   d) With `\\`

Key: c) With `\0` – Null character marks string end.

7. What does `const char *source` mean in `myStringCopy`?
   a) Source pointer cannot be changed
   b) Source string cannot be modified
   c) Source must be constant
   d) Both a and b

Key: b) Source string cannot be modified – Pointer to constant char.

8. If `yptr = &y[2]` and `y` is `int[10]`, what is `yptr - &y[0]`?
   a) 2
   b) 8 bytes
   c) 2 elements
   d) Error

Key: c) 2 elements – Pointer subtraction gives element count difference.

❓ Short Answer Questions

Q1: Write three equivalent ways to access the 3rd element of array `arr`.
A1:

```
cpp
arr[2];
*(arr + 2);
*(&arr[0] + 2);
```

Q2: What is the difference between `*ptr++` and `(*ptr)++`?
A2: `*ptr++` → Gets value at ptr, then increments pointer. `(*ptr)++` → Increments value at ptr, pointer unchanged.

Q3: Why must strings have an extra character for `\0`?

A3: `\0` marks string end; without space for it, string may overflow or not terminate properly, causing undefined behavior.

Q4: Write the string copy function using pointers.

A4:

```cpp
void myStringCopy(char *dest, const char *src) {
    while(*src != '\0') {
        *dest++ = *src++;
    }
    *dest = '\0';
}
```

Q5: What is pointer subtraction used for?

A5: Finds number of elements between two pointers in same array.

Q6: How does pointer arithmetic account for data type size?

A6: `ptr++` moves by `sizeof(type)` bytes, not 1 byte. Example: `int*` increments by 4 bytes.

Q7: What is wrong with: `char str[5] = "Hello";`?

A7: No space for `\0`; needs `char str[6]` to store 5 characters + null terminator.

Q8: Explain: `while(*ptr != '\0')` in string traversal.

A8: Loop continues until null terminator is found, traversing string character by character. `*ptr` accesses current character.

💻 Programming Exercises

1. **Array length using pointers:**

```cpp
```

```cpp
#include <iostream>
using namespace std;

int arrayLength(int *arr, int size) {
    int *end = arr + size;
    return end - arr;  // Returns size
}

int main() {
    int arr[5] = {1,2,3,4,5};
    cout << "Length: " << arrayLength(arr, 5);
    return 0;
}
```

2. **strlen() using pointers:**

```cpp
int myStrlen(const char *str) {
    const char *ptr = str;
    while(*ptr != '\0') {
        ptr++;
    }
    return ptr - str;
}
```

3. **String compare using pointers:**

```cpp
int myStrcmp(const char *s1, const char *s2) {
    while(*s1 && *s2 && *s1 == *s2) {
        s1++;
        s2++;
    }
    return *s1 - *s2;
}
```

4. **Reverse array using pointers:**

```cpp
```

```cpp
void reverseArray(int *arr, int size) {
    int *start = arr;
    int *end = arr + size - 1;

    while(start < end) {
        int temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }
}
```

5. **Concatenate strings using pointers:**

```cpp
void myStrcat(char *dest, const char *src) {
    // Move to end of dest
    while(*dest != '\0') dest++;

    // Copy src to dest
    while(*src != '\0') {
        *dest++ = *src++;
    }
    *dest = '\0';
}
```

6. **Sum of array using pointers:**

```cpp
int arraySum(int *arr, int size) {
    int sum = 0;
    int *end = arr + size;

    while(arr < end) {
        sum += *arr;
        arr++;
    }
    return sum;
```

```
}
```

7. **Find character in string:**

```cpp
char* findChar(char *str, char ch) {
    while(*str != '\0') {
        if(*str == ch) return str;
        str++;
    }
    return NULL;  // Not found
}
```

8. **Array copy using pointers:**

```cpp
void arrayCopy(int *dest, const int *src, int size) {
    const int *end = src + size;
    while(src < end) {
        *dest++ = *src++;
    }
}
```

□ Conceptual/Long Questions

Q1: Explain with diagrams how `arr[i]` is equivalent to `*(arr + i)` for an integer array.

A1:

```text
arr[0] = *(arr + 0) = value at base address
arr[1] = *(arr + 1) = value at (base + 4 bytes) for int
arr[2] = *(arr + 2) = value at (base + 8 bytes)
```

Memory: `[10][20][30]` at addresses 1000, 1004, 1008. `arr[1]` accesses 1004 = value 20.

Q2: Write a complete string library implementation using only pointers (strlen, strcpy, strcmp, strcat).

A2:

```cpp
// strlen
int strlen(const char *s) { const char *p=s; while(*p) p++; return p-s; }

// strcpy
char* strcpy(char *d, const char *s) {
    char *r=d; while((*d++=*s++)); return r;
}

// strcmp
int strcmp(const char *s1, const char *s2) {
    while(*s1 && *s1==*s2) { s1++; s2++; }
    return *s1 - *s2;
}

// strcat
char* strcat(char *d, const char *s) {
    char *r=d; while(*d) d++; while((*d++=*s++)); return r;
}
```

Q3: Trace through the execution of `myStringCopy("Hello", dest)` step-by-step.

A3:

1. `*src='H', *dest='H'`, increment both
2. `*src='e', *dest='e'`, increment both
3. `*src='l', *dest='l'`, increment both
4. `*src='l', *dest='l'`, increment both
5. `*src='o', *dest='o'`, increment both
6. `*src='\0', loop ends
7. `*dest='\0'` added

Q4: What are the dangers of pointer arithmetic? How can they be avoided?
A4: Dangers: Out-of-bounds access, dereferencing null/invalid pointers, pointer arithmetic on different arrays. **Avoid:** Check bounds, initialize pointers, use array notation when possible.

Q5: Write a function that uses pointers to count vowels in a string.
A5:

```cpp
int countVowels(const char *str) {
    int count = 0;
    while(*str) {
        char c = tolower(*str);
        if(c=='a'||c=='e'||c=='i'||c=='o'||c=='u') count++;
        str++;
    }
    return count;
}
```

Q6: Explain how `const` correctness helps prevent errors when using pointers with strings.
A6: `const char*` prevents modifying source string, catches accidental writes at compile time. Example: `strcpy` prototype uses `const` for source parameter.

Q7: Design a function that finds the longest string in an array of strings using pointers.
A7:

```cpp
const char* longestString(const char *strings[], int n) {
    const char *longest = strings[0];
    for(int i=1; i<n; i++) {
        if(strlen(strings[i]) > strlen(longest)) {
            longest = strings[i];
        }
    }
    return longest;
}
```

Q8: Compare array subscript notation `arr[i]` with pointer notation `*(arr+i)`. When is each preferred?
A8: arr[i]: More readable, less error-prone, preferred for general use. ***(arr+i):** More flexible for pointer arithmetic, used in low-level code. Most compilers generate same code for both.

Lecture 16: Multi-dimensional Arrays & Pointers

Objective:
Extract key concepts related to multi-dimensional arrays, pointers to pointers, arrays of pointers, command line arguments, and a comprehensive card deck example from Lecture 16.

Categories & Extracted Content:

1. **Multi-dimensional Arrays**
   • Declaration: `char multi[5][10];` → 5 rows, 10 columns
   • Memory layout: Stored sequentially row-by-row
   • Dereferencing:
   o `multi[2][3]` → Element at row 2, column 3
   o `*multi` → First row (pointer to first element)
   o `**multi` → First element of first row
2. **Pointer Expressions with 2D Arrays**
   • `multi + 3` → Pointer to 4th row
   • `*(multi + 3)` → 4th row (pointer to first element of 4th row)
   • `*(multi + 3) + 3` → Pointer to 4th element in 4th row
   • `*(*(multi + 3) + 3)` → Value at row 3, column 3

3. **Traversing 2D Array with Single Pointer**

```cpp
int multi[5][6];
int *ptr;
ptr = *multi; // Points to first element
for(row=0; row<5; row++) {
    for(col=0; col<10; col++) {
        cout << *(ptr++) << " ";
    }
    cout << endl;
}
```

4. **Pointers to Pointers**
   - Array of pointers: `char *myArray[10];` → Array of 10 char pointers
   - Initialization:

```cpp
char *myArray[] = {"Amir", "Jahangir"};
```

- Storing pointers:

```cpp
int *p1, *p2, *p3;
int *b[3];
b[0] = p1; b[1] = p2; b[2] = p3;
```

5. **Command Line Arguments**
   - `argc` → Argument count (number of arguments)
   - `argv` → Argument vector (array of strings)
   - Declaration: `main(int argc, char **argv)` or `main(int argc, char *argv[])`
   - `*argv` → First argument (program name)
6. **Comprehensive Example: Card Deck Program**

   Data Structures:

```cpp
const char *suit[4] = {"Spades", "Hearts", "Diamonds", "Clubs"};
```

```
const char *face[13] = {"Ace","Deuce","Three","Four","Five","Six",
                        "Seven","Eight","Nine","Ten","Jack","Queen","King"};
int deck[4][13] = {0};  // 4 suits × 13 faces
```

Functions:

1. `shuffle()`: Randomly assigns cards 1-52 to deck positions
   o Uses `rand() % 4` for suit, `rand() % 13` for face
   o Ensures no duplicate cards
2. `deal()`: Prints cards in order using face and suit arrays
   o Searches deck for each card number
   o Prints corresponding face and suit
7. **Key Programming Techniques**
   • Array of strings using pointer arrays (efficient memory)
   • 2D array traversal with single pointer (linear access)
   • Random shuffling with collision detection
   • Command line argument handling
   • Pointer arithmetic with multi-dimensional arrays
8. **Memory Representation**

```text

multi[5][10] in memory:
[Row0-Col0][Row0-Col1]...[Row0-Col9][Row1-Col0][Row1-Col1]...
```

Result:

Key concepts extracted: multi-dimensional array memory layout; pointer arithmetic with 2D arrays; arrays of pointers for string arrays; command line argument parsing; complete card deck example with shuffling and dealing; pointer-to-pointer relationships; efficient string storage with pointer arrays.

Takeaways for Exam:

• Understand `array[row][col] = *(*(array + row) + col)`
• Know how to traverse 2D arrays with single pointer
• Use arrays of pointers for string collections

• Handle command line arguments
• Implement shuffling algorithms with collision avoidance
• Apply pointer arithmetic to multi-dimensional arrays

✅ Multiple Choice Questions (MCQs)

1. What does `char multi[5][10];` declare?
   a) A 5×10 character array
   b) A pointer to a 10-element array
   c) An array of 5 pointers
   d) A 10×5 integer array

Key: a) A 5×10 character array – 5 rows, 10 columns.

2. In memory, how is a 2D array stored?
   a) Column-by-column
   b) Row-by-row
   c) Randomly
   d) In separate memory blocks

Key: b) Row-by-row – Row-major order.

3. What is `multi + 3` if `multi` is a 2D array?
   a) Value at row 3
   b) Pointer to row 3
   c) Address of element [0][3]
   d) Address of element [3][0]

Key: b) Pointer to row 3 – Adds 3 rows offset.

4. Which correctly accesses element [2][3] in a 2D array?
   a) `*(multi + 2) + 3`
   b) `*(*(multi + 2) + 3)`
   c) `**(multi + 2) + 3`
   d) `multi[2] + 3`

Key: b) `*(*(multi + 2) + 3)` – Double dereference for 2D.

5. What does `char *myArray[10];` declare?
   a) Array of 10 characters
   b) Array of 10 strings
   c) Array of 10 character pointers
   d) Pointer to 10-character array

Key: c) Array of 10 character pointers – Can point to strings.

6. In `main(int argc, char **argv)`, what is `argc`?
   a) Array of arguments
   b) Number of arguments
   c) First argument
   d) Last argument

Key: b) Number of arguments – Argument count.

7. What is the purpose of the `shuffle()` function in the card example?
   a) Prints the deck
   b) Randomly assigns cards to positions
   c) Sorts the deck
   d) Deals cards to players

Key: b) Randomly assigns cards to positions – Distributes 52 cards randomly.

8. How does the card deck avoid duplicate cards during shuffling?
   a) Uses a set
   b) Checks if position is already occupied

c) Sorts after shuffling

d) Uses unique random seeds

Key: b) Checks if position is already occupied – Tries until finds empty slot.

❓ Short Answer Questions

Q1: Write three equivalent ways to access element `[i][j]` in a 2D array.

A1:

```cpp
arr[i][j];
*(*(arr + i) + j);
*(arr[i] + j);
```

Q2: What is the difference between `char multi[5][10]` and `char *multi[5]`?

A2: `char multi[5][10]` – 5×10 contiguous character block (fixed size). `char *multi[5]` – Array of 5 pointers to strings (variable length strings).

Q3: How are command line arguments passed to `main()`?

A3:

```cpp
main(int argc, char *argv[])
// argc = argument count
// argv = array of argument strings
```

Q4: Write the expression to get a pointer to the 3rd element in the 2nd row of a 2D array.

A4: `*(arr + 2) + 3` or `&arr[2][3]`

Q5: Why use an array of pointers for strings instead of a 2D char array?

A5: Array of pointers: Efficient for variable-length strings, saves memory. 2D char array: Fixed maximum length for all strings, may waste memory.

Q6: Explain the shuffling algorithm in the card example.

A6:

1. Generate random row (0-3) and column (0-12)
2. Check if `deck[row][column]` is 0 (empty)
3. If empty, assign card number (1-52)
4. Repeat for all 52 cards

Q7: What does `argv[0]` contain?

A7: The program name/path.

Q8: How would you traverse a 2D array using a single pointer?

A8:

```
cpp
int *ptr = &arr[0][0];
for(int i=0; i<rows*cols; i++) {
    cout << *(ptr + i) << " ";
}
```

💻 Programming Exercises

1. **Print 3×3 matrix with pointer arithmetic:**

```
cpp

#include <iostream>
using namespace std;

void printMatrix(int (*mat)[3], int rows) {
    for(int i=0; i<rows; i++) {
        for(int j=0; j<3; j++) {
            cout << *(*(mat + i) + j) << " ";
        }
```

```
        cout << endl;
    }
}

int main() {
    int mat[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
    printMatrix(mat, 3);
    return 0;
}
```

2. **Array of 5 strings using pointers:**

```cpp
#include <iostream>
using namespace std;

int main() {
    const char *names[5] = {"Ali", "Babar", "Chaudhry", "Dawood", "Ehsan"};

    for(int i=0; i<5; i++) {
        cout << "Name " << i+1 << ": " << names[i] << endl;
    }
    return 0;
}
```

3. **Command line argument printer:**

```cpp
#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    cout << "Number of arguments: " << argc << endl;

    for(int i=0; i<argc; i++) {
        cout << "Argument " << i << ": " << argv[i] << endl;
    }
    return 0;
}
```

4. **Sum of 2D array using pointers:**

```cpp
int sum2DArray(int *arr, int rows, int cols) {
    int sum = 0;
    int *end = arr + (rows * cols);

    while(arr < end) {
        sum += *arr;
        arr++;
    }
    return sum;
}

// Usage:
int mat[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
int total = sum2DArray(&mat[0][0], 3, 3);
```

5. **Simplified card deck (20 cards):**

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

void shuffle(int deck[][5], int suits, int faces) {
    srand(time(0));
    for(int card=1; card<=20; card++) {
        int row, col;
        do {
            row = rand() % suits;
            col = rand() % faces;
        } while(deck[row][col] != 0);
        deck[row][col] = card;
    }
}

void deal(int deck[][5], const char *suit[], const char *face[]) {
```

```cpp
    for(int card=1; card<=20; card++) {
        for(int i=0; i<4; i++) {
            for(int j=0; j<5; j++) {
                if(deck[i][j] == card) {
                    cout << face[j] << " of " << suit[i] << endl;
                }
            }
        }
    }
}

int main() {
    const char *suit[4] = {"Hearts", "Diamonds", "Clubs", "Spades"};
    const char *face[5] = {"Ace", "King", "Queen", "Jack", "10"};
    int deck[4][5] = {0};

    shuffle(deck, 4, 5);
    deal(deck, suit, face);

    return 0;
}
```

6. **Transpose matrix with pointer arithmetic:**

```cpp
void transpose(int (*mat)[3], int rows) {
    for(int i=0; i<rows; i++) {
        for(int j=i+1; j<3; j++) {
            // Swap mat[i][j] and mat[j][i]
            int temp = *(*(mat + i) + j);
            *(*(mat + i) + j) = *(*(mat + j) + i);
            *(*(mat + j) + i) = temp;
        }
    }
}
```

7. **Sort strings using array of pointers:**

```cpp
```

```cpp
#include <iostream>
#include <cstring>
using namespace std;

void sortStrings(const char *arr[], int n) {
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if(strcmp(arr[j], arr[j+1]) > 0) {
                // Swap pointers
                const char *temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

int main() {
    const char *names[] = {"Zebra", "Apple", "Mango", "Banana"};
    int n = 4;

    sortStrings(names, n);

    for(int i=0; i<n; i++) {
        cout << names[i] << endl;
    }
    return 0;
}
```

8. **Multiply 2×2 matrices using pointers:**

```cpp

void multiplyMatrices(int (*a)[2], int (*b)[2], int (*result)[2]) {
    for(int i=0; i<2; i++) {
        for(int j=0; j<2; j++) {
            *(*(result + i) + j) = 0;
            for(int k=0; k<2; k++) {
                *(*(result + i) + j) += *(*(a + i) + k) * *(*(b + k) + j);
            }
```

```
        }
    }
}
```

□ Conceptual/Long Questions

Q1: Explain row-major and column-major storage with memory diagrams.

A1:

- **Row-major:** Elements stored row by row: `[R0C0][R0C1][R0C2][R1C0][R1C1][R1C2]`
- **Column-major:** Elements stored column by column: `[R0C0][R1C0][R0C1][R1C1][R0C2][R1C2]`
  C++ uses row-major order.

Q2: Write a complete card deck program that shuffles and deals 52 cards.

A2:

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    const char *suit[4] = {"Spades", "Hearts", "Diamonds", "Clubs"};
    const char *face[13] = {"Ace","2","3","4","5","6","7","8","9","10","Jack","Queen"
,"King"};
    int deck[4][13] = {0};

    srand(time(0));
    // Shuffle
    for(int card=1; card<=52; card++) {
        int row, col;
        do {
```

```
            row = rand() % 4;
            col = rand() % 13;
        } while(deck[row][col] != 0);
        deck[row][col] = card;
    }

    // Deal
    for(int card=1; card<=52; card++) {
        for(int i=0; i<4; i++) {
            for(int j=0; j<13; j++) {
                if(deck[i][j] == card) {
                    cout << face[j] << " of " << suit[i] << endl;
                }
            }
        }
    }
    return 0;
}
```

Q3: How does `argv` relate to arrays of pointers? Draw the memory structure.

A3: `argv` is array of pointers: `char *argv[]`. Each `argv[i]` points to a string argument.

```text
argv[0] → "program.exe\0"
argv[1] → "arg1\0"
argv[2] → "arg2\0"
...
argv[argc] → NULL
```

Q4: Compare 2D arrays with arrays of pointers for storing multiple strings.

A4:

2D array: Fixed size strings, contiguous memory, efficient access but wasted space.

Array of pointers: Variable size strings, non-contiguous, flexible but extra pointer overhead.

Q5: Write a function that takes command line arguments and performs calculations (e.g., calculator).

A5:

```cpp
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc, char *argv[]) {
    if(argc != 4) {
        cout << "Usage: " << argv[0] << " num1 operator num2\n";
        return 1;
    }

    double a = atof(argv[1]);
    double b = atof(argv[3]);
    char op = argv[2][0];

    switch(op) {
        case '+': cout << a + b; break;
        case '-': cout << a - b; break;
        case '*': cout << a * b; break;
        case '/': cout << a / b; break;
        default: cout << "Invalid operator";
    }
    return 0;
}
```

Q6: Explain how to dynamically allocate a 2D array using pointers to pointers.

A6:

```cpp
int **arr = new int*[rows];
for(int i=0; i<rows; i++) {
    arr[i] = new int[cols];
}
// Access: arr[i][j]
// Deallocate in reverse
```

Q7: Design a program that reads student names and marks using command line arguments.

A7:

```cpp
int main(int argc, char *argv[]) {
    if(argc < 3 || argc % 2 == 0) {
        cout << "Usage: " << argv[0] << " name1 mark1 name2 mark2 ...\n";
        return 1;
    }

    for(int i=1; i<argc; i+=2) {
        cout << "Student: " << argv[i] << ", Marks: " << argv[i+1] << endl;
    }
    return 0;
}
```

Q8: What are jagged arrays? How are they implemented in C++?

A8: Jagged arrays: 2D arrays with rows of different lengths. **Implementation:** Using arrays of pointers:

```cpp
int *jagged[3];
jagged[0] = new int[2]{1,2};
jagged[1] = new int[3]{3,4,5};
jagged[2] = new int[1]{6};
```

Lecture 17: String & File Handling

Objective:

Extract key concepts related to character/string handling, ctype.h functions, string conversion, string manipulation functions, and file I/O with random access from Lecture 17.

Categories & Extracted Content:

1. **Character Representation & ASCII**
   - ASCII: 1 byte = 8 bits
   - Example: Print ASCII table (0-255 characters)
2. **Character Type Functions (ctype.h)**

   Character Classification:
   - `isdigit(c)` → Check if digit (0-9)
   - `isalpha(c)` → Check if alphabetic (A-Z, a-z)
   - `isalnum(c)` → Check if alphanumeric
   - `isxdigit(c)` → Check if hexadecimal digit
   - `islower(c)` → Check if lowercase
   - `isupper(c)` → Check if uppercase
   - `isspace(c)` → Check if whitespace (space, tab, newline)
   - `iscntrl(c)` → Check if control character
   - `ispunct(c)` → Check if punctuation
   - `isprint(c)` → Check if printable
   - `isgraph(c)` → Check if graphical (printable except space)

Character Conversion:

- `tolower(c)` → Convert to lowercase
- `toupper(c)` → Convert to uppercase

Example Usage:

```cpp
while((c = getchar()) != '\n') {
    if(islower(c)) lc++;
    else if(isupper(c)) uc++;
    else if(isdigit(c)) dig++;
    // etc.
}
```

3. **String Conversion Functions**
   - `double atof(char *str)` → String to double
   - `int atoi(char *str)` → String to integer
   - `long atol(char *str)` → String to long
4. **String Manipulation Functions**

   Copying:
   - `char *strcpy(char *s1, const char *s2)` → Copy s2 to s1
   - `char *strncpy(char *s1, const char *s2, int n)` → Copy first n chars

Concatenation:

- `char *strcat(char *s1, const char *s2)` → Append s2 to s1
- `char *strncat(char *s1, const char *s2, int n)` → Append first n chars

Comparison:

- `int strcmp(const char *s1, const char *s2)` → Compare strings
- `int strncmp(const char *s1, const char *s2, int n)` → Compare first n chars

Length:

- `int strlen(const char *s)` → String length

5. **Search Functions**
   - `char *strstr(const char *s1, const char *s2)` → Find substring s2 in s1
   - `char *strtok(char *s1, const char *s2)` → Tokenize string using delimiters in s2
6. **File I/O Functions**

   Block I/O:
   - `fwrite(void *ptr, size_t size, size_t nmemb, FILE *fp)`

   o Writes nmemb elements of given size

   o Example: `fwrite(earnings, sizeof(double), 10, fp)`
   - `fread(void *ptr, size_t size, size_t nmemb, FILE *fp)`

   o Reads nmemb elements of given size

Random Access:

- `fseek(FILE *fp, long offset, int mode)`

o Modes: `SEEK_SET` (beginning), `SEEK_CUR` (current), `SEEK_END` (end)

- `ftell(FILE *fp)` → Returns current file position
- `rewind(FILE *fp)` → Reset to beginning (`fseek(fp, 0L, SEEK_SET)`)

Example Structure I/O:

```cpp
typedef struct {
    char name[20];
    int serial_code;
    int amount;
    float cost;
} component;

fwrite(&comp_data, sizeof(comp_data), 1, fp);
```

7. **Command Line Arguments**
   - `main(int argc, char **argv)`
   - `argc` → Argument count
   - `argv` → Argument vector (array of strings)
8. **Text vs Binary Mode Storage**
   - Integer storage: Text mode stores ASCII characters, binary mode stores raw bytes
   - Example: Integer 32 in binary vs ASCII representation

Result:

Key concepts extracted: character classification and conversion (ctype.h); string conversion (atoi, atof, atol); string manipulation (copy, concat, compare, length); string search (strstr, strtok); file I/O with structures (fwrite, fread); random file access (fseek, ftell, rewind); command line argument handling; text vs binary file storage.

Takeaways for Exam:

- Know ctype.h functions and their return values
- Use string conversion functions safely
- Implement string operations without library functions
- Understand file position pointers
- Use fseek for random file access

• Handle command line arguments
• Differentiate text vs binary file operations

✅ Multiple Choice Questions (MCQs)

1. Which function checks if a character is a digit?
   a) `isalpha()`
   b) `isdigit()`
   c) `isalnum()`
   d) `isspace()`

Key: b) `isdigit()` – Returns true for '0'-'9'.

2. What does `atoi("123")` return?
   a) 123 (integer)
   b) "123" (string)
   c) 123.0 (float)
   d) Error

Key: a) 123 (integer) – Converts string to integer.

3. Which string function finds a substring?
   a) `strstr()`
   b) `strtok()`
   c) `strcmp()`
   d) `strcat()`

Key: a) `strstr()` – Returns pointer to first occurrence.

4. What does `fseek(fp, 10L, SEEK_SET)` do?
   a) Moves 10 bytes from current position

b) Moves 10 bytes from end of file

c) Moves 10 bytes from beginning of file

d) Reads 10 bytes

Key: c) Moves 10 bytes from beginning of file – SEEK_SET means from start.

5. Which function converts a character to uppercase?

   a) `toupper()`

   b) `toUpper()`

   c) `isupper()`

   d) `upper()`

Key: a) `toupper()` – Converts lowercase to uppercase.

6. What is the purpose of `strtok()`?

   a) Compare strings

   b) Copy strings

   c) Tokenize strings

   d) Concatenate strings

Key: c) Tokenize strings – Splits string by delimiters.

7. Which mode opens a file for binary writing?

   a) "w"

   b) "wb"

   c) "r"

   d) "rb"

Key: b) "wb" – Binary write mode.

8. What does `argv[0]` contain?

   a) First command line argument

   b) Program name

   c) Last argument

   d) Argument count

Key: b) Program name – First element is program name.

❓ Short Answer Questions

Q1: List five functions from ctype.h and their purposes.

A1:

1. `isdigit()` – Checks if digit (0-9)
2. `isalpha()` – Checks if alphabetic (A-Z, a-z)
3. `islower()` – Checks if lowercase
4. `toupper()` – Converts to uppercase
5. `isspace()` – Checks if whitespace

Q2: Write the prototype for `strcpy()` and explain its parameters.

A2: `char *strcpy(char *dest, const char *src);`

- `dest`: Destination string (must have enough space)
- `src`: Source string (won't be modified)

Q3: What is the difference between `strcmp()` and `strncmp()`?

A3: `strcmp()` compares entire strings. `strncmp()` compares first n characters only.

Q4: How do you reset a file pointer to the beginning of a file?

A4:

```
cpp
rewind(fp);
// OR
fseek(fp, 0L, SEEK_SET);
```

Q5: Write code to check if a character is alphanumeric.

A5:

```cpp
if(isalnum(ch)) {
    // Character is letter or digit
}
```

Q6: What is the purpose of `fwrite()` and what are its parameters?

A6:

```cpp
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *stream);
```

- `ptr`: Pointer to data to write
- `size`: Size of each element in bytes
- `count`: Number of elements to write
- `stream`: File pointer

Q7: How does `strtok()` work for tokenizing a string?

A7:

- First call: `strtok(string, delimiters)`
- Subsequent calls: `strtok(NULL, delimiters)`
- Returns tokens one by one, replaces delimiters with `\0`

Q8: What are the three modes for `fseek()`?

A8:

1. `SEEK_SET` – From beginning of file
2. `SEEK_CUR` – From current position
3. `SEEK_END` – From end of file

💻 Programming Exercises

1. **Count vowels, consonants, digits:**

```cpp
#include <iostream>
#include <ctype.h>
using namespace std;

int main() {
    char str[100];
    int vowels = 0, consonants = 0, digits = 0;

    cout << "Enter a string: ";
    cin.getline(str, 100);

    for(int i = 0; str[i] != '\0'; i++) {
        char ch = tolower(str[i]);

        if(isalpha(str[i])) {
            if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
                vowels++;
            else
                consonants++;
        }
        else if(isdigit(str[i])) {
            digits++;
        }
    }

    cout << "Vowels: " << vowels << endl;
    cout << "Consonants: " << consonants << endl;
    cout << "Digits: " << digits << endl;

    return 0;
}
```

2. **Custom strlen():**

```cpp
int myStrlen(const char *str) {
    int length = 0;
```

```
    while(*str != '\0') {
        length++;
        str++;
    }
    return length;
}
```

3. **String to uppercase:**

```cpp
#include <iostream>
#include <ctype.h>
using namespace std;

void toUpperCase(char *str) {
    for(int i = 0; str[i] != '\0'; i++) {
        str[i] = toupper(str[i]);
    }
}

int main() {
    char str[100];
    cout << "Enter string: ";
    cin.getline(str, 100);

    toUpperCase(str);
    cout << "Uppercase: " << str << endl;

    return 0;
}
```

4. **Structure to binary file:**

```cpp
#include <iostream>
#include <fstream>
using namespace std;

struct Student {
```

```
    char name[50];
    int roll;
    float gpa;
};

int main() {
    Student s;

    // Write to file
    ofstream outFile("students.dat", ios::binary);
    cout << "Enter name: "; cin.getline(s.name, 50);
    cout << "Enter roll: "; cin >> s.roll;
    cout << "Enter GPA: "; cin >> s.gpa;

    outFile.write((char*)&s, sizeof(s));
    outFile.close();

    // Read from file
    ifstream inFile("students.dat", ios::binary);
    inFile.read((char*)&s, sizeof(s));

    cout << "\nRead from file:\n";
    cout << "Name: " << s.name << endl;
    cout << "Roll: " << s.roll << endl;
    cout << "GPA: " << s.gpa << endl;

    inFile.close();
    return 0;
}
```

5. **Reverse string:**

```cpp
void reverseString(char *str) {
    int len = 0;
    while(str[len] != '\0') len++;

    for(int i = 0; i < len/2; i++) {
        char temp = str[i];
```

```
        str[i] = str[len-1-i];
        str[len-1-i] = temp;
    }
}
```

6. **Custom strcmp():**

```cpp
int myStrcmp(const char *s1, const char *s2) {
    while(*s1 && *s2 && *s1 == *s2) {
        s1++;
        s2++;
    }
    return *s1 - *s2;
}
```

7. **Find file size:**

```cpp
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    FILE *fp = fopen("example.txt", "rb");

    if(fp == NULL) {
        cout << "File not found!\n";
        return 1;
    }

    fseek(fp, 0L, SEEK_END);
    long size = ftell(fp);

    cout << "File size: " << size << " bytes\n";

    fclose(fp);
    return 0;
}
```

8. **Tokenize sentence:**

```cpp
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char sentence[200];
    char *token;

    cout << "Enter a sentence: ";
    cin.getline(sentence, 200);

    token = strtok(sentence, " ,.!?");

    cout << "Tokens:\n";
    while(token != NULL) {
        cout << token << endl;
        token = strtok(NULL, " ,.!?");
    }

    return 0;
}
```

□ Conceptual/Long Questions

Q1: Compare text mode and binary mode file I/O. When would you use each?

A1:

- **Text mode:** Stores data as ASCII characters, platform-dependent newline conversion. Use for human-readable files (.txt, .csv).
- **Binary mode:** Stores raw bytes, no conversion. Use for non-text data (.dat, images, structures, binary data).

Q2: Write a complete student record system that saves/loads structures to binary files.

A2:

```cpp
struct Student { char name[50]; int roll; float marks; };

void saveStudent() {
    Student s;
    cout << "Enter name: "; cin.getline(s.name, 50);
    cout << "Enter roll: "; cin >> s.roll;
    cout << "Enter marks: "; cin >> s.marks;

    ofstream file("students.dat", ios::binary | ios::app);
    file.write((char*)&s, sizeof(s));
    file.close();
}

void loadStudent(int rollNo) {
    ifstream file("students.dat", ios::binary);
    Student s;
    while(file.read((char*)&s, sizeof(s))) {
        if(s.roll == rollNo) {
            cout << "Name: " << s.name << endl;
            cout << "Marks: " << s.marks << endl;
            file.close();
            return;
        }
    }
    cout << "Student not found!\n";
    file.close();
}
```

Q3: Explain how `strtok()` modifies the original string and why it uses `NULL` in subsequent calls.

A3: `strtok()` replaces delimiters with `\0` to create substrings. Uses `NULL` in subsequent calls because it maintains internal pointer to continue from last token.

Q4: Design a program that reads a CSV file and processes the data using string functions.

A4:

```cpp
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main() {
    ifstream file("data.csv");
    char line[200];

    while(file.getline(line, 200)) {
        char *token = strtok(line, ",");
        while(token != NULL) {
            cout << token << "\t";
            token = strtok(NULL, ",");
        }
        cout << endl;
    }
    file.close();
    return 0;
}
```

Q5: What are the security risks with functions like `strcpy()` and `strcat()`? How can they be avoided?

A5: Risks: Buffer overflow if destination too small. **Avoid:** Use `strncpy()` and `strncat()` with length limits, or use C++ `string` class.

Q6: Write a function that validates a filename extension using string functions.

A6:

```cpp
bool isValidExtension(const char *filename, const char *extension) {
    char *dot = strrchr(filename, '.');
    if(dot == NULL) return false;
```

```
    return (strcmp(dot + 1, extension) == 0);
}
```

Q7: Explain how random access files work with `fseek()`, `ftell()`, and `rewind()`.

A7:

- `fseek()`: Moves file pointer to specific position
- `ftell()`: Returns current position
- `rewind()`: Resets to beginning

  Used for database-like operations, editing specific records.

Q8: Design a text analysis program that counts words, lines, and characters using file I/O and string functions.

A8:

```cpp
void analyzeFile(const char *filename) {
    ifstream file(filename);
    char line[1000];
    int lines = 0, words = 0, chars = 0;

    while(file.getline(line, 1000)) {
        lines++;
        chars += strlen(line);

        char *token = strtok(line, " \t");
        while(token != NULL) {
            words++;
            token = strtok(NULL, " \t");
        }
    }

    cout << "Lines: " << lines << endl;
    cout << "Words: " << words << endl;
    cout << "Characters: " << chars << endl;
    file.close();
}
```

Lecture 18: File Handling in C++

Objective:

Extract key concepts related to file handling in C++, including file types, streams, opening modes, reading/writing operations, and text file processing techniques.

Categories & Extracted Content:

1. **Types of Files**
   - Text Files: Human-readable content (e.g., .txt)
   - Executable Programs: Binary files (.exe)
   - Volatility: Memory is volatile; files provide persistent storage
2. **File Handling Steps**

1. Open the file
2. Read/Write data
3. Close the file
3. **Streams and Header Files**
   - Header: `#include <fstream.h>`
   - Input File Stream: `ifstream` (for reading)
   - Output File Stream: `ofstream` (for writing)
4. **Opening Files**

```cpp

ifstream myFile;
myFile.open("payRoll.txt");
// OR with constructor
ifstream myFile("payRoll.txt");
```

5. **Error Checking**

```cpp
```

```
if(!myFile) {
    cout << "File could not be opened";
}
```

6. **File Opening Modes**

Mode	Purpose	Default For
`ios::in`	Open for reading	`ifstream`
`ios::out`	Open for writing	`ofstream`
`ios::app`	Append to end	-
`ios::ate`	Start at end (random access)	-
`ios::trunc`	Truncate to zero length	-
`ios::nocreate`	Error if file doesn't exist	-
`ios::noreplace`	Error if file exists	-
`ios::binary`	Binary mode	-

7. **Reading from Files**
   - Using `>>` operator: `myFile >> var1 >> var2;`
   - Using `get()`: `myFile.get(ch);` (reads single character)
   - End-of-file check: `while(!myFile.eof())`
8. **Writing to Files**
   - Using `<<` operator: `myFile << data;`
   - Using `put()`: `myFile.put(ch);`
9. **getline() Function**

```cpp
myFile.getline(charArray, maxSize, delimiter);
```

- Example: `myFile.getline(arr, 20, 'W');`
- Reads until delimiter or maxSize reached

10. **Tokenization with strtok()**

```cpp
char *token = strtok(string, delimiter);
// Subsequent calls
```

```
token = strtok(NULL, delimiter);
```

• Example: Parsing "Amir 1000"

```cpp
namePtr = strtok(arr, " ");
salaryPtr = strtok(NULL, " ");
fSalary = atof(salaryPtr);
```

11. **File Processing Example**

```cpp
ifstream inFile("data.txt");
char arr[30];
inFile.getline(arr, 30, '\n');
// Tokenize and convert
```

12. **Closing Files**

```cpp
myFile.close();
```

Result:

Key concepts extracted: file stream types (ifstream, ofstream); file opening modes and their purposes; reading/writing techniques (>>, <<, get(), put()); line-based input with getline(); string tokenization with strtok(); error handling for file operations; text file parsing examples.

Takeaways for Exam:

• Know the difference between ifstream and ofstream
• Understand file opening modes (in, out, app, ate, etc.)
• Use getline() for reading lines with delimiters
• Parse structured text files with strtok()
• Always check if file opened successfully
• Close files after use

✅ Multiple Choice Questions (MCQs)

1. Which header file is needed for file handling in C++?
   a) iostream.h
   b) fstream.h
   c) stdio.h
   d) stdlib.h

Key: b) fstream.h – Contains file stream classes.

2. What does ifstream stand for?
   a) Input file stream
   b) Output file stream
   c) Input/output file stream
   d) File stream

Key: a) Input file stream – For reading from files.

3. Which mode opens a file for appending?
   a) ios::in
   b) ios::out
   c) ios::app
   d) ios::ate

Key: c) ios::app – Append mode adds to end of file.

4. How do you check if a file opened successfully?
   a) if(myFile)
   b) if(!myFile)
   c) if(myFile.eof())
   d) if(myFile.open())

Key: b) if(!myFile) – Checks if file stream is in error state.

5. What does myFile.get(ch) do?
   a) Writes a character to file
   b) Reads a character from file
   c) Checks end of file
   d) Closes the file

Key: b) Reads a character from file – Reads single character.

6. Which function reads a line with a delimiter?
   a) get()
   b) getline()
   c) read()
   d) gets()

Key: b) getline() – Reads until delimiter or newline.

7. What does strtok(NULL, " ") return?
   a) First token
   b) Last token
   c) Next token
   d) Error

Key: c) Next token – NULL tells strtok to continue from last position.

8. Which mode truncates a file to zero length if it exists?
   a) ios::app
   b) ios::trunc
   c) ios::ate
   d) ios::nocreate

Key: b) ios::trunc – Truncates existing file.

❓ Short Answer Questions

Q1: What are the three basic steps in file handling?

A1:

1. Open the file
2. Read/Write data
3. Close the file

Q2: Differentiate between ifstream and ofstream.

A2:

- **ifstream:** Input file stream (for reading from files)
- **ofstream:** Output file stream (for writing to files)

Q3: Write the syntax for opening a file in append mode.

A3:

```
cpp
ofstream myFile("data.txt", ios::app);
```

Q4: How do you read until end of file?

A4:

```
cpp
while(!myFile.eof()) {
    myFile >> data;
}
```

Q5: What is the purpose of getline() function? Write its syntax.

A5: Reads a line of text until delimiter or newline.

```
cpp
myFile.getline(charArray, maxSize, delimiter);
```

Q6: Explain how strtok() works with an example.

A6:

- First call: `strtok(string, delimiters)` returns first token
- Subsequent calls: `strtok(NULL, delimiters)` returns next tokens
- Example: Parsing "John 25 3.8"

```cpp
char *name = strtok(str, " ");
char *age = strtok(NULL, " ");
char *gpa = strtok(NULL, " ");
```

Q7: Why should you close a file after use?

A7:

- Releases system resources
- Ensures all data is written to disk
- Prevents data corruption
- Allows other programs to access the file

Q8: List four file opening modes and their purposes.

A8:

1. `ios::in` – Open for reading
2. `ios::out` – Open for writing (overwrites)
3. `ios::app` – Append to end of file
4. `ios::ate` – Start at end of file

💻 Programming Exercises

1. **Create file and write "Hello World":**

```cpp
#include <fstream.h>
#include <iostream.h>

int main() {
    ofstream outFile("hello.txt");

    if(!outFile) {
        cout << "Error creating file!";
        return 1;
    }

    outFile << "Hello World";
    outFile.close();

    cout << "File created successfully!";
    return 0;
}
```

2. **Count lines in a text file:**

```cpp
#include <fstream.h>
#include <iostream.h>

int main() {
    ifstream inFile("data.txt");
    char line[200];
    int count = 0;

    if(!inFile) {
        cout << "File not found!";
        return 1;
    }

    while(!inFile.eof()) {
        inFile.getline(line, 200);
        count++;
```

```
    }

    inFile.close();
    cout << "Number of lines: " << count;
    return 0;
}
```

3. **Append user input to file:**

```cpp
#include <fstream.h>
#include <iostream.h>

int main() {
    ofstream outFile("notes.txt", ios::app);
    char note[100];

    if(!outFile) {
        cout << "Error opening file!";
        return 1;
    }

    cout << "Enter note (max 100 chars): ";
    cin.getline(note, 100);

    outFile << note << endl;
    outFile.close();

    cout << "Note appended successfully!";
    return 0;
}
```

4. **Copy file contents:**

```cpp
#include <fstream.h>
#include <iostream.h>

int main() {
```

```
    ifstream inFile("source.txt");
    ofstream outFile("destination.txt");
    char ch;

    if(!inFile || !outFile) {
        cout << "Error opening files!";
        return 1;
    }

    while(!inFile.eof()) {
        inFile.get(ch);
        if(!inFile.eof()) {
            outFile.put(ch);
        }
    }

    inFile.close();
    outFile.close();

    cout << "File copied successfully!";
    return 0;
}
```

5. **Read student data from file:**

```cpp
#include <fstream.h>
#include <iostream.h>
#include <string.h>

int main() {
    ifstream inFile("students.txt");
    char line[100];
    char name[50];
    int roll;
    float marks;

    if(!inFile) {
        cout << "File not found!";
```

```
        return 1;
    }

    cout << "Student Data:\n";
    cout << "-------------\n";

    while(!inFile.eof()) {
        inFile.getline(line, 100);
        if(strlen(line) > 0) {
            // Parse line: "Ali,101,85.5"
            char *token = strtok(line, ",");
            strcpy(name, token);

            token = strtok(NULL, ",");
            roll = atoi(token);

            token = strtok(NULL, ",");
            marks = atof(token);

            cout << "Name: " << name << ", Roll: " << roll
                 << ", Marks: " << marks << endl;
        }
    }

    inFile.close();
    return 0;
}
```

6. **Count vowels in text file:**

```
cpp

#include <fstream.h>
#include <iostream.h>
#include <ctype.h>

int main() {
    ifstream inFile("text.txt");
    char ch;
    int vowels = 0;
```

```
    if(!inFile) {
        cout << "File not found!";
        return 1;
    }

    while(!inFile.eof()) {
        inFile.get(ch);
        ch = tolower(ch);

        if(ch == 'a' || ch == 'e' || ch == 'i' ||
           ch == 'o' || ch == 'u') {
            vowels++;
        }
    }

    inFile.close();
    cout << "Number of vowels: " << vowels;
    return 0;
}
```

7. **Parse CSV data with strtok():**

```cpp
#include <fstream.h>
#include <iostream.h>
#include <string.h>

int main() {
    ifstream inFile("data.csv");
    char line[200];

    if(!inFile) {
        cout << "File not found!";
        return 1;
    }

    cout << "CSV Data:\n";
    cout << "---------\n";
```

```
    // Skip header
    inFile.getline(line, 200);

    while(!inFile.eof()) {
        inFile.getline(line, 200);
        if(strlen(line) > 0) {
            char *token = strtok(line, ",");
            int col = 0;

            while(token != NULL) {
                cout << "Column " << ++col << ": " << token << endl;
                token = strtok(NULL, ",");
            }
            cout << "---\n";
        }
    }

    inFile.close();
    return 0;
}
```

8. **Find and replace word in file:**

```
cpp

#include <fstream.h>
#include <iostream.h>
#include <string.h>

int main() {
    ifstream inFile("input.txt");
    ofstream outFile("temp.txt");
    char line[200];
    char oldWord[20], newWord[20];
    char *pos;

    cout << "Enter word to replace: ";
    cin >> oldWord;
    cout << "Enter new word: ";
```

```
    cin >> newWord;

    if(!inFile || !outFile) {
        cout << "Error opening files!";
        return 1;
    }

    while(!inFile.eof()) {
        inFile.getline(line, 200);

        // Simple find and replace
        char result[200] = "";
        char *token = strtok(line, " ");

        while(token != NULL) {
            if(strcmp(token, oldWord) == 0) {
                strcat(result, newWord);
            } else {
                strcat(result, token);
            }
            strcat(result, " ");
            token = strtok(NULL, " ");
        }

        outFile << result << endl;
    }

    inFile.close();
    outFile.close();

    // Rename temp file to original
    remove("input.txt");
    rename("temp.txt", "input.txt");

    cout << "Word replaced successfully!";
    return 0;
}
```

□ Conceptual/Long Questions

Q1: Compare different methods of reading from files (>>, get(), getline()). When would you use each?

A1:

- **>> operator:** Formatted input, skips whitespace. Use for reading numbers, single words.
- **get():** Reads single character (including whitespace). Use for character-by-character processing.
- **getline():** Reads entire line (including spaces). Use for reading sentences, CSV lines.

Q2: Write a complete student record management system using files.

A2:

```cpp
#include <fstream.h>
#include <iostream.h>
#include <string.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

void addStudent() {
    ofstream file("students.dat", ios::binary | ios::app);
    Student s;

    cout << "Enter name: "; cin.getline(s.name, 50);
    cout << "Enter roll: "; cin >> s.roll;
    cout << "Enter marks: "; cin >> s.marks;
    cin.ignore(); // Clear newline
```

```
    file.write((char*)&s, sizeof(s));
    file.close();
}

void searchStudent(int rollNo) {
    ifstream file("students.dat", ios::binary);
    Student s;
    bool found = false;

    while(file.read((char*)&s, sizeof(s))) {
        if(s.roll == rollNo) {
            cout << "Name: " << s.name << endl;
            cout << "Marks: " << s.marks << endl;
            found = true;
            break;
        }
    }

    if(!found) cout << "Student not found!\n";
    file.close();
}

void displayAll() {
    ifstream file("students.dat", ios::binary);
    Student s;

    cout << "All Students:\n";
    while(file.read((char*)&s, sizeof(s))) {
        cout << "Roll: " << s.roll << ", Name: " << s.name
             << ", Marks: " << s.marks << endl;
    }
    file.close();
}
```

Q3: Explain the different file opening modes with examples of when to use each.

A3:

- `ios::in` – Read only (default for ifstream)
- `ios::out` – Write, overwrites (default for ofstream)
- `ios::app` – Append, add to end (log files)
- `ios::ate` – Start at end, can seek (random access)
- `ios::binary` – Binary mode (images, structures)
- `ios::trunc` – Truncate if exists (fresh start)

Q4: How would you handle errors in file operations? Write code with proper error handling.

A4:

```cpp
#include <fstream.h>
#include <iostream.h>

int main() {
    ifstream file("data.txt");

    if(!file) {
        cout << "Error: Could not open file!\n";
        return 1; // Exit with error code
    }

    // Check for specific errors
    if(file.fail()) {
        cout << "File operation failed!\n";
        file.clear(); // Clear error state
    }

    if(file.eof()) {
        cout << "End of file reached\n";
    }

    file.close();
    return 0;
}
```

Q5: Design a program that merges two text files into a third file.

A5:

```cpp
#include <fstream.h>
#include <iostream.h>

void mergeFiles(const char* file1, const char* file2, const char* output) {
    ifstream in1(file1);
    ifstream in2(file2);
    ofstream out(output);

    if(!in1 || !in2 || !out) {
        cout << "Error opening files!\n";
        return;
    }

    char line[200];

    // Copy first file
    while(!in1.eof()) {
        in1.getline(line, 200);
        if(!in1.eof()) out << line << endl;
    }

    // Copy second file
    while(!in2.eof()) {
        in2.getline(line, 200);
        if(!in2.eof()) out << line << endl;
    }

    in1.close();
    in2.close();
    out.close();
    cout << "Files merged successfully!\n";
}
```

Q6: What are the advantages of using binary files over text files for storing data?

A6:

- **Smaller size** (no ASCII conversion)
- **Faster** read/write (no parsing)
- **Preserves precision** (floating-point values)
- **Platform independent** (no newline issues)
- **Can store structures** directly

Q7: Write a program that encrypts a text file by shifting each character by a key value.

A7:

```cpp
#include <fstream.h>
#include <iostream.h>

void encryptFile(const char* input, const char* output, int key) {
    ifstream in(input);
    ofstream out(output);

    if(!in || !out) {
        cout << "Error opening files!\n";
        return;
    }

    char ch;
    while(!in.eof()) {
        in.get(ch);
        if(!in.eof()) {
            ch = ch + key; // Simple Caesar cipher
            out.put(ch);
        }
    }

    in.close();
    out.close();
    cout << "File encrypted!\n";
}
```

Q8: How would you implement a simple database system using files for storing records?

A8:

```cpp
// Structure for record
struct Record {
    int id;
    char name[50];
    float value;
    bool deleted; // For soft delete
};

// Functions needed:
// 1. addRecord() - Append to file
// 2. searchRecord() - Linear search
// 3. updateRecord() - Read, modify, write back
// 4. deleteRecord() - Mark as deleted
// 5. compactFile() - Remove deleted records
// Use fixed-size records for random access
```

Lecture 19: Random Access & Binary Files

Objective:

Extract key concepts related to random access files, binary file I/O, file position pointers, and block read/write operations from Lecture 19.

Categories & Extracted Content:

1. **Random Access Files**
   - Non-sequential access: Read/write at any position
   - Key functions: `fseek()`, `ftell()`, `rewind()`

2. **File Position Pointer**
   - System maintains pointer for each open file
   - Indicates current read/write location
   - Can be moved with `fseek()` or `seekg()`/`seekp()`
3. **C-Style Binary I/O Functions**

   fwrite():

```cpp
size_t fwrite(void *ptr, size_t size, size_t nmemb, FILE *fp);
```

- Writes nmemb elements of size bytes from ptr to file
- Example: `fwrite(earnings, sizeof(double), 10, fp);`

fread():

```cpp
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *fp);
```

- Reads nmemb elements of size bytes into ptr from file

4. **Structure Example (Component Data)**

```cpp
typedef struct {
    char name[20];
    int serial_code;
    int amount;
    float cost;
} component;
```

- Writing structure: `fwrite(&comp_data, sizeof(comp_data), 1, fp);`

5. **fseek() Function**

```cpp
int fseek(FILE *fp, long offset, int mode);
```

• Modes:

o `SEEK_SET` - From beginning of file

o `SEEK_CUR` - From current position

o `SEEK_END` - From end of file

• Offset: Positive (forward), negative (backward), or zero

6. **ftell() and rewind()**
   - `ftell(fp)`: Returns current file position
   - `rewind(fp)`: Resets to beginning (`fseek(fp, 0L, SEEK_SET)`)
7. **C++ File Positioning Functions**

   For Input Files (ifstream):
   - `tellg()`: Get read position
   - `seekg(offset, origin)`: Set read position

For Output Files (ofstream):

• `tellp()`: Get write position

• `seekp(offset, origin)`: Set write position

Origins:

• `ios::beg` - Beginning of file

• `ios::cur` - Current position

• `ios::end` - End of file

8. **Examples**

   Example 1: Get file length

```cpp
ifstream inFile("myFile.txt");
inFile.seekg(0L, ios::end);
length = inFile.tellg();
```

Example 2: Reverse file display

• Uses `fseek()` to go to end, then reads backward

Example 3: Block read/write

```cpp
char str[10000];
myInputFile.read(str, 10000);
myOutputFile.write(str, 10000);
```

9. **Block I/O with read() and write()**
   - `read(char *buff, int count)`: Reads count bytes into buffer
   - `write(char *buff, int count)`: Writes count bytes from buffer
10. **Binary vs Text Mode**
    - Text mode: Stores integers as ASCII characters
    - Binary mode: Stores raw bytes
    - Example: Integer 32 stored differently in each mode
11. **fstream for Both Input/Output**

```cpp
fstream myFile("Sample.txt", ios::in | ios::out);
```

12. **Writing Integers to Binary File**

```cpp
for(i=0; i<100; i++) {
    myOutputFile.write(&i, sizeof(i));
}
```

13. **File Modes**
    - `ios::app` - Append
    - `ios::trunc` - Truncate
    - `ios::ate` - Start at end

Result:

Key concepts extracted: random file access with position pointers; binary I/O functions (fwrite(), fread()); file positioning (fseek(), ftell(), seekg(), seekp()); structure I/O to binary files; block read/write operations; text vs binary file storage; C++ stream positioning methods.

Takeaways for Exam:

• Know difference between sequential and random access

• Use fseek() with correct mode and offset

• Write/read structures with fwrite()/fread()

• Get file size using seekg() and tellg()

• Understand text vs binary storage

• Use read()/write() for block transfers

• Position file pointers for non-sequential access

✅ Multiple Choice Questions (MCQs)

1. Which function moves the file position pointer?
   a) ftell()
   b) fseek()
   c) rewind()
   d) Both b and c

Key: d) Both b and c – fseek() moves to specific position, rewind() moves to beginning.

2. What does `fwrite(&data, sizeof(data), 1, fp)` do?
   a) Reads one element from file
   b) Writes one element to file
   c) Moves pointer one byte
   d) Checks file size

Key: b) Writes one element to file – Writes one structure/element.

3. Which mode in fseek() starts from the end of file?
   a) SEEK_SET
   b) SEEK_CUR

c) SEEK_END

d) SEEK_EOF

Key: c) SEEK_END – From end of file.

4. What does ftell(fp) return?

   a) File size

   b) Current position in file

   c) Error status

   d) Number of bytes read

Key: b) Current position in file – Offset from beginning in bytes.

5. Which C++ function gets the read position?

   a) tellp()

   b) tellg()

   c) seekg()

   d) seekp()

Key: b) tellg() – "get" position for input streams.

6. What is the purpose of rewind(fp)?

   a) Move to end of file

   b) Reset to beginning

   c) Close the file

   d) Delete file contents

Key: b) Reset to beginning – Same as fseek(fp, 0L, SEEK_SET).

7. In binary mode, how is integer 32 stored?

   a) As characters '3' and '2'

   b) As 4-byte binary value

   c) As 2-byte value

   d) As 8-byte value

Key: b) As 4-byte binary value – Raw binary representation (4 bytes for int).

8. Which function reads a block of bytes?
   a) get()
   b) read()
   c) fread()
   d) Both b and c

Key: d) Both b and c – read() in C++, fread() in C.

❓ Short Answer Questions

Q1: Differentiate between sequential and random file access.
A1:

- **Sequential:** Read/write in order from beginning to end (tape analogy)
- **Random:** Read/write at any position directly (CD/DVD analogy)

Q2: Write the syntax of fseek() and explain its parameters.
A2:

```
cpp
int fseek(FILE *fp, long offset, int mode);
```

- `fp`: File pointer
- `offset`: Bytes to move (+ forward, - backward)
- `mode`: SEEK_SET (beginning), SEEK_CUR (current), SEEK_END (end)

Q3: How do you find the size of a file using C++ streams?
A3:

```
cpp
ifstream file("data.txt", ios::binary | ios::ate);
```

```
int size = file.tellg();
file.close();
```

Q4: What is the difference between tellg() and tellp()?

A4:

- `tellg()`: Get read position (for input streams)
- `tellp()`: Get write position (for output streams)

Q5: Write code to write a structure to a binary file.

A5:

```
cpp
struct Student { char name[50]; int roll; float marks; };
Student s;
fstream file("data.bin", ios::binary | ios::out);
file.write((char*)&s, sizeof(s));
```

Q6: How does binary mode differ from text mode for storing integers?

A6:

- **Text mode:** Stores as ASCII characters (e.g., '3','2' for 32)
- **Binary mode:** Stores as raw bytes (4 bytes for int: 0x00000020)

Q7: What does `seekg(-10L, ios::cur)` do?

A7: Moves read pointer 10 bytes backward from current position.

Q8: Explain the purpose of read() and write() functions.

A8:

- `read()`: Reads block of bytes from file to memory buffer
- `write()`: Writes block of bytes from memory buffer to file

💻 Programming Exercises

1. **Find file size:**

```cpp
#include <fstream.h>
#include <iostream.h>

int main() {
    ifstream file("data.txt", ios::binary | ios::ate);

    if(!file) {
        cout << "File not found!";
        return 1;
    }

    long size = file.tellg();
    file.close();

    cout << "File size: " << size << " bytes";
    return 0;
}
```

2. **Write 100 integers, read 50th:**

```cpp
#include <fstream.h>
#include <iostream.h>

int main() {
    // Write 100 integers
    ofstream outFile("numbers.bin", ios::binary);
    for(int i=0; i<100; i++) {
        outFile.write((char*)&i, sizeof(int));
    }
    outFile.close();

    // Read 50th integer (49th index)
```

```
    ifstream inFile("numbers.bin", ios::binary);
    int num;

    // Position to 49th integer: 49 * sizeof(int)
    inFile.seekg(49 * sizeof(int), ios::beg);
    inFile.read((char*)&num, sizeof(int));

    cout << "50th integer: " << num;
    inFile.close();

    return 0;
}
```

3. **Append to binary file:**

```
cpp

#include <fstream.h>
#include <iostream.h>

int main() {
    // Append mode in binary
    ofstream file("data.bin", ios::binary | ios::app);

    int data = 999;
    file.write((char*)&data, sizeof(int));

    file.close();
    cout << "Data appended successfully!";
    return 0;
}
```

4. **Read structure at specific position:**

```
cpp

#include <fstream.h>
#include <iostream.h>

struct Employee {
    char name[30];
```

```
    int id;
    float salary;
};

int main() {
    fstream file("employees.bin", ios::binary | ios::in);
    Employee emp;

    // Read 3rd employee (index 2)
    int recordSize = sizeof(Employee);
    file.seekg(2 * recordSize, ios::beg);
    file.read((char*)&emp, recordSize);

    cout << "Name: " << emp.name << endl;
    cout << "ID: " << emp.id << endl;
    cout << "Salary: " << emp.salary << endl;

    file.close();
    return 0;
}
```

5. **Display file in reverse order:**

```
cpp

#include <fstream.h>
#include <iostream.h>

int main() {
    ifstream file("text.txt", ios::binary | ios::ate);

    if(!file) {
        cout << "File not found!";
        return 1;
    }

    long size = file.tellg();
    char ch;

    for(long i=1; i<=size; i++) {
```

```
        file.seekg(-i, ios::end);
        file.get(ch);
        cout << ch;
    }

    file.close();
    return 0;
}
```

6. **Update specific record:**

```cpp
#include <fstream.h>
#include <iostream.h>

struct Record {
    int id;
    char data[50];
};

int main() {
    fstream file("records.bin", ios::binary | ios::in | ios::out);
    Record rec;
    int recordNum;

    cout << "Enter record number to update (0-indexed): ";
    cin >> recordNum;

    // Move to record position
    file.seekg(recordNum * sizeof(Record), ios::beg);
    file.read((char*)&rec, sizeof(Record));

    cout << "Current data: " << rec.data << endl;
    cout << "Enter new data: ";
    cin.ignore();
    cin.getline(rec.data, 50);

    // Move back and write updated record
    file.seekp(recordNum * sizeof(Record), ios::beg);
```

```
    file.write((char*)&rec, sizeof(Record));

    file.close();
    cout << "Record updated successfully!";
    return 0;
}
```

7. **Copy file using block operations:**

```cpp
#include <fstream.h>
#include <iostream.h>

int main() {
    ifstream srcFile("source.bin", ios::binary);
    ofstream destFile("destination.bin", ios::binary);

    if(!srcFile || !destFile) {
        cout << "Error opening files!";
        return 1;
    }

    // Get file size
    srcFile.seekg(0, ios::end);
    long size = srcFile.tellg();
    srcFile.seekg(0, ios::beg);

    // Use buffer for block copy
    const int BUFFER_SIZE = 4096;
    char buffer[BUFFER_SIZE];
    long bytesRead;

    while(size > 0) {
        int toRead = (size > BUFFER_SIZE) ? BUFFER_SIZE : size;
        srcFile.read(buffer, toRead);
        destFile.write(buffer, toRead);
        size -= toRead;
    }
```

```
    srcFile.close();
    destFile.close();

    cout << "File copied successfully!";
    return 0;
}
```

8. **Student record system:**

```cpp
#include <fstream.h>
#include <iostream.h>

struct Student {
    int roll;
    char name[50];
    float gpa;
};

void addStudent() {
    ofstream file("students.bin", ios::binary | ios::app);
    Student s;

    cout << "Enter roll: "; cin >> s.roll;
    cout << "Enter name: "; cin.ignore(); cin.getline(s.name, 50);
    cout << "Enter GPA: "; cin >> s.gpa;

    file.write((char*)&s, sizeof(Student));
    file.close();
}

void displayStudent(int rollNum) {
    ifstream file("students.bin", ios::binary);
    Student s;

    while(file.read((char*)&s, sizeof(Student))) {
        if(s.roll == rollNum) {
            cout << "Roll: " << s.roll << endl;
            cout << "Name: " << s.name << endl;
```

```
            cout << "GPA: " << s.gpa << endl;
            file.close();
            return;
        }
    }

    cout << "Student not found!";
    file.close();
}

void updateStudent(int rollNum) {
    fstream file("students.bin", ios::binary | ios::in | ios::out);
    Student s;

    while(file.read((char*)&s, sizeof(Student))) {
        if(s.roll == rollNum) {
            cout << "Enter new GPA: ";
            cin >> s.gpa;

            // Move back one record and update
            file.seekp(-sizeof(Student), ios::cur);
            file.write((char*)&s, sizeof(Student));

            cout << "Record updated!";
            file.close();
            return;
        }
    }

    cout << "Student not found!";
    file.close();
}

int main() {
    int choice, roll;

    do {
        cout << "\n1. Add Student\n2. Display Student\n3. Update Student\n4. Exit\n";
        cout << "Enter choice: ";
```

```
        cin >> choice;

        switch(choice) {
            case 1: addStudent(); break;
            case 2:
                cout << "Enter roll to display: ";
                cin >> roll;
                displayStudent(roll);
                break;
            case 3:
                cout << "Enter roll to update: ";
                cin >> roll;
                updateStudent(roll);
                break;
            case 4: cout << "Exiting..."; break;
            default: cout << "Invalid choice!";
        }
    } while(choice != 4);

    return 0;
}
```

□ Conceptual/Long Questions

Q1: Explain the concept of file position pointers with diagrams. How do they work in sequential vs random access?

A1:

File position pointer: Internal marker tracking current read/write location.

- **Sequential:** Pointer moves forward automatically after each operation
- **Random:** Pointer can be moved anywhere using `fseek()`/`seekg()`
  Diagram: File as array of bytes, pointer as index that can jump anywhere.

Q2: Write a complete database program for employee records using random access files.

A2:

```cpp
struct Employee {
    int id;
    char name[50];
    char department[30];
    float salary;
    bool active; // For soft delete
};

class EmployeeDB {
    fstream file;

public:
    EmployeeDB(const char* filename) {
        file.open(filename, ios::binary | ios::in | ios::out);
        if(!file) {
            // Create new file
            file.open(filename, ios::binary | ios::out);
            file.close();
            file.open(filename, ios::binary | ios::in | ios::out);
        }
    }

    void addEmployee(Employee &e) {
        file.seekp(0, ios::end);
        file.write((char*)&e, sizeof(Employee));
    }

    Employee getEmployee(int id) {
        Employee e;
        file.seekg(0, ios::beg);
        while(file.read((char*)&e, sizeof(Employee))) {
            if(e.id == id && e.active) return e;
        }
        // Return empty employee if not found
```

```
        Employee empty = {0};
        return empty;
    }

    void updateEmployee(int id, Employee &newData) {
        Employee e;
        file.seekg(0, ios::beg);
        long pos = 0;
        while(file.read((char*)&e, sizeof(Employee))) {
            if(e.id == id) {
                file.seekp(pos, ios::beg);
                file.write((char*)&newData, sizeof(Employee));
                return;
            }
            pos = file.tellg();
        }
    }
};
```

Q3: Compare text files and binary files for storing different data types (integers, floats, strings, structures).

A3:

- **Integers:** Text: "123" (3+ bytes), Binary: 4 bytes (int)
- **Floats:** Text: "3.14" (4+ bytes), Binary: 4 bytes (float)
- **Strings:** Text: as-is, Binary: as-is (no difference)
- **Structures:** Text: serialize each field, Binary: write entire structure directly

Q4: How would you implement an index for fast record retrieval in a binary file?

A4:

Create separate index file storing (key, file_position) pairs:

```
cpp
struct IndexEntry {
    int key; // e.g., employee ID
    long position; // position in data file
};
```

```
// Keep index sorted, use binary search for fast lookup
```

Q5: Explain error handling in file operations (EOF, fail, bad bits).

A5:

- `eof()`: End of file reached
- `fail()`: Logical error (e.g., type mismatch)
- `bad()`: Physical error (e.g., disk failure)
- `good()`: No errors

  Always check stream state after operations.

Q6: Design a program that can recover deleted records in a binary file (soft delete implementation).

A6:

```cpp
struct Record {
    int id;
    char data[100];
    bool deleted; // Soft delete flag
};

void deleteRecord(int id) {
    // Mark as deleted instead of removing
    Record r;
    fstream file("data.bin", ios::binary | ios::in | ios::out);
    file.seekg(0, ios::beg);
    long pos = 0;
    while(file.read((char*)&r, sizeof(Record))) {
        if(r.id == id) {
            r.deleted = true;
            file.seekp(pos, ios::beg);
            file.write((char*)&r, sizeof(Record));
            break;
        }
        pos = file.tellg();
    }
```

```
    file.close();
}

void compactFile() {
    // Remove deleted records periodically
    ifstream in("data.bin", ios::binary);
    ofstream out("temp.bin", ios::binary);
    Record r;

    while(in.read((char*)&r, sizeof(Record))) {
        if(!r.deleted) {
            out.write((char*)&r, sizeof(Record));
        }
    }

    in.close();
    out.close();
    remove("data.bin");
    rename("temp.bin", "data.bin");
}
```

Q7: Write a function that searches for a pattern in a binary file.

A7:

```
cpp
bool searchInFile(const char* filename, const char* pattern) {
    ifstream file(filename, ios::binary);
    if(!file) return false;

    // Get file size
    file.seekg(0, ios::end);
    long size = file.tellg();
    file.seekg(0, ios::beg);

    // Read entire file
    char* buffer = new char[size];
    file.read(buffer, size);
    file.close();
```

```
    // Simple search (for demonstration)
    int patternLen = strlen(pattern);
    for(int i=0; i<=size-patternLen; i++) {
        if(strncmp(&buffer[i], pattern, patternLen) == 0) {
            delete[] buffer;
            return true;
        }
    }

    delete[] buffer;
    return false;
}
```

Q8: What are the performance considerations when working with large binary files?

A8:

1. **Buffer size:** Larger buffers reduce system calls
2. **Sequential vs random access:** Sequential is faster
3. **Block size:** Match OS/hardware block size
4. **Caching:** OS may cache frequently accessed data
5. **Direct I/O:** Bypass cache for very large files
6. **Memory mapping:** Map file to memory for fastest access

Lecture 20: Structures & Unions

Objective:

Extract key concepts related to structures and unions in C++, including their declaration, initialization, usage, arrays, pointers, and differences between structures and unions.

Categories & Extracted Content:

1. **Structures (struct)**

   Definition:

```cpp
struct Student {
    char name[60];
    char address[100];
    double gpa;
};  // Note: Semicolon required
```

Key Components:

• Keyword: struct

• Structure name: Student

• Members/variables: name, address, gpa

Declaration:

```cpp
Student s1, s2, s3;  // Variables of type structure
```

2. **Nested Structures**

```cpp
struct Address {
    char streetAddress[100];
    char city[30];
    char country[30];
};

struct Student {
    char name[60];
    Address add;  // Nested structure
    double gpa;
};
```

3. **Structure Operations**

   Allowed:

- Assignment: `s1 = s2;`
- Passing to functions
- Returning from functions
- Array of structures: `Student s[100];`
- Pointers to structures: `Student *sPtr;`

Not Allowed:

- Arithmetic: `stdnt1 + stdnt2;` ❌
- Direct string assignment: `s1.name = "Abbas Ali";` ❌

(Use `strcpy(s1.name, "Abbas Ali");`)

4. **Initializing Structures**

```cpp
Student s1 = {"Amara", "cs201", 19, 2002};
```

5. **Accessing Structure Members**
   - Direct access: `s1.name`, `s1.age`
   - Via pointer:
     - `(*sPtr).name` or
     - `sPtr->name` (arrow operator)
6. **Arrays of Structures**

```cpp
Student s[10];
s[0].name;  // Access first student's name
s[1].name;  // Access second student's name
```

7. **Example: Student Database**

```cpp
struct Student {
    char firstName[30];
    char lastName[30];
    char course[15];
    char rollNo[10];
```

```
    int age;
    float gpa;
};

Student s[10];
// Input loop for 10 students
for(int i=0; i<10; i++) {
    cin >> s[i].firstName >> s[i].lastName >> s[i].course
        >> s[i].rollNo >> s[i].age >> s[i].gpa;
}
```

8. **Functions with Structures**

   Returning structure:

```cpp
Student getData() {
    Student tempStudent;
    // Input data
    return tempStudent;
}

Student s1 = getData();
```

Reading from file:

```cpp
Student getData() {
    Student tempStudent;
    inFile >> tempStudent.firstName;
    inFile >> tempStudent.lastName;
    // etc.
    return tempStudent;
}
```

9. **Unions**

   Definition:

```cpp
union intOrChar {
```

```
    int i;
    char c;
};
```

Key Characteristics:

• Shared memory: All members share same memory location

• Size: Size of union = size of largest member

• Only one member active at a time

Example:

```cpp
union intOrDouble {
    int ival;
    double dval;
};

intOrDouble u1;
u1.ival = 10;     // Now ival is active
cout << u1.ival;  // Correct: 10
cout << u1.dval;  // Incorrect: garbage value
```

10. **Union vs Structure**

Feature	Structure	Union
Memory	Each member has own memory	All members share memory
Size	Sum of all members' sizes	Size of largest member
Usage	All members can store values	Only one member active at a time

11. **Example: Union for Type Conversion**

```cpp
union iandc {
    char c[4];
    int x;
};
```

• Allows viewing integer as 4 characters and vice versa

12. **Important Points**
    - `sizeof()` can be used with structures
    - Structures can be passed by value or by reference to functions
    - Structures support nested definitions
    - Unions save memory but require careful usage
    - Arrow operator `->` for pointer access to members

Result:
Key concepts extracted: structure syntax and usage; nested structures; arrays and pointers to structures; structure initialization and member access; unions and their memory sharing; differences between structures and unions; practical examples (student database, type conversion).

Takeaways for Exam:
- Know structure declaration syntax
- Understand member access (`.` and `->`)
- Use arrays of structures for data collections
- Implement functions that return/accept structures
- Recognize union memory sharing behavior
- Differentiate between structure and union usage
- Apply proper string handling in structures

✅ Multiple Choice Questions (MCQs)

1. Which keyword is used to define a structure in C++?
   a) class
   b) struct
   c) structure
   d) union

Key: b) struct – Structure definition keyword.

2. How do you access a member of a structure using a pointer?
   a) ptr.member
   b) ptr->member
   c) *ptr.member
   d) Both b and c

Key: b) ptr->member – Arrow operator for pointer access.

3. What is the size of a union?
   a) Sum of all members' sizes
   b) Size of smallest member
   c) Size of largest member
   d) Always 8 bytes

Key: c) Size of largest member – All members share memory.

4. Which operation is NOT allowed on structures?
   a) s1 = s2; (assignment)
   b) s1 + s2; (addition)
   c) Passing to functions
   d) Returning from functions

Key: b) s1 + s2; (addition) – Arithmetic not defined for structures.

5. How do you initialize a structure?
   a) Student s1 = {"Ali", 20, 3.5};
   b) Student s1("Ali", 20, 3.5);
   c) s1 = {"Ali", 20, 3.5};
   d) Both a and b

Key: a) Student s1 = {"Ali", 20, 3.5}; – Initialization at declaration.

6. What happens when you assign to one member of a union?
   a) All members get the same value
   b) Only that member gets the value
   c) Memory is corrupted
   d) All members become invalid

Key: b) Only that member gets the value – Others contain garbage.

7. Which is correct for string assignment in a structure?
   a) s1.name = "Ali";
   b) strcpy(s1.name, "Ali");
   c) s1.name[] = "Ali";
   d) s1.name <- "Ali";

Key: b) strcpy(s1.name, "Ali"); – Must use strcpy for character arrays.

8. What does sizeof(structure) return?
   a) Size of first member
   b) Sum of all members' sizes
   c) Size of largest member
   d) May include padding bytes

Key: d) May include padding bytes – Compiler may add padding for alignment.

❓ Short Answer Questions

Q1: What is the difference between a structure and a union?
A1:

- **Structure:** Each member has separate memory, size = sum of all members
- **Union:** All members share memory, size = largest member, only one active at a time

Q2: Write the syntax for defining a structure with 3 members.

A2:

```cpp
struct Example {
    int id;
    char name[50];
    float value;
};
```

Q3: How do you create an array of structures? Give an example.

A3:

```cpp
struct Student {
    char name[50];
    int roll;
};
Student students[100];  // Array of 100 Student structures
```

Q4: What is the arrow operator (->) used for?

A4: Used to access structure/class members through a pointer.

```cpp
Student *ptr = &s1;
cout << ptr->name;  // Instead of (*ptr).name
```

Q5: Why can't you use arithmetic operators on structures?

A5: Structures can contain different data types; C++ doesn't define how to add/multiply heterogeneous collections.

Q6: Write code to declare and initialize a union with int and char members.

A6:

```cpp
union Data {
    int i;
    char c;
};
```

```
Data d;
d.i = 65;  // Now i is active
```

Q7: How do you pass a structure to a function by reference?

A7:

```cpp
void modifyStudent(Student &s) {
    s.age = 20;
}
```

Q8: What is nested structure? Give an example.

A8: Structure within another structure.

```cpp
struct Address {
    char city[30];
    char street[50];
};

struct Person {
    char name[50];
    Address addr;  // Nested structure
};
```

💻 Programming Exercises

1. **Book structure:**

```cpp

#include <iostream>
#include <cstring>
using namespace std;
```

```cpp
struct Book {
    char title[100];
    char author[50];
    float price;
};

int main() {
    Book b;

    cout << "Enter book title: ";
    cin.getline(b.title, 100);

    cout << "Enter author: ";
    cin.getline(b.author, 50);

    cout << "Enter price: ";
    cin >> b.price;

    cout << "\nBook Details:\n";
    cout << "Title: " << b.title << endl;
    cout << "Author: " << b.author << endl;
    cout << "Price: $" << b.price << endl;

    return 0;
}
```

2. **Array of employee structures:**

```cpp
cpp

#include <iostream>
using namespace std;

struct Employee {
    char name[50];
    int id;
    float salary;
};

int main() {
```

```
    Employee emp[5];

    for(int i=0; i<5; i++) {
        cout << "\nEmployee " << i+1 << ":\n";
        cout << "Name: ";
        cin.ignore();
        cin.getline(emp[i].name, 50);
        cout << "ID: ";
        cin >> emp[i].id;
        cout << "Salary: ";
        cin >> emp[i].salary;
    }

    cout << "\nEmployee Records:\n";
    for(int i=0; i<5; i++) {
        cout << "Name: " << emp[i].name
             << ", ID: " << emp[i].id
             << ", Salary: $" << emp[i].salary << endl;
    }

    return 0;
}
```

3. **Union with int, float, char:**

```
cpp

#include <iostream>
using namespace std;

union Data {
    int i;
    float f;
    char c;
};

int main() {
    Data d;

    d.i = 10;
```

```cpp
    cout << "Integer: " << d.i << endl;

    d.f = 3.14;
    cout << "Float: " << d.f << endl;

    d.c = 'A';
    cout << "Character: " << d.c << endl;

    // Note: Only one value is valid at a time
    return 0;
}
```

4. **Function with structure pointer:**

```cpp
#include <iostream>
using namespace std;

struct Student {
    char name[50];
    int age;
    float gpa;
};

void updateStudent(Student *s) {
    s->age = 20;
    s->gpa = 3.8;
}

int main() {
    Student s = {"Ali", 19, 3.5};

    cout << "Before: Age=" << s.age << ", GPA=" << s.gpa << endl;
    updateStudent(&s);
    cout << "After: Age=" << s.age << ", GPA=" << s.gpa << endl;

    return 0;
}
```

5. **Nested structure:**

```cpp
#include <iostream>
using namespace std;

struct Address {
    char city[30];
    char street[50];
    int zip;
};

struct Person {
    char name[50];
    int age;
    Address addr;  // Nested structure
};

int main() {
    Person p;

    cout << "Enter name: ";
    cin.getline(p.name, 50);

    cout << "Enter age: ";
    cin >> p.age;
    cin.ignore();

    cout << "Enter city: ";
    cin.getline(p.addr.city, 30);

    cout << "Enter street: ";
    cin.getline(p.addr.street, 50);

    cout << "Enter ZIP: ";
    cin >> p.addr.zip;

    cout << "\nPerson Details:\n";
    cout << "Name: " << p.name << endl;
```

```
    cout << "Age: " << p.age << endl;
    cout << "Address: " << p.addr.street << ", "
         << p.addr.city << " - " << p.addr.zip << endl;

    return 0;
}
```

6. **Highest salary finder:**

```cpp
#include <iostream>
using namespace std;

struct Employee {
    char name[50];
    float salary;
};

int main() {
    Employee emp[5];

    // Input employees
    for(int i=0; i<5; i++) {
        cout << "Employee " << i+1 << " name: ";
        cin.ignore();
        cin.getline(emp[i].name, 50);
        cout << "Salary: ";
        cin >> emp[i].salary;
    }

    // Find highest salary
    int maxIndex = 0;
    for(int i=1; i<5; i++) {
        if(emp[i].salary > emp[maxIndex].salary) {
            maxIndex = i;
        }
    }

    cout << "\nEmployee with highest salary:\n";
```

```
    cout << "Name: " << emp[maxIndex].name << endl;
    cout << "Salary: $" << emp[maxIndex].salary << endl;

    return 0;
}
```

7. **Union for student data:**

```cpp
#include <iostream>
#include <cstring>
using namespace std;

union StudentData {
    int roll;
    char name[30];
};

struct Student {
    int type;  // 0 for roll, 1 for name
    StudentData data;
};

int main() {
    Student s;

    cout << "Enter 0 for roll number or 1 for name: ";
    cin >> s.type;
    cin.ignore();

    if(s.type == 0) {
        cout << "Enter roll number: ";
        cin >> s.data.roll;
        cout << "Student roll: " << s.data.roll << endl;
    } else {
        cout << "Enter name: ";
        cin.getline(s.data.name, 30);
        cout << "Student name: " << s.data.name << endl;
    }
```

```
    return 0;
}
```

8. **Compare two structures:**

```cpp
#include <iostream>
#include <cstring>
using namespace std;

struct Point {
    int x;
    int y;
};

bool comparePoints(Point p1, Point p2) {
    return (p1.x == p2.x && p1.y == p2.y);
}

int main() {
    Point p1 = {10, 20};
    Point p2 = {10, 20};
    Point p3 = {30, 40};

    if(comparePoints(p1, p2)) {
        cout << "p1 and p2 are equal\n";
    } else {
        cout << "p1 and p2 are not equal\n";
    }

    if(comparePoints(p1, p3)) {
        cout << "p1 and p3 are equal\n";
    } else {
        cout << "p1 and p3 are not equal\n";
    }

    return 0;
}
```

□ Conceptual/Long Questions

Q1: Design a complete student management system using structures, arrays, and file I/O.

A1:

```cpp
struct Student {
    int roll;
    char name[50];
    char course[30];
    float marks[5];
    float average;
};

class StudentManager {
    Student students[100];
    int count;

public:
    void addStudent() {
        Student s;
        cout << "Enter roll: "; cin >> s.roll;
        cout << "Enter name: "; cin.ignore(); cin.getline(s.name, 50);
        cout << "Enter course: "; cin.getline(s.course, 30);

        float total = 0;
        for(int i=0; i<5; i++) {
            cout << "Enter mark " << i+1 << ": ";
            cin >> s.marks[i];
            total += s.marks[i];
        }
        s.average = total / 5;

        students[count++] = s;
    }
```

```
    void saveToFile() {
        ofstream file("students.dat", ios::binary);
        file.write((char*)students, count * sizeof(Student));
        file.close();
    }

    void loadFromFile() {
        ifstream file("students.dat", ios::binary);
        count = 0;
        while(file.read((char*)&students[count], sizeof(Student))) {
            count++;
        }
        file.close();
    }

    void displayAll() {
        for(int i=0; i<count; i++) {
            cout << "Roll: " << students[i].roll
                 << ", Name: " << students[i].name
                 << ", Average: " << students[i].average << endl;
        }
    }
};
```

Q2: Explain memory layout of structures with and without padding. Give examples.

A2:

Without padding:

```cpp
struct Example1 { char a; int b; char c; };
// Size might be 1 + 4 + 1 = 6 bytes (but usually not due to padding)
```

With padding (typical 4-byte alignment):

```cpp
struct Example1 {
    char a;    // 1 byte
    // 3 bytes padding
```

```
    int b;      // 4 bytes
    char c;     // 1 byte
    // 3 bytes padding
}; // Total: 12 bytes
```

Packed structure (no padding):

```cpp
#pragma pack(1)
struct Example1 { char a; int b; char c; }; // 6 bytes
```

Q3: How would you implement a simple database using structures and random access files?

A3:

```cpp
struct Record {
    int id;
    char name[50];
    float value;
    bool deleted;
};

class Database {
    fstream file;

    long findPosition(int id) {
        Record r;
        file.seekg(0, ios::beg);
        long pos = 0;
        while(file.read((char*)&r, sizeof(Record))) {
            if(r.id == id && !r.deleted) return pos;
            pos = file.tellg();
        }
        return -1;
    }

public:
    Database(const char* filename) {
```

```
    file.open(filename, ios::binary | ios::in | ios::out);
    if(!file) {
        file.open(filename, ios::binary | ios::out);
        file.close();
        file.open(filename, ios::binary | ios::in | ios::out);
    }
}

void addRecord(Record &r) {
    file.seekp(0, ios::end);
    file.write((char*)&r, sizeof(Record));
}

Record getRecord(int id) {
    long pos = findPosition(id);
    if(pos == -1) {
        Record empty = {0};
        return empty;
    }
    Record r;
    file.seekg(pos, ios::beg);
    file.read((char*)&r, sizeof(Record));
    return r;
}

void updateRecord(int id, Record &newRecord) {
    long pos = findPosition(id);
    if(pos != -1) {
        file.seekp(pos, ios::beg);
        file.write((char*)&newRecord, sizeof(Record));
    }
}

void deleteRecord(int id) {
    long pos = findPosition(id);
    if(pos != -1) {
        Record r;
        file.seekg(pos, ios::beg);
        file.read((char*)&r, sizeof(Record));
```

```
            r.deleted = true;
            file.seekp(pos, ios::beg);
            file.write((char*)&r, sizeof(Record));
        }
    }
};
```

Q4: Compare structures with classes in C++. When would you use each?

A4:

Structures: Default public members, used for plain data containers.

Classes: Default private members, used for objects with behavior.

Use structure when: Only data, no methods, or simple POD (Plain Old Data).

Use class when: Encapsulation, inheritance, polymorphism needed.

Q5: Design a union that can store different geometric shapes (circle, rectangle, triangle).

A5:

```cpp
struct Circle { float radius; };
struct Rectangle { float length, width; };
struct Triangle { float base, height; };

union ShapeData {
    Circle circle;
    Rectangle rectangle;
    Triangle triangle;
};

enum ShapeType { CIRCLE, RECTANGLE, TRIANGLE };

struct Shape {
    ShapeType type;
    ShapeData data;

    float area() {
        switch(type) {
            case CIRCLE:
                return 3.14159 * data.circle.radius * data.circle.radius;
```

```
            case RECTANGLE:
                return data.rectangle.length * data.rectangle.width;
            case TRIANGLE:
                return 0.5 * data.triangle.base * data.triangle.height;
            default: return 0;
        }
    }
};
```

Q6: Write a program that uses structures to represent complex numbers and performs operations.

A6:

```
cpp
struct Complex {
    float real;
    float imag;

    Complex add(Complex c) {
        Complex result;
        result.real = real + c.real;
        result.imag = imag + c.imag;
        return result;
    }

    Complex multiply(Complex c) {
        Complex result;
        result.real = (real * c.real) - (imag * c.imag);
        result.imag = (real * c.imag) + (imag * c.real);
        return result;
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};
```

Q7: How would you implement a linked list using structures?

A7:

```cpp
struct Node {
    int data;
    Node* next;
};

class LinkedList {
    Node* head;

public:
    LinkedList() : head(NULL) {}

    void insert(int value) {
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = head;
        head = newNode;
    }

    void display() {
        Node* temp = head;
        while(temp != NULL) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }

    // Add more functions: delete, search, etc.
};
```

Q8: Explain how unions can be used for type punning and its dangers.

A8:

Type punning: Using union to interpret same memory as different types.

```cpp
union Punning {
    int i;
    float f;
```

```
};

Punning p;
p.i = 1065353216;  // Bit pattern for 1.0f
cout << p.f;       // Prints 1.0
```

Dangers:

1. Platform dependent (endianness, size differences)
2. Violates strict aliasing rules (undefined behavior in C++)
3. Use `memcpy()` instead for safe type punning.

Arrays in C++

Slide Information

- An **array** is a collection of a fixed number of components where all components have the same data type
- **Declaration:** `dataType arrayName[intExp];` where `intExp` evaluates to a positive integer
- Array index always starts at **0** (e.g., for `int num[5];`, indices are `num[0]` to `num[4]`)
- **Array Index Out of Bounds:** Occurs when index is `< 0` or `> ARRAY_SIZE-1`. C++ does not check for this
- **Initialization:** Arrays can be initialized at declaration: `double sales[] = {12.25, 32.50, 16.90};`
- **Partial Initialization:** Unspecified elements are set to 0 `int list[10] = {8, 5, 12};` initializes first three elements, rest become 0
- **Restriction:** Arrays cannot be copied using assignment operator (`=`); must use component-wise copy
- **Arrays as Parameters:** Always passed by reference (base address is passed). Size can be omitted in function parameters. Use `const` to prevent modification
- **Base Address:** Memory location of first element (`list[0]`)
- **C-Strings:** Character arrays that are null-terminated (`'\0'`). `"A"` is actually `'A'` followed by `'\0'`
- **String Comparison:** Done character-by-character using system's collating sequence (ASCII)

- **String I/O:** `cin >> name;` (stops at whitespace) or `cin.get(str, m+1);` for strings with spaces. `cout << name;` for output
- **Parallel Arrays:** Multiple arrays where corresponding elements hold related data
- **Two-Dimensional Arrays:** Tables with rows and columns. Declared as `dataType arrayName[rows][cols];`
- When passing 2D arrays to functions, column size must be specified

MCQs

1. What is the valid index range for an array declared as `float values[20];`?
   a) 1 to 20
   b) 0 to 19
   c) 0 to 20
   d) 1 to 19
   Ans. b) 0 to 19
2. When an array is passed to a function in C++, what is actually passed?
   a) A copy of all elements
   b) The value of the first element
   c) The base address of the array
   d) The size of the array
   Ans. c) The base address of the array
3. What will be output of: `int arr[5] = {1, 2}; cout << arr[3] << arr[4];`?
   a) 00
   b) Garbage values
   c) 12
   d) Runtime error
   Ans. a) 00
4. Which statement about C-strings is FALSE?
   a) They are null-terminated
   b) `"A"` occupies 2 bytes in memory
   c) `'A'` and `"A"` are the same
   d) Can be stored in character arrays
   Ans. c) `'A'` and `"A"` are the same
5. What is the size of array declared as: `int list[] = {5, 6, 3};`?
   a) 3 bytes
   b) 3 elements
   c) 12 bytes
   d) Unknown
   Ans. b) 3 elements

6. What does "array index out of bounds" mean?
   a) Array is too large
   b) Index is negative or greater than array size
   c) Array is not initialized
   d) Memory allocation failed
   Ans. b) Index is negative or greater than array size
7. Which is true about array initialization?
   a) All elements must be initialized
   b) Uninitialized elements get garbage values
   c) Uninitialized elements become 0
   d) Array size must match number of initializers
   Ans. c) Uninitialized elements become 0
8. What is a parallel array?
   a) Array with elements in parallel memory
   b) Multiple arrays with related data at same indices
   c) 2D array
   d) Array passed to parallel functions
   Ans. b) Multiple arrays with related data at same indices
9. How are 2D arrays stored in memory?
   a) Column-major order
   b) Random order
   c) Row-major order
   d) Depends on compiler
   Ans. c) Row-major order
10. When passing 2D array to function, which dimension can be omitted?
    a) First (rows)
    b) Second (columns)
    c) Both
    d) Neither
    Ans. a) First (rows)
11. What is base address of an array?
    a) Address of last element
    b) Address of first element
    c) Total size of array
    d) Middle address
    Ans. b) Address of first element
12. How to prevent function from modifying array parameter?
    a) Pass by value
    b) Use `const` keyword
    c) Use `static` keyword

d) Use `volatile` keyword
Ans. b) Use `const` keyword

13. What is `cin.get(str, m+1);` used for?
    a) Read single character
    b) Read string with spaces
    c) Read integer
    d) Read floating point
    Ans. b) Read string with spaces
14. Which is NOT a restriction on array processing?
    a) Cannot use assignment operator
    b) Cannot compare arrays directly
    c) Cannot pass to functions
    d) Cannot return from functions
    Ans. c) Cannot pass to functions
15. What is the null character in C-strings?
    a) `'0'`
    b) `'\n'`
    c) `'\0'`
    d) `' '`
    Ans. c) `'\0'`

Answer Questions

Q1. What is an array and how is it declared in C++?
A: An array is a collection of a fixed number of components of the same data type. It is declared as: `dataType arrayName[intExp];` where `intExp` evaluates to a positive integer specifying the number of elements.

Q2. Explain "array index out of bounds" and how C++ handles it.
A: Array index out of bounds occurs when an index is less than 0 or greater than ARRAY_SIZE-1. C++ does not check for this condition, and accessing out-of-bounds indices leads to undefined behavior, potentially corrupting memory or crashing the program.

Q3. How can arrays be initialized at declaration? Give examples.
A: Arrays can be initialized at declaration using braces:

- `double sales[] = {12.25, 32.50, 16.90};` (size determined automatically)
- `int list[10] = {0};` (all elements initialized to 0)
- `int arr[5] = {1, 2};` (first two elements initialized, rest become 0)

Q4. What are the restrictions on array processing in C++?
A: 1. Cannot use assignment operator to copy arrays (must copy element by element)
2. Cannot compare arrays directly (must compare element by element)
3. Cannot return entire array from a function
4. Cannot pass arrays by value to functions

Q5. How are arrays passed to functions and why?
A: Arrays are always passed by reference. The base address (address of first element) is passed to the function. This is done for efficiency, as copying large arrays would be expensive in terms of time and memory.

Q6. What is the base address of an array?
A: The base address of an array is the memory location of its first element. For array `list`, the base address is the address of `list[0]`.

Q7. Differentiate between C-string `"A"` and character `'A'`.
A: `'A'` is a single character literal occupying 1 byte. `"A"` is a C-string consisting of two characters: `'A'` followed by the null terminator `'\0'`, occupying 2 bytes.

Q8. How are C-strings compared?
A: C-strings are compared character-by-character using the system's collating sequence (typically ASCII). Comparison stops at the first differing character or when null terminator is reached.

Q9. Explain how to read strings with spaces in C++.
A: To read strings containing spaces, use `cin.get(str, m+1);` where `str` is the character array and `m` is the maximum number of characters to read (excluding null terminator).

Q10. What are parallel arrays? Give an example.
A: Parallel arrays are two or more arrays where corresponding elements hold related information. Example:
`int studentId[50];` and `char courseGrade[50];` where `studentId[i]` and `courseGrade[i]` belong to the same student.

Q11. How are two-dimensional arrays declared and accessed?
A: 2D arrays are declared as: `dataType arrayName[rows][cols];`
Accessed using: `arrayName[rowIndex][colIndex]` where both indices start from 0.

Q12. What happens during array initialization if fewer values are provided?
A: If fewer initializers are provided than array size, the specified elements are initialized with given values, and all remaining elements are automatically initialized to 0 (for numeric types).

Q13. Why must column size be specified when passing 2D arrays to functions?
A: Column size must be specified so the compiler can calculate memory offsets correctly. In row-major storage, to access `array[i][j]`, compiler needs to know number of columns: `base + (i * cols + j) * sizeof(element)`.

Q14. What is the purpose of `const` with array parameters?
A: Using `const` with array parameters prevents the function from modifying the array elements, ensuring data integrity and communicating intent to other programmers.

Q15. How does C++ handle string output with `cout`?
A: When using `cout << name;` where `name` is a C-string, characters are output until the null terminator `'\0'` is encountered. If no null terminator exists, output continues into adjacent memory, causing undefined behavior.

Programming Exercise

Exercise 1: Find Largest Element

```cpp
#include <iostream>
using namespace std;

int findLargest(int arr[], int size) {
    int largest = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > largest) {
            largest = arr[i];
        }
    }
    return largest;
}

int main() {
    int numbers[10];

    cout << "Enter 10 integers: ";
    for (int i = 0; i < 10; i++) {
        cin >> numbers[i];
    }
```

```
    int largest = findLargest(numbers, 10);
    cout << "Largest element: " << largest << endl;

    return 0;
}
```

Exercise 2: Reverse Array In-Place

```cpp
#include <iostream>
using namespace std;

void reverseArray(int arr[], int size) {
    int start = 0;
    int end = size - 1;

    while (start < end) {
        // Swap elements
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;

        start++;
        end--;
    }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5, 6};
    int size = 6;

    cout << "Original array: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    reverseArray(arr, size);

    cout << "Reversed array: ";
```

```
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Exercise 3: Compare Two Arrays

```cpp
#include <iostream>
using namespace std;

bool areArraysEqual(int arr1[], int arr2[], int size) {
    for (int i = 0; i < size; i++) {
        if (arr1[i] != arr2[i]) {
            return false;
        }
    }
    return true;
}

int main() {
    int arr1[] = {1, 2, 3, 4, 5};
    int arr2[] = {1, 2, 3, 4, 5};
    int arr3[] = {1, 2, 3, 4, 6};

    cout << "arr1 == arr2: " << (areArraysEqual(arr1, arr2, 5) ? "true" :
"false") << endl;
    cout << "arr1 == arr3: " << (areArraysEqual(arr1, arr3, 5) ? "true" :
"false") << endl;

    return 0;
}
```

Exercise 4: Count Vowels in C-String

```cpp
#include <iostream>
using namespace std;

bool isVowel(char c) {
```

```
    c = tolower(c);
    return (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u');
}

int countVowels(char str[]) {
    int count = 0;
    for (int i = 0; str[i] != '\0'; i++) {
        if (isVowel(str[i])) {
            count++;
        }
    }
    return count;
}

int main() {
    char input[100];

    cout << "Enter a string: ";
    cin.getline(input, 100);

    int vowelCount = countVowels(input);
    cout << "Number of vowels: " << vowelCount << endl;

    return 0;
}
```

Exercise 5: Sum of 2D Array Elements

```cpp
#include <iostream>
using namespace std;

int sum2DArray(int arr[][3], int rows, int cols) {
    int total = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            total += arr[i][j];
        }
    }
    return total;
}
```

```
int main() {
    int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

    int total = sum2DArray(matrix, 2, 3);
    cout << "Sum of all elements: " << total << endl;

    return 0;
}
```

Conceptual Question

Why are arrays considered a fundamental data structure in programming, and what are their main advantages and limitations?
Answer: Arrays are fundamental because they provide efficient, contiguous memory storage for homogeneous data with constant-time access using indices. Advantages include: 1) Fast random access O(1), 2) Memory efficiency (no overhead per element), 3) Cache-friendly due to contiguous storage. Limitations include: 1) Fixed size after declaration, 2) Costly insertion/deletion in middle, 3) Cannot store different data types, 4) Wasted space if allocated size exceeds needed size. Understanding arrays is crucial as they form the basis for more complex data structures like strings, matrices, and dynamic arrays.

Memory Allocation in C++

- **Memory Allocation** refers to reserving memory space during program execution
- **C-style allocation:** Uses `malloc()`, `calloc()`, `realloc()`, and `free()`
  - `malloc(size)` allocates raw memory block
  - `calloc(num, size)` allocates and initializes to zero
  - `realloc(ptr, new_size)` resizes existing block
  - `free(ptr)` deallocates memory
- **C++-style allocation:** Uses `new` and `delete` operators
  - `new` allocates memory and calls constructors
  - `delete` deallocates memory and calls destructors

- **Single element allocation:** `int *iptr = new int;` then `delete iptr;`
- **Array allocation:** `int *iptr = new int[10];` then `delete [] iptr;`
- **Object allocation:** `Date *dptr = new Date;` then `delete dptr;`
- **Array of objects:** `Date *dptr = new Date[10];` then `delete [] dptr;`
- **Memory leak** occurs when allocated memory is not deallocated
- **Dangling pointer** occurs when pointer points to deallocated memory
- **Key differences:** `new/delete` are operators, type-safe, call constructors/destructors; `malloc/free` are functions, not type-safe
- **Proper deallocation** is crucial to prevent memory leaks
- `delete []` must be used for arrays, `delete` for single objects
- **Accessing allocated memory:** Use `->` operator for pointers to objects: `dptr->setDate();`

MCQs

1. Which operator is used for dynamic memory allocation in C++?

   a) `malloc`

   b) `alloc`

   c) `new`

   d) `create`

   Ans. c) `new`
2. How do you deallocate memory for an array allocated with `new`?

   a) `delete objPtr;`

   b) `delete [] objPtr;`

   c) `free(objPtr);`

   d) `dealloc(objPtr);`

   Ans. b) `delete [] objPtr;`
3. What is the main advantage of `new` over `malloc()`?

   a) `new` is faster

   b) `new` calls constructors, `malloc()` does not

   c) `new` uses less memory

d) `new` works with all data types

Ans. b) `new` calls constructors, `malloc()` does not

4. What is a memory leak?

   a) When memory is accessed out of bounds

   b) When dynamically allocated memory is not deallocated

   c) When stack memory overflows

   d) When two pointers point to the same memory

   Ans. b) When dynamically allocated memory is not deallocated

5. Which statement correctly allocates an array of 10 integers?

   a) `int *iptr = new int(10);`

   b) `int *iptr = new int[10];`

   c) `int iptr = new int[10];`

   d) `int *iptr = malloc(10);`

   Ans. b) `int *iptr = new int[10];`

6. What happens if you use `delete` instead of `delete []` for an array?

   a) Only first element is deallocated

   b) All elements are deallocated properly

   c) Compilation error

   d) Runtime error

   Ans. a) Only first element is deallocated

7. Which function is used to resize memory in C-style allocation?

   a) `malloc()`

   b) `calloc()`

   c) `realloc()`

   d) `resize()`

   Ans. c) `realloc()`

8. What is a dangling pointer?

   a) Pointer that is not initialized

   b) Pointer that points to deallocated memory

   c) Pointer that points to wrong type

d) Pointer with null value

Ans. b) Pointer that points to deallocated memory

9. How do you access member functions through an object pointer?

   a) `dptr.setDate();`

   b) `dptr->setDate();`

   c) `*dptr.setDate();`

   d) `dptr::setDate();`

   Ans. b) `dptr->setDate();`

10. Which allocates and initializes memory to zero in C?

    a) `malloc(10 * sizeof(int));`

    b) `calloc(10, sizeof(int));`

    c) `new int[10];`

    d) `alloc(10, sizeof(int));`

    Ans. b) `calloc(10, sizeof(int));`

11. What does `new` return if allocation fails?

    a) NULL

    b) 0

    c) Throws `bad_alloc` exception

    d) Garbage value

    Ans. c) Throws `bad_alloc` exception

12. Which is NOT a valid deallocation?

    a) `delete ptr;` after `ptr = new int;`

    b) `delete [] ptr;` after `ptr = new int[5];`

    c) `free(ptr);` after `ptr = new int;`

    d) `delete ptr;` after `ptr = new int[5];`

    Ans. d) `delete ptr;` after `ptr = new int[5];`

13. What is the purpose of `realloc()`?

    a) Allocate new memory

    b) Deallocate memory

    c) Resize existing allocated memory

d) Initialize memory

Ans. c) Resize existing allocated memory

14. How to allocate memory for a single character?

    a) `char *c = new char;`

    b) `char *c = new char();`

    c) `char *c = new char[1];`

    d) All of the above

    Ans. d) All of the above

15. What is the correct sequence?

    a) Allocate → Use → Deallocate

    b) Use → Allocate → Deallocate

    c) Deallocate → Allocate → Use

    d) Allocate → Deallocate → Use

    Ans. a) Allocate → Use → Deallocate

Answer Questions

Q1. Differentiate between `malloc()` and `new`.

A: `malloc()` is a C library function that allocates raw memory without initialization, while `new` is a C++ operator that allocates memory and calls constructors for objects. `new` is type-safe and can throw exceptions on failure.

Q2. What is a memory leak and how can it be prevented?

A: A memory leak occurs when dynamically allocated memory is not deallocated, causing the program to consume increasing memory. It can be prevented by ensuring every `new` has a corresponding `delete`, and every `new []` has a corresponding `delete []`.

Q3. Explain the difference between `delete` and `delete []`.

A: `delete` is used to deallocate memory for a single object allocated with `new`, while `delete []` is used to deallocate arrays allocated with `new []`. Using the wrong form leads to undefined behavior.

Q4. What are dangling pointers and how do they occur?

A: Dangling pointers are pointers that point to memory that has been deallocated. They occur when memory is freed but the pointer is not set to NULL, or when a pointer points to local variables that have gone out of scope.

Q5. How does `realloc()` work?

A: `realloc()` changes the size of a previously allocated memory block. It may move the block to a new location if necessary. If successful, it returns a pointer to the resized memory; if it fails, it returns NULL and leaves the original block unchanged.

Q6. Why should you use `new` instead of `malloc()` in C++?

A: `new` should be used in C++ because: 1) It calls constructors for object initialization, 2) It's type-safe, 3) It can be overloaded for custom types, 4) It throws exceptions on failure, 5) It works with C++ features like inheritance and polymorphism.

Q7. What happens if you don't deallocate dynamically allocated memory?

A: If dynamically allocated memory is not deallocated, it remains allocated until the program terminates, causing memory leaks. This can lead to reduced available memory, slower performance, and eventually program or system crashes.

Q8. How do you allocate and deallocate an array of objects?

A: Allocation: `ClassName *arr = new ClassName[size];`
Deallocation: `delete [] arr;`
This ensures constructors are called for each object during allocation and destructors during deallocation.

Q9. What is the arrow operator (`->`) used for?

A: The arrow operator (`->`) is used to access members (data or functions) of an object through a pointer. Example: `ptr->member` is equivalent to `(*ptr).member`.

Q10. Can you mix `malloc/free` with `new/delete`?

A: No, you should not mix them. Memory allocated with `malloc()` should be freed

with `free()`, and memory allocated with `new` should be deallocated with `delete`. Mixing them leads to undefined behavior.

Q11. What does `calloc()` do differently from `malloc()`?

A: `calloc()` allocates memory and initializes all bytes to zero, while `malloc()` allocates memory without initialization (contains garbage values). `calloc()` also takes two arguments: number of elements and size of each element.

Q12. How to check if memory allocation succeeded?

A: For `new`, catch `bad_alloc` exception or use `new(nothrow)`. For `malloc()/calloc()`, check if returned pointer is NULL. Example: `int *p = new(nothrow) int[100]; if(!p) { /* handle error */ }`

Q13. What is the purpose of the `nothrow` version of `new`?

A: `new(nothrow)` returns NULL instead of throwing an exception when allocation fails. This allows C-style error checking: `int *p = new(nothrow) int[100]; if(p == NULL) { /* handle error */ }`

Q14. How does dynamic memory allocation differ from static allocation?

A: Static allocation happens at compile time with fixed size (stack or global memory), while dynamic allocation happens at runtime with flexible size (heap memory). Dynamic allocation allows resizing and controlling lifetime explicitly.

Q15. What precautions should be taken when using dynamic memory?

A: 1) Always check allocation success, 2) Match allocation with correct deallocation, 3) Set pointers to NULL after deletion, 4) Avoid memory leaks by deallocating all allocated memory, 5) Don't access freed memory.

Programming Exercise

Exercise 1: Memory Leak Demonstration

```cpp
#include <iostream>
using namespace std;
```

```cpp
void createMemoryLeak() {
    int* ptr = new int[100];  // Allocate memory
    // Forget to delete it - MEMORY LEAK!
    // No delete [] ptr;
}

int main() {
    cout << "Creating memory leak..." << endl;
    for (int i = 0; i < 1000; i++) {
        createMemoryLeak();
    }
    cout << "Memory leaked (check task manager)" << endl;

    return 0;
}
```

Exercise 2: Compare malloc vs new

```cpp
#include <iostream>
#include <cstdlib>  // For malloc, free
using namespace std;

class MyClass {
private:
    int data;
public:
    MyClass() : data(0) {
        cout << "Constructor called" << endl;
    }
    ~MyClass() {
        cout << "Destructor called" << endl;
    }
    void setData(int d) { data = d; }
    int getData() { return data; }
};

int main() {
    cout << "Using new (C++ style):" << endl;
    MyClass* obj1 = new MyClass();
```

```
    obj1->setData(42);
    cout << "Data: " << obj1->getData() << endl;
    delete obj1;

    cout << "\nUsing malloc (C style):" << endl;
    MyClass* obj2 = (MyClass*)malloc(sizeof(MyClass));
    // Constructor NOT called
    obj2->setData(42);  // Unsafe - object not properly constructed
    cout << "Data: " << obj2->getData() << endl;
    free(obj2);  // Destructor NOT called

    return 0;
}
```

Exercise 3: Dynamic Array Function

```cpp
#include <iostream>
using namespace std;

int* createDynamicArray(int size) {
    int* arr = new int[size];

    // Initialize with values 1 to size
    for (int i = 0; i < size; i++) {
        arr[i] = i + 1;
    }

    return arr;
}

int main() {
    int size;
    cout << "Enter array size: ";
    cin >> size;

    int* dynamicArray = createDynamicArray(size);

    cout << "Array elements: ";
    for (int i = 0; i < size; i++) {
        cout << dynamicArray[i] << " ";
```

```cpp
    }
    cout << endl;

    // Don't forget to delete!
    delete [] dynamicArray;

    return 0;
}
```

Exercise 4: Proper vs Improper Deallocation

```cpp
#include <iostream>
using namespace std;

void properDeallocation() {
    cout << "Proper deallocation:" << endl;
    int* arr = new int[5];
    for (int i = 0; i < 5; i++) {
        arr[i] = i * 10;
    }

    // CORRECT: delete [] for array
    delete [] arr;
    cout << "Array properly deleted" << endl;
}

void improperDeallocation() {
    cout << "\nImproper deallocation (using delete instead of delete []):" << endl;
    int* arr = new int[5];
    for (int i = 0; i < 5; i++) {
        arr[i] = i * 10;
    }

    // WRONG: delete instead of delete []
    delete arr;  // Only first element properly deallocated
    // Memory leak for other 4 elements!
    cout << "Warning: Improper deallocation - memory leak!" << endl;
}

int main() {
```

```
    properDeallocation();
    improperDeallocation();

    return 0;
}
```

Exercise 5: Resize Array with realloc

```cpp
#include <iostream>
#include <cstdlib>
#include <cstring>
using namespace std;

int main() {
    int* arr = NULL;
    int size = 0;

    cout << "Enter initial array size: ";
    cin >> size;

    arr = (int*)malloc(size * sizeof(int));

    // Initialize array
    for (int i = 0; i < size; i++) {
        arr[i] = i * 2;
    }

    cout << "Original array: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Resize array
    int newSize;
    cout << "Enter new size: ";
    cin >> newSize;

    arr = (int*)realloc(arr, newSize * sizeof(int));
```

```
    // Initialize new elements if array grew
    if (newSize > size) {
        for (int i = size; i < newSize; i++) {
            arr[i] = i * 3;
        }
    }

    cout << "Resized array: ";
    for (int i = 0; i < newSize; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    free(arr);

    return 0;
}
```

Conceptual Question

Why is proper memory management crucial in C++ programming, and what are the common pitfalls programmers face with dynamic memory allocation?
Answer: Proper memory management is crucial because: 1) It prevents memory leaks that degrade performance, 2) It avoids dangling pointers that cause crashes, 3) It ensures efficient use of limited memory resources, 4) It prevents security vulnerabilities like buffer overflows. Common pitfalls include: 1) Memory leaks (forgetting `delete`), 2) Double deletion (calling `delete` twice), 3) Using wrong deallocation form (`delete` vs `delete []`), 4) Dangling pointers (using freed memory), 5) Memory corruption (writing beyond allocated bounds), 6) Not checking allocation success. Modern C++ encourages using smart pointers and containers to automate memory management.

Friend Functions & References

- **Friend Function:** A non-member function granted access to private and protected members of a class
- Declared inside class with `friend` keyword before function prototype
- Friend functions are **not** member functions (no `this` pointer, called without object)
- Can access private/protected data of class that declares it as friend
- **Limitations:** Friendship is not transitive (friend of friend is not friend), not inherited, not associative
- **References (`&`):** An alias (alternative name) for an existing variable
- Must be initialized when declared: `int &j = i;` (j becomes alias for i)
- Cannot be NULL, cannot be reassigned to refer to different variable
- Used for: 1) Function parameters (call by reference), 2) Function return values
- **Reference vs Pointer:** References are safer (cannot be NULL), syntax cleaner (no dereferencing `*` needed)
- **Reference parameters:** Allow functions to modify original arguments
- **Constant references:** `const &` prevents modification but avoids copying
- **Dangling reference:** Reference to destroyed object (undefined behavior)
- **References as return values:** Allows function calls on left side of assignment
- **Friend function example:** Allows external function to access private data of multiple classes

MCQs

1. What is true about friend functions?
   a) They are member functions
   b) They have access to private members
   c) They can be called using object and dot operator
   d) Friendship is inherited
   Ans. b) They have access to private members
2. How is a reference different from a pointer?
   a) Reference can be NULL

b) Reference can be reassigned

c) Reference doesn't need dereferencing

d) Reference takes more memory

Ans. c) Reference doesn't need dereferencing

3. What is output: `int x=5; int &y=x; y=10; cout<<x;`?

   a) 5

   b) 10

   c) Garbage

   d) Error

   Ans. b) 10

4. Which is NOT true about references?

   a) Must be initialized

   b) Can be NULL

   c) Cannot be reassigned

   d) Is an alias

   Ans. b) Can be NULL

5. Where is `friend` keyword placed?

   a) In function definition

   b) In function prototype inside class

   c) In main function

   d) In header file

   Ans. b) In function prototype inside class

6. What is a dangling reference?

   a) Reference to constant

   b) Reference to destroyed object

   c) Reference to pointer

   d) Uninitialized reference

   Ans. b) Reference to destroyed object

7. How to pass by reference in function?

   a) `void func(int x)`

b) `void func(int *x)`

c) `void func(int &x)`

d) `void func(&int x)`

Ans. c) `void func(int &x)`

8. What does friendship NOT provide?

   a) Access to private members

   b) Transitive relationship

   c) One-way relationship

   d) Can be given to non-member functions

   Ans. b) Transitive relationship

9. What is purpose of `const` with reference parameter?

   a) Allows modification

   b) Prevents copying but not modification

   c) Prevents modification but not copying

   d) Makes parameter constant

   Ans. c) Prevents modification but not copying

10. Can friend function access `this` pointer?

    a) Yes

    b) No

    c) Only if static

    d) Only if virtual

    Ans. b) No

11. What is output: `int i=5; int &r=i; int *p=&i; r=10; cout<<*p;`?

    a) 5

    b) 10

    c) Address

    d) Error

    Ans. b) 10

12. Which shows correct friend function declaration?

    a) `friend void func() {}`

b) `void friend func();`

c) `friend void func();`

d) `void func() friend;`

Ans. c) `friend void func();`

13. Why use reference instead of pointer for parameters?

    a) More efficient

    b) Safer (can't be NULL)

    c) Allows array access

    d) Can be reassigned

    Ans. b) Safer (can't be NULL)

14. What is limitation of friend functions?

    a) Cannot access public members

    b) Violates encapsulation

    c) Cannot be overloaded

    d) Must be defined inside class

    Ans. b) Violates encapsulation

15. How to return reference from function?

    a) `int& func()`

    b) `&int func()`

    c) `int func&()`

    d) `func() &int`

    Ans. a) `int& func()`

Answer Questions

Q1. What is a friend function and why is it used?

A: A friend function is a non-member function that is granted special access to the private and protected members of a class. It is used when a function needs to access private data of a class but doesn't logically belong as a member function, often for operations involving multiple classes.

Q2. Differentiate between references and pointers.

A: References: 1) Must be initialized, 2) Cannot be NULL, 3) Cannot be reassigned, 4) Accessed directly (no `*`), 5) Safer. Pointers: 1) Can be uninitialized, 2) Can be NULL, 3) Can be reassigned, 4) Need dereferencing (`*`), 5) More flexible but error-prone.

Q3. Explain "friendship is not transitive" with example.

A: If Class A is friend of Class B, and Class B is friend of Class C, Class A is NOT automatically friend of Class C. Each friendship must be explicitly declared. Example: `friend class B;` in A doesn't make A friend of C even if B is friend of C.

Q4. What are the advantages of using reference parameters?

A: Advantages: 1) Functions can modify original arguments, 2) Avoids copying large objects (efficiency), 3) Cleaner syntax than pointers, 4) Safer than pointers (cannot be NULL), 5) Supports operator overloading naturally.

Q5. How do you declare a function as friend of a class?

A: Inside the class definition, use: `friend returnType functionName(parameters);` The friend declaration can appear anywhere in the class (private, protected, or public section).

Q6. What is a dangling reference and how can it occur?

A: A dangling reference is a reference that refers to memory that has been deallocated. It can occur when: 1) Returning reference to local variable, 2) Reference to dynamically allocated memory after `delete`, 3) Reference to object that went out of scope.

Q7. Why might friend functions be considered to violate encapsulation?

A: Friend functions violate encapsulation because they allow external functions to access private implementation details of a class, which breaks the principle of data hiding. This can make the class harder to maintain and modify.

Q8. How are references implemented internally by compilers?

A: Internally, references are typically implemented as constant pointers (`T* const`) that

are automatically dereferenced. The compiler handles the dereferencing transparently, making them syntactically cleaner than explicit pointers.

Q9. What are constant references and when are they useful?

A: Constant references (`const T&`) are references that cannot modify the referred object. They are useful for: 1) Passing large objects to functions efficiently without copying, 2) Preventing accidental modification, 3) Accepting both lvalues and rvalues.

Q10. Can a class be friend of another class?

A: Yes, using `friend class ClassName;` declaration. This makes all member functions of the friend class able to access private/protected members of the class granting friendship.

Q11. What happens when you return a reference from a function?

A: Returning a reference allows the function call to be used as an lvalue (on left side of assignment). Example: `arr[i] = 5;` where `operator[]` returns reference. However, you must ensure the reference doesn't become dangling.

Q12. How does call by reference differ from call by value?

A: Call by value copies the argument, so changes inside function don't affect original. Call by reference passes the original variable (or its reference), so changes inside function affect the original. Reference avoids copying overhead.

Q13. Can you have reference to a reference in C++?

A: No, C++ doesn't allow references to references directly. However, you can have reference to pointer, or in template contexts, reference collapsing rules apply.

Q14. What is the syntax for declaring a reference?

A: `type& refName = varName;` Example: `int x = 5; int &ref = x;` The `&` is part of the type, not the variable name. `int& a, b;` declares `a` as reference, `b` as normal int.

Q15. When should you use friend functions versus member functions?

A: Use member functions when operation logically belongs to the object (uses `this`).

Use friend functions when: 1) Function needs access to multiple classes' private data, 2) Function is symmetric (like `operator<<`), 3) Function doesn't conceptually belong to any single class.

Programming Exercise

Exercise 1: Two Classes with Common Friend Function

```cpp
#include <iostream>
using namespace std;

class ClassB;  // Forward declaration

class ClassA {
private:
    int secretA;
public:
    ClassA() : secretA(100) {}
    friend void showSecrets(ClassA, ClassB);
};

class ClassB {
private:
    int secretB;
public:
    ClassB() : secretB(200) {}
    friend void showSecrets(ClassA, ClassB);
};

// Friend function accessing private members of both classes
void showSecrets(ClassA a, ClassB b) {
    cout << "ClassA secret: " << a.secretA << endl;
    cout << "ClassB secret: " << b.secretB << endl;
    cout << "Sum of secrets: " << a.secretA + b.secretB << endl;
}

int main() {
```

```
    ClassA objA;
    ClassB objB;

    showSecrets(objA, objB);

    return 0;
}
```

Exercise 2: Operator<< Overloading with Friend Function

```cpp
#include <iostream>
using namespace std;

class Date {
private:
    int day, month, year;
public:
    Date(int d, int m, int y) : day(d), month(m), year(y) {}

    // Friend function for output
    friend ostream& operator<<(ostream& out, const Date& d);
};

// Friend function definition
ostream& operator<<(ostream& out, const Date& d) {
    out << d.day << "/" << d.month << "/" << d.year;
    return out;
}

int main() {
    Date today(15, 3, 2024);
    Date birthday(10, 5, 2000);

    cout << "Today: " << today << endl;
    cout << "Birthday: " << birthday << endl;

    // Can chain output
    cout << "Dates: " << today << " and " << birthday << endl;

    return 0;
```

```
}
```

Exercise 3: Class with Friend Function Display

```cpp
#include <iostream>
using namespace std;

class BankAccount {
private:
    int accountNumber;
    double balance;
    string ownerName;
public:
    BankAccount(int accNo, double bal, string name)
        : accountNumber(accNo), balance(bal), ownerName(name) {}

    // Friend function declaration
    friend void displayAccount(const BankAccount& account);
};

// Friend function definition - can access private members
void displayAccount(const BankAccount& account) {
    cout << "Account Details:" << endl;
    cout << "Account Number: " << account.accountNumber << endl;
    cout << "Owner: " << account.ownerName << endl;
    cout << "Balance: $" << account.balance << endl;
    cout << "-------------------" << endl;
}

int main() {
    BankAccount acc1(12345, 1500.75, "John Doe");
    BankAccount acc2(67890, 25000.50, "Jane Smith");

    displayAccount(acc1);
    displayAccount(acc2);

    return 0;
}
```

Exercise 4: Distance Calculator with Friend Function

```cpp
```

```cpp
#include <iostream>
#include <cmath>
using namespace std;

class Point {
private:
    double x, y;
public:
    Point(double xVal, double yVal) : x(xVal), y(yVal) {}

    // Friend function to calculate distance
    friend double distanceBetween(const Point& p1, const Point& p2);

    // Display point
    void display() {
        cout << "(" << x << ", " << y << ")";
    }
};

// Friend function accessing private members
double distanceBetween(const Point& p1, const Point& p2) {
    double dx = p1.x - p2.x;
    double dy = p1.y - p2.y;
    return sqrt(dx*dx + dy*dy);
}

int main() {
    Point p1(1.0, 2.0);
    Point p2(4.0, 6.0);
    Point p3(0.0, 0.0);

    cout << "Distance between ";
    p1.display();
    cout << " and ";
    p2.display();
    cout << " = " << distanceBetween(p1, p2) << endl;

    cout << "Distance between ";
    p2.display();
```

```
    cout << " and ";
    p3.display();
    cout << " = " << distanceBetween(p2, p3) << endl;

    return 0;
}
```

Exercise 5: Friend Class Example

```cpp
#include <iostream>
using namespace std;

class SecretHolder {
private:
    int secretNumber;
    string secretMessage;
public:
    SecretHolder(int num, string msg)
        : secretNumber(num), secretMessage(msg) {}

    // Declare FriendClass as friend
    friend class FriendClass;
};

class FriendClass {
public:
    void revealSecrets(const SecretHolder& holder) {
        cout << "Revealing secrets from FriendClass:" << endl;
        cout << "Secret Number: " << holder.secretNumber << endl;
        cout << "Secret Message: " << holder.secretMessage << endl;
    }

    void modifySecret(SecretHolder& holder, int newNum, string newMsg) {
        holder.secretNumber = newNum;
        holder.secretMessage = newMsg;
        cout << "Secrets modified!" << endl;
    }
};

int main() {
```

```
    SecretHolder secret(42, "The answer to everything");
    FriendClass friendObj;

    friendObj.revealSecrets(secret);

    cout << "\nModifying secrets..." << endl;
    friendObj.modifySecret(secret, 99, "New secret message");

    cout << "\nRevealing modified secrets:" << endl;
    friendObj.revealSecrets(secret);

    return 0;
}
```

Conceptual Question

How do friend functions and references balance between encapsulation/security and flexibility/performance in C++ programming?

Answer: Friend functions provide controlled violation of encapsulation for cases where operations logically span multiple classes or don't fit as member functions. They offer flexibility while maintaining explicit permission (must be declared as friend). References provide performance benefits (avoid copying) and safety advantages (cannot be NULL) compared to pointers, while enabling efficient parameter passing and return values. Together, they allow C++ programmers to write efficient, flexible code while generally maintaining good encapsulation practices. References enable call-by-reference semantics safely, and friend functions enable specific cross-class operations without exposing private data globally. The key is judicious use: references for efficiency without sacrificing safety, friend functions for necessary access without breaking encapsulation entirely.

Functions in C++ (Part 1)

- **Functions:** Building blocks that divide programs into manageable pieces
- **Predefined functions:** `sqrt(x)`, `pow(x,y)`, `floor(x)`, `ceil(x)` in `<cmath>`
- **Value-returning functions:** Have a return type, used in expressions
- **Void functions:** No return type, called as standalone statements
- **Function components:** Name, parameters, return type, body
- **Formal parameters:** Variables in function definition
- **Actual parameters:** Values/variables in function call
- **Function prototype:** Declaration before use: `returnType functionName(parameterTypes);`
- `return` **statement:** Returns value and exits function immediately
- **Flow of execution:** Starts at `main()`, functions execute when called
- **Function call transfers control** to function, returns to caller after completion
- **Value-returning functions** replace call with returned value
- **Break statement:** Exits loops/switches immediately
- **Continue statement:** Skips to next iteration of loop
- `exit()` **function:** Terminates program immediately from anywhere
- **Examples:** Mathematical functions, custom functions like `larger()`, `maximum()`

MCQs

1. Which header contains mathematical functions like `sqrt()`?

   a) `<iostream>`

   b) `<cmath>`

   c) `<math.h>`

   d) Both b and c

   Ans. d) Both b and c
2. What does `pow(2,3)` return?

   a) 6

   b) 8

   c) 9

d) 5

Ans. b) 8

3. Which function rounds down to nearest integer?

   a) `ceil()`

   b) `floor()`

   c) `round()`

   d) `abs()`

   Ans. b) `floor()`

4. What is a function prototype?

   a) Function definition

   b) Function declaration without body

   c) Function call

   d) Function parameter

   Ans. b) Function declaration without body

5. Which statement immediately exits a function?

   a) `break`

   b) `continue`

   c) `return`

   d) `exit`

   Ans. c) `return`

6. What does `ceil(3.2)` return?

   a) 3

   b) 3.0

   c) 4

   d) 4.0

   Ans. d) 4.0

7. Where does program execution always start?

   a) First function in file

   b) `main()` function

   c) Global variables

d) First #include

Ans. b) `main()` function

8. What is actual parameter?

   a) Variable in function definition

   b) Value in function call

   c) Return type

   d) Function name

   Ans. b) Value in function call

9. Which jumps to next loop iteration?

   a) `break`

   b) `continue`

   c) `return`

   d) `goto`

   Ans. b) `continue`

10. What terminates entire program?

    a) `return 0;` in main

    b) `exit(0);`

    c) Both a and b

    d) `break;`

    Ans. c) Both a and b

11. Which is value-returning function?

    a) `void print()`

    b) `int calculate()`

    c) Both

    d) Neither

    Ans. b) `int calculate()`

12. What is `sqrt(16)`?

    a) 4

    b) 8

    c) 256

d) 2

Ans. a) 4

13. How are function parameters matched?

    a) By name

    b) By position

    c) By type

    d) Both b and c

    Ans. d) Both b and c

14. What does function prototype end with?

    a) `{}`

    b) `;`

    c) `()`

    d) `:`

    Ans. b) `;`

15. Can `main()` call itself?

    a) Yes, recursion

    b) No, never

    c) Only with special syntax

    d) Only once

    Ans. a) Yes, recursion

Answer Questions

Q1. What are the advantages of using functions in programming?

A: 1) Divide complex programs into manageable pieces, 2) Enable code reuse, 3) Allow different people to work on different functions, 4) Simplify debugging and testing, 5) Improve readability and maintenance.

Q2. Differentiate between value-returning and void functions.

A: Value-returning functions have a return type and return a value

using `return` statement, used in expressions. Void functions have no return type, don't return values, called as standalone statements.

Q3. Explain formal vs actual parameters.

A: Formal parameters are variables declared in function definition. Actual parameters are values/variables passed during function call. They are matched by position and must be compatible in type and number.

Q4. What is the purpose of function prototype?

A: Function prototype: 1) Informs compiler about function before its definition, 2) Specifies return type and parameters, 3) Enables type checking of calls, 4) Allows functions to be defined in any order.

Q5. How does `break` differ from `continue`?

A: `break` exits the entire loop or switch immediately. `continue` skips remaining statements in current iteration and proceeds to next iteration of the loop.

Q6. What does `exit()` function do?

A: `exit()` terminates the program immediately from any point. It takes an integer argument (usually 0 for success) and returns it to the operating system. Defined in `<cstdlib>`.

Q7. Describe the flow of execution when a function is called.

A: 1) Control transfers from caller to function, 2) Actual parameters are evaluated and passed, 3) Function body executes, 4) `return` statement sends value back (if any), 5) Control returns to point after call.

Q8. What are predefined functions? Give examples.

A: Predefined functions are library functions provided by C++.
Examples: `sqrt()`, `pow()`, `floor()`, `ceil()` in `<cmath>`; `strlen()`, `strcpy()` in `<cstring>`.

Q9. How do you call a value-returning function?

A: Call it within an expression where its return value is used: in assignment (`result =`

`func()`), output (`cout << func()`), condition (`if(func() > 0)`), or as argument to another function.

Q10. What happens if a function doesn't have a `return` statement?
A: For void functions, it returns implicitly at end. For value-returning functions, missing `return` causes undefined behavior (compiler may warn). In `main()`, missing `return 0;` is allowed and returns 0 by default.

Q11. Can a function call another function?
A: Yes, functions can call other functions. This is called function nesting or composition. `main()` can call functions, and those functions can call other functions.

Q12. What is function overloading?
A: Function overloading allows multiple functions with same name but different parameter lists (number or types). The compiler determines which version to call based on arguments.

Q13. How are mathematical functions like `pow()` used?
A: `pow(x, y)` returns x raised to power y. Include `<cmath>`, both parameters and return are `double`. Example: `pow(2.0, 3.0)` returns 8.0.

Q14. What is recursion?
A: Recursion occurs when a function calls itself directly or indirectly. It must have a base case to terminate. Example: factorial calculation: `n! = n * (n-1)!`.

Q15. Why is `main()` function special?
A: `main()` is special because: 1) Execution starts here, 2) It can have parameters (`argc`, `argv`), 3) Returning from `main()` ends program, 4) Implicit `return 0;` if omitted, 5) Cannot be called explicitly (except recursively).

Programming Exercise

Exercise 1: Factorial Function

```cpp
```

```cpp
#include <iostream>
using namespace std;

// Recursive factorial function
long long factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

// Iterative factorial function
long long factorialIterative(int n) {
    long long result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    if (num < 0) {
        cout << "Factorial is not defined for negative numbers." << endl;
    } else {
        cout << "Recursive factorial(" << num << ") = " << factorial(num) << endl;
        cout << "Iterative factorial(" << num << ") = " << factorialIterative(num) <<
endl;
    }

    return 0;
}
```

Exercise 2: Quadratic Equation Solver

```cpp
#include <iostream>
#include <cmath>
```

```cpp
using namespace std;

void solveQuadratic(double a, double b, double c) {
    double discriminant = b * b - 4 * a * c;

    if (discriminant > 0) {
        double root1 = (-b + sqrt(discriminant)) / (2 * a);
        double root2 = (-b - sqrt(discriminant)) / (2 * a);
        cout << "Two real roots: " << root1 << " and " << root2 << endl;
    } else if (discriminant == 0) {
        double root = -b / (2 * a);
        cout << "One real root: " << root << endl;
    } else {
        double realPart = -b / (2 * a);
        double imaginaryPart = sqrt(-discriminant) / (2 * a);
        cout << "Complex roots: " << realPart << " + " << imaginaryPart << "i and "
             << realPart << " - " << imaginaryPart << "i" << endl;
    }
}

int main() {
    double a, b, c;

    cout << "Enter coefficients a, b, c: ";
    cin >> a >> b >> c;

    if (a == 0) {
        cout << "This is not a quadratic equation (a cannot be 0)" << endl;
    } else {
        solveQuadratic(a, b, c);
    }

    return 0;
}
```

Exercise 3: Largest of Three Numbers

```cpp
#include <iostream>
using namespace std;
```

```cpp
int findLargest(int a, int b, int c) {
    int largest = a;

    if (b > largest) {
        largest = b;
    }
    if (c > largest) {
        largest = c;
    }

    return largest;
}

int findLargestUsingMax(int a, int b, int c) {
    return max(a, max(b, c));
}

int main() {
    int x, y, z;

    cout << "Enter three integers: ";
    cin >> x >> y >> z;

    cout << "Largest (custom function): " << findLargest(x, y, z) << endl;
    cout << "Largest (using max): " << findLargestUsingMax(x, y, z) << endl;

    return 0;
}
```

Exercise 4: Break and Continue Demo

```cpp
#include <iostream>
using namespace std;

void breakContinueDemo() {
    cout << "Demonstrating break statement:" << endl;
    for (int i = 1; i <= 10; i++) {
        if (i == 5) {
            cout << "Breaking at i = " << i << endl;
            break;  // Exit loop completely
```

```cpp
        }
        cout << i << " ";
    }

    cout << "\n\nDemonstrating continue statement:" << endl;
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue;  // Skip even numbers
        }
        cout << i << " ";  // Only odd numbers printed
    }

    cout << "\n\nBreak in nested loops:" << endl;
    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
            if (i * j > 4) {
                cout << "\nBreaking inner loop at i=" << i << ", j=" << j;
                break;
            }
            cout << "(" << i << "," << j << ") ";
        }
    }
}

int main() {
    breakContinueDemo();
    return 0;
}
```

Exercise 5: Fibonacci Recursive Function

```cpp
#include <iostream>
using namespace std;

int fibonacci(int n) {
    if (n <= 0) {
        return 0;
    } else if (n == 1) {
        return 1;
    } else {
```

```
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}

void printFibonacciSeries(int n) {
    cout << "Fibonacci series up to " << n << " terms: ";
    for (int i = 0; i < n; i++) {
        cout << fibonacci(i) << " ";
    }
    cout << endl;
}

int main() {
    int terms;

    cout << "Enter number of terms: ";
    cin >> terms;

    if (terms <= 0) {
        cout << "Please enter a positive number." << endl;
    } else {
        printFibonacciSeries(terms);

        cout << "\nIndividual calculations:" << endl;
        for (int i = 0; i < terms; i++) {
            cout << "fibonacci(" << i << ") = " << fibonacci(i) << endl;
        }
    }

    return 0;
}
```

Conceptual Question

Why are functions considered fundamental to structured programming, and how do they contribute to software engineering principles like modularity and reusability?

Answer: Functions are fundamental because they enable structured programming by breaking complex problems into smaller, manageable pieces (modularity). They promote: 1) **Abstraction** (hide implementation details), 2) **Reusability** (write once, use many times), 3) **Maintainability** (isolate changes to specific functions), 4) **Testing** (test functions independently), 5) **Collaboration** (multiple programmers can work on different functions). In software engineering, functions embody the "divide and conquer" strategy, making large systems comprehensible and manageable. They enforce separation of concerns, reduce code duplication, and facilitate code organization—key principles for building reliable, scalable software systems.

Functions in C++ (Part 2)

- **Void Functions:** Functions that do not return a value
  - Heading starts with `void` keyword
  - Can have parameters or be parameterless
  - Called as standalone statements
  - Can use `return;` (without value) to exit early
- **Parameter Types:**
  - **Value Parameters:** Receive copy of actual parameter (changes don't affect original)
  - **Reference Parameters (`&`):** Receive address/location of actual parameter (changes affect original)
- **Reference Parameters are used when:**

1. Function needs to modify actual parameter
2. Function needs to return multiple values
3. Passing large objects efficiently (avoid copying)

- **Memory Allocation for Parameters:**
  - Value parameters: Copy of value stored in function's data area
  - Reference parameters: Address of actual parameter stored
- **Scope of Identifiers:**
  - **Local identifiers:** Declared within function/block (accessible only within that scope)

- **Global identifiers:** Declared outside all functions (accessible throughout program after declaration)
- **Scope Resolution Operator (`::`):** Accesses global variable when local variable with same name exists
- **Global Variable Issues:** Can cause side effects, reduce modularity, make debugging difficult
- **Storage Classes:**
  - **Automatic variables:** Default for local variables, memory allocated/deallocated with block entry/exit
  - **Static variables:** `static` keyword, memory persists throughout program execution
  - **Static local variables:** Retain value between function calls
- **Example Program:** Using functions to count odds/evens from list of integers
- **Function Categories in Example:**
  - `initialize()`: Sets counters to zero
  - `getNumber()`: Reads a number
  - `classifyNumber()`: Classifies number and increments appropriate counter
  - `printResults()`: Outputs final counts

MCQs

1. What keyword indicates a function returns no value?
   a) `null`
   b) `void`
   c) `none`
   d) `empty`
   Ans. b) `void`
2. How are void functions called?
   a) In expressions
   b) As standalone statements
   c) With return value
   d) Only from main
   Ans. b) As standalone statements
3. Which parameter type allows function to modify original argument?
   a) Value parameter
   b) Reference parameter
   c) Constant parameter
   d) Static parameter
   Ans. b) Reference parameter

4. What is scope of a local variable?
   a) Entire program
   b) Only within function/block
   c) Multiple functions
   d) Global
   Ans. b) Only within function/block
5. Which operator accesses global variable when local has same name?
   a) `:`
   b) `::`
   c) `.`
   d) `->`
   Ans. b) `::`
6. What does `static` do for local variable?
   a) Makes it global
   b) Preserves value between calls
   c) Makes it constant
   d) Speeds up access
   Ans. b) Preserves value between calls
7. When should reference parameters be used?
   a) To prevent modification
   b) To modify arguments
   c) To save memory for small data
   d) For all parameters
   Ans. b) To modify arguments
8. What is problem with global variables?
   a) Too fast
   b) Cause side effects
   c) Limited scope
   d) Can't be modified
   Ans. b) Cause side effects
9. How is reference parameter indicated?
   a) `*` before parameter
   b) `&` after parameter
   c) `&` in parameter type
   d) `ref` keyword
   Ans. c) `&` in parameter type
10. What is automatic variable?
   a) Declared with `auto`
   b) Global variable
   c) Local variable (default)

d) Static variable
Ans. c) Local variable (default)

11. Which is NOT advantage of reference parameters?
    a) Modify arguments
    b) Avoid copying
    c) Return multiple values
    d) Prevent errors
    Ans. d) Prevent errors
12. Where are global variables declared?
    a) Inside `main()`
    b) Inside any function
    c) Outside all functions
    d) In header files
    Ans. c) Outside all functions
13. What does `return;` do in void function?
    a) Returns 0
    b) Returns random value
    c) Exits function early
    d) Causes error
    Ans. c) Exits function early
14. How are value parameters passed?
    a) By reference
    b) By copy
    c) By address
    d) By name
    Ans. b) By copy
15. What persists static variable value?
    a) Between blocks
    b) Between function calls
    c) Between programs
    d) Between files
    Ans. b) Between function calls

Answer Questions

Q1. What are void functions and how are they different from value-returning functions?
A: Void functions do not return a value (return type is `void`), are called as standalone

statements, and can use `return;` without value to exit early. Value-returning functions have a return type, return a value using `return expression;`, and are used in expressions.

Q2. Explain the difference between value parameters and reference parameters.
A: Value parameters receive a copy of the actual argument; changes inside function don't affect original. Reference parameters receive the address/location of actual argument; changes inside function directly modify the original variable. Reference parameters use `&` in declaration.

Q3. When should you use reference parameters instead of value parameters?
A: Use reference parameters when: 1) Function needs to modify the actual argument, 2) Passing large objects/structures (to avoid copying overhead), 3) Function needs to "return" multiple values, 4) Implementing certain operators or algorithms that need direct access.

Q4. What is the scope of an identifier and what are local vs global identifiers?
A: Scope is where in program an identifier is accessible. Local identifiers are declared within a function/block and accessible only there. Global identifiers are declared outside all functions and accessible from their point of declaration onward (can be accessed using `::` if hidden by local variable).

Q5. What are the problems with using global variables?
A: Problems include: 1) Side effects (unintended modifications), 2) Reduced modularity (functions become dependent), 3) Harder debugging (changes can come from anywhere), 4) Namespace pollution, 5) Less reusable code, 6) Can lead to spaghetti code.

Q6. Differentiate between automatic and static variables.
A: Automatic variables (default for locals) have memory allocated at block entry and deallocated at exit; lose value between calls. Static variables (declared with `static`) have persistent memory throughout program execution; retain value between function calls.

Q7. How does the scope resolution operator (`::`) work?
A: The `::` operator accesses the global version of a variable when a local variable with the same name exists. Example: `::globalVar` accesses global variable even if there's a local `globalVar` in current function.

Q8. What is the purpose of `static` for local variables?
A: `static` for local variables: 1) Preserves value between function calls, 2) Memory allocated once (at program start) not each call, 3) Variable initialized only once (first time function is called), 4) Useful for counters, caches, or maintaining state.

Q9. How are parameters stored in memory during function calls?
A: For value parameters: copy of value stored in function's activation record/stack frame. For reference parameters: address/reference to actual argument stored. Local variables (automatic) are also stored in function's activation record.

Q10. Can a void function have a `return` statement?
A: Yes, void functions can have `return;` statement (without value) to exit the function early. This is useful for conditional exits. Without explicit `return;`, function returns implicitly at closing brace.

Q11. What is function signature and why is it important?
A: Function signature consists of function name and parameter list (types and order). It's important for: 1) Overloading resolution, 2) Type checking, 3) Ensuring correct function calls, 4) Separate compilation. Return type is not part of signature for overloading.

Q12. How can functions return multiple values?
A: Functions can return multiple values by: 1) Using reference/pointer parameters (modify arguments), 2) Returning structures/classes (single object containing multiple values), 3) Using tuples (C++11 and later), 4) Using output parameters.

Q13. What is the lifetime of a static local variable?
A: Lifetime of static local variable is the entire program execution. It's initialized once (first time control reaches declaration) and retains its value between function calls. Destroyed when program terminates.

Q14. Why are global variables initialized to default values?
A: Global variables are initialized to default values (0 for numeric types, NULL for pointers, etc.) because they are in static storage duration. This happens before `main()` starts. Local variables (automatic) are not automatically initialized (contain garbage unless explicitly initialized).

Q15. How do you prevent a function from modifying reference parameters?
A: Use `const` with reference parameters: `void func(const Type& param)`. This allows passing by reference (avoiding copy) while preventing modification. Useful for large objects that shouldn't be changed.

Programming Exercise

Functions Part 2 Programming Exercises

Exercise 1: Swap Function with References

```cpp
#include <iostream>
using namespace std;

// Swap using references
void swapByReference(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

// Compare with swap by value (doesn't work)
void swapByValue(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;

    cout << "Before swap: x = " << x << ", y = " << y << endl;

    swapByValue(x, y);
    cout << "After swapByValue: x = " << x << ", y = " << y << endl;

    swapByReference(x, y);
    cout << "After swapByReference: x = " << x << ", y = " << y << endl;

    // Swap back
    swapByReference(x, y);
    cout << "Swapped back: x = " << x << ", y = " << y << endl;

    return 0;
}
```

Exercise 2: Static Variable Counter

```cpp
#include <iostream>
using namespace std;

void functionWithStatic() {
    static int callCount = 0;  // Initialized only once
    callCount++;
    cout << "This function has been called " << callCount << " time(s)" << endl;
}

void functionWithoutStatic() {
    int callCount = 0;  // Initialized every call
    callCount++;
    cout << "Local variable: " << callCount << " (resets each call)" << endl;
}

int main() {
    cout << "Function WITH static variable:" << endl;
    for (int i = 0; i < 5; i++) {
        functionWithStatic();
    }

    cout << "\nFunction WITHOUT static variable:" << endl;
    for (int i = 0; i < 5; i++) {
        functionWithoutStatic();
    }

    return 0;
}
```

Exercise 3: Value vs Reference Parameters Demo

```cpp
#include <iostream>
using namespace std;

// Value parameter - gets copy
```

```
void modifyValue(int x) {
    x = x * 2;
    cout << "Inside modifyValue: x = " << x << endl;
}

// Reference parameter - works on original
void modifyReference(int &x) {
    x = x * 2;
    cout << "Inside modifyReference: x = " << x << endl;
}

// Pointer parameter - also works on original
void modifyPointer(int *x) {
    *x = *x * 2;
    cout << "Inside modifyPointer: *x = " << *x << endl;
}

int main() {
    int num = 5;

    cout << "Original value: " << num << endl << endl;

    cout << "Calling modifyValue(num):" << endl;
    modifyValue(num);
    cout << "After modifyValue: num = " << num << endl << endl;

    cout << "Calling modifyReference(num):" << endl;
    modifyReference(num);
    cout << "After modifyReference: num = " << num << endl << endl;

    cout << "Calling modifyPointer(&num):" << endl;
    modifyPointer(&num);
    cout << "After modifyPointer: num = " << num << endl;

    return 0;
}
```

Exercise 4: Global Variables Problem Demo

```cpp
#include <iostream>
using namespace std;

// Global variables - BAD PRACTICE
int globalCounter = 0;

void incrementGlobal() {
    globalCounter++;
    cout << "Global counter: " << globalCounter << endl;
}

void decrementGlobal() {
    globalCounter--;
    cout << "Global counter: " << globalCounter << endl;
}

// Better approach - pass counter as reference
void incrementLocal(int &counter) {
    counter++;
    cout << "Local counter: " << counter << endl;
}

void decrementLocal(int &counter) {
    counter--;
    cout << "Local counter: " << counter << endl;
}

int main() {
    cout << "=== PROBLEM with Global Variables ===" << endl;
    cout << "Multiple functions modifying same global variable:" << endl;

    incrementGlobal();
    decrementGlobal();
    incrementGlobal();
    incrementGlobal();
    decrementGlobal();

    cout << "\nFinal global counter: " << globalCounter << endl;
```

```
    cout << "\n=== BETTER Approach with Local Variables ===" << endl;
    int localCounter = 0;

    incrementLocal(localCounter);
    decrementLocal(localCounter);
    incrementLocal(localCounter);
    incrementLocal(localCounter);
    decrementLocal(localCounter);

    cout << "\nFinal local counter: " << localCounter << endl;

    // Demonstration of the problem: another counter
    cout << "\n=== Creating second counter ===" << endl;
    int anotherCounter = 10;
    incrementLocal(anotherCounter);
    cout << "First counter still: " << localCounter << endl;
    cout << "Second counter: " << anotherCounter << endl;

    return 0;
}
```

Exercise 5: Return Multiple Values via References

```cpp
#include <iostream>
using namespace std;

// Function that "returns" multiple values through reference parameters
void analyzeArray(int arr[], int size, int &min, int &max, int &sum, double &average)
{
    if (size <= 0) {
        min = max = sum = 0;
        average = 0.0;
        return;
    }

    // Initialize min and max with first element
    min = max = sum = arr[0];
```

```
    // Process rest of array
    for (int i = 1; i < size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
        if (arr[i] > max) {
            max = arr[i];
        }
        sum += arr[i];
    }

    average = static_cast<double>(sum) / size;
}

void getMinMax(int a, int b, int c, int &minimum, int &maximum) {
    minimum = a;
    maximum = a;

    if (b < minimum) minimum = b;
    if (b > maximum) maximum = b;

    if (c < minimum) minimum = c;
    if (c > maximum) maximum = c;
}

int main() {
    // Example 1: Array analysis
    int numbers[] = {5, 2, 8, 1, 9, 3};
    int minVal, maxVal, total;
    double avg;

    analyzeArray(numbers, 6, minVal, maxVal, total, avg);

    cout << "Array Analysis:" << endl;
    cout << "Minimum: " << minVal << endl;
    cout << "Maximum: " << maxVal << endl;
    cout << "Sum: " << total << endl;
    cout << "Average: " << avg << endl;
```

```
    // Example 2: Min/Max of three numbers
    int x = 10, y = 5, z = 15;
    int minimum, maximum;

    getMinMax(x, y, z, minimum, maximum);

    cout << "\nMin/Max of " << x << ", " << y << ", " << z << ":" << endl;
    cout << "Minimum: " << minimum << endl;
    cout << "Maximum: " << maximum << endl;

    return 0;
}
```

Conceptual Question

How do value parameters, reference parameters, and the concept of scope work together to support both safe programming practices and efficient code execution in C++?

Answer**:** Value parameters provide safety by preventing accidental modification of original data (defensive programming), while reference parameters enable efficiency by avoiding expensive copying operations, especially for large objects. The scoping rules (local vs global) further enforce encapsulation and modularity by limiting variable accessibility. Together, they allow programmers to: 1) Use value parameters for simple data where copying is cheap and safety is priority, 2) Use `const` references for large read-only data to get efficiency without risking modification, 3) Use non-`const` references when modification is intended, 4) Limit variable visibility through local scope to prevent unintended interactions, 5) Use global variables sparingly with awareness of their risks. This combination enables writing code that is both efficient (minimizing unnecessary copies) and robust (controlling data access and modification), balancing performance needs with software engineering best practices

Lecture 28: Memory Allocation in C++

- **Memory Allocation in C++:** How memory is allocated and managed
- **C-style memory allocation functions:**
  - `malloc()`: Allocates raw memory block
  - `calloc()`: Allocates and initializes to zero
  - `realloc()`: Resizes allocated memory
  - `free()`: Deallocates memory
- **C++-style memory allocation operators:**
  - `new`: Allocates memory and calls constructors
  - `delete`: Deallocates memory and calls destructors
- **Syntax Examples:**
  - Single element: `int *iptr = new int;` then `delete iptr;`
  - Array: `int *iptr = new int[10];` then `delete [] iptr;`
  - Object: `Date *dptr = new Date;` then `delete dptr;`
  - Array of objects: `Date *dptr = new Date[10];` then `delete [] dptr;`
- **Key Differences from C:**
  - `new/delete` are operators, not functions
  - `new` calls constructors, `delete` calls destructors
  - Type-safe allocation
  - Can be overloaded for classes
- **Memory Allocation Process:**

1. Allocate enough space for new data
2. Populate that space with data
3. Delete previous space when no longer needed
4. Update pointers to point to new space

- **Proper Deallocation:** Critical to prevent memory leaks
  - Use `delete` for single objects

- Use `delete []` for arrays
- **Accessing Object Members:** Use arrow operator `->` for pointers to objects
  - `dptr->setDate();` is equivalent to `(*dptr).setDate();`
- **Dynamic Arrays:** Can be resized using `realloc()` in C or reallocation logic in C++

MCQs

1. Which C function allocates memory and initializes it to zero?

   a) `malloc()`

   b) `calloc()`

   c) `realloc()`

   d) `alloc()`

   Ans. b) `calloc()`
2. What is the correct syntax to allocate an array of 10 integers in C++?

   a) `int *iptr = new int(10);`

   b) `int *iptr = new int[10];`

   c) `int iptr = new int[10];`

   d) `int *iptr = malloc(10);`

   Ans. b) `int *iptr = new int[10];`
3. Which operator calls the constructor during allocation?

   a) `malloc()`

   b) `calloc()`

   c) `new`

   d) `alloc()`

   Ans. c) `new`
4. How do you deallocate an array of objects in C++?

   a) `delete objPtr;`

   b) `delete [] objPtr;`

   c) `free(objPtr);`

d) `dealloc(objPtr);`

Ans. b) `delete [] objPtr;`

5. What does `realloc()` do?

   a) Allocates new memory

   b) Deallocates memory

   c) Changes size of allocated memory

   d) Initializes memory

   Ans. c) Changes size of allocated memory

6. Which is NOT an advantage of `new` over `malloc()`?

   a) Type safety

   b) Calls constructor

   c) Faster execution

   d) Can be overloaded

   Ans. c) Faster execution

7. What is the arrow operator `->` used for?

   a) Pointer arithmetic

   b) Accessing members through pointer

   c) Dereferencing

   d) Memory allocation

   Ans. b) Accessing members through pointer

8. What happens if you use `delete` instead of `delete []` for an array?

   a) Only first element is properly deallocated

   b) All elements are properly deallocated

   c) Compilation error

   d) Runtime error

   Ans. a) Only first element is properly deallocated

9. Which C function is used to resize memory?

   a) `malloc()`

   b) `calloc()`

   c) `realloc()`

d) `size()`

Ans. c) `realloc()`

10. How to allocate memory for a single character in C++?

    a) `char *c = new char;`

    b) `char *c = new char();`

    c) `char *c = new char[1];`

    d) All of the above

    Ans. d) All of the above

11. What does `malloc(10 * sizeof(int))` do?

    a) Allocates 10 bytes

    b) Allocates memory for 10 integers

    c) Allocates and initializes 10 integers

    d) Reallocates memory

    Ans. b) Allocates memory for 10 integers

12. What is a memory leak?

    a) Memory accessed out of bounds

    b) Allocated memory not deallocated

    c) Stack overflow

    d) Pointer errors

    Ans. b) Allocated memory not deallocated

13. Which is the correct sequence?

    a) Allocate → Use → Deallocate

    b) Use → Allocate → Deallocate

    c) Deallocate → Allocate → Use

    d) Allocate → Deallocate → Use

    Ans. a) Allocate → Use → Deallocate

14. How to access member function through object pointer?

    a) `ptr.function()`

    b) `ptr->function()`

    c) `*ptr.function()`

d) `ptr::function()`

Ans. b) `ptr->function()`

15. What does `free()` do?

    a) Allocates memory

    b) Initializes memory

    c) Deallocates memory allocated by `malloc`

    d) Resizes memory

    Ans. c) Deallocates memory allocated by `malloc`

Answer Questions

Q1. Explain the difference between C-style and C++-style memory allocation.

A: C-style uses functions `malloc()`, `calloc()`, `realloc()`, and `free()` which allocate raw memory without calling constructors/destructors. C++-style uses operators `new` and `delete` which are type-safe, call constructors/destructors, and can be overloaded for classes.

Q2. What are the advantages of using `new` instead of `malloc()`?

A: Advantages include: 1) Calls constructors for proper object initialization, 2) Type-safe allocation, 3) Can throw exceptions on failure, 4) Can be overloaded for custom memory management, 5) Automatically calculates size based on type.

Q3. How do you properly deallocate different types of allocated memory?

A: Single object: `delete ptr;`. Array: `delete [] ptr;`. For C allocations: `free(ptr);`. Never mix allocation and deallocation methods (don't use `free()` on `new` allocated memory or `delete` on `malloc()` allocated memory).

Q4. What is the purpose of the arrow operator (`->`)?

A: The arrow operator `->` is used to access members (data or functions) of an object through a pointer. `ptr->member` is equivalent to `(*ptr).member`. It combines dereferencing and member access in one operation.

Q5. Explain what happens during memory allocation with `new`.

A: When `new` is used: 1) Memory is allocated from heap, 2) Constructor is called for objects, 3) Pointer to allocated memory is returned, 4) If allocation fails, `bad_alloc` exception is thrown (or NULL with `nothrow`).

Q6. What is a memory leak and how can it be prevented?

A: A memory leak occurs when dynamically allocated memory is not deallocated, causing program to consume increasing memory. Prevent by: 1) Ensuring every `new` has corresponding `delete`, 2) Using smart pointers, 3) Properly managing ownership, 4) Using RAII (Resource Acquisition Is Initialization).

Q7. How does `realloc()` work?

A: `realloc()` changes size of previously allocated memory block. It may: 1) Expand/shrink existing block if possible, or 2) Allocate new block, copy data, and free old block. Returns pointer to resized memory or NULL if fails.

Q8. What is the difference between `delete` and `delete []`?

A: `delete` is for single objects allocated with `new`, calls destructor once. `delete []` is for arrays allocated with `new []`, calls destructor for each element. Using wrong form causes undefined behavior.

Q9. How do you check if memory allocation succeeded?

A: For `new`: catch `bad_alloc` exception or use `new(nothrow)` which returns NULL on failure. For `malloc()/calloc()`: check if returned pointer is NULL. Always check allocation success.

Q10. What are dangling pointers and how do they occur?

A: Dangling pointers point to memory that has been deallocated. They occur when: 1) Memory is freed but pointer not set to NULL, 2) Pointer points to local variable that went out of scope, 3) Multiple pointers point to same memory and one frees it.

Q11. Explain the memory allocation process for arrays of objects.

A: For `new ClassName[size]`: 1) Memory for `size` objects allocated, 2) Default constructor

called for each object, 3) Pointer to first element returned. For `delete [] ptr`: Destructor called for each object, then memory deallocated.

Q12. What is the `nothrow` version of `new`?

A: `new(nothrow)` returns NULL instead of throwing exception when allocation fails. Syntax: `Type* ptr = new(nothrow) Type;`. Allows C-style error checking: `if(ptr == NULL) { /* handle error */ }`.

Q13. How does dynamic memory differ from static/automatic memory?

A: Dynamic memory (heap) is allocated at runtime with `new/malloc`, size can change, lifetime controlled by programmer. Static/automatic memory (stack/data segment) is allocated at compile time, fixed size, lifetime determined by scope.

Q14. What precautions should be taken with dynamic memory?

A: 1) Check allocation success, 2) Match allocation/deallocation methods, 3) Set pointers to NULL after deletion, 4) Avoid memory leaks, 5) Don't access freed memory, 6) Use RAII/smart pointers when possible.

Q15. How can memory be resized in C++?

A: In C++, there's no direct `realloc()` equivalent. To resize: 1) Allocate new larger block with `new`, 2) Copy data from old to new, 3) Delete old block, 4) Update pointer. Or use containers like `vector` that handle resizing automatically.

Programming Exercise

1. **Dynamic Array Allocation**

```cpp
#include <iostream>
using namespace std;

int main() {
    int size;

    cout << "Enter array size: ";
    cin >> size;
```

```cpp
    // Dynamic allocation
    int* dynamicArray = new int[size];

    // Fill array
    for (int i = 0; i < size; i++) {
        dynamicArray[i] = i * 10;
    }

    // Display array
    cout << "Array elements: ";
    for (int i = 0; i < size; i++) {
        cout << dynamicArray[i] << " ";
    }
    cout << endl;

    // Calculate sum
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += dynamicArray[i];
    }
    cout << "Sum of elements: " << sum << endl;

    // Proper deallocation
    delete [] dynamicArray;

    return 0;
}
```

2. **Object Allocation with new**

```cpp
#include <iostream>
using namespace std;

class Student {
private:
    string name;
    int age;
    double gpa;
public:
```

```cpp
    Student(string n, int a, double g) : name(n), age(a), gpa(g) {}

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "GPA: " << gpa << endl;
    }

    void updateGPA(double newGPA) {
        gpa = newGPA;
    }
};

int main() {
    // Dynamic object allocation
    Student* student1 = new Student("Ali", 20, 3.5);
    Student* student2 = new Student("Sara", 21, 3.8);

    cout << "Student 1:" << endl;
    student1->display();

    cout << "\nStudent 2:" << endl;
    student2->display();

    // Modify through pointer
    student1->updateGPA(3.7);
    cout << "\nAfter update - Student 1:" << endl;
    student1->display();

    // Proper deallocation
    delete student1;
    delete student2;

    return 0;
}
```

3. **Array of Objects**

```cpp
#include <iostream>
using namespace std;
```

```
class Book {
private:
    string title;
    string author;
    double price;
public:
    Book() : title(""), author(""), price(0.0) {}

    void setDetails(string t, string a, double p) {
        title = t;
        author = a;
        price = p;
    }

    void display() {
        cout << title << " by " << author << " - $" << price << endl;
    }

    double getPrice() { return price; }
};

int main() {
    int numBooks;

    cout << "Enter number of books: ";
    cin >> numBooks;
    cin.ignore();  // Clear newline

    // Allocate array of objects
    Book* library = new Book[numBooks];

    // Input book details
    for (int i = 0; i < numBooks; i++) {
        string title, author;
        double price;

        cout << "\nBook " << i+1 << ":" << endl;
        cout << "Enter title: ";
```

```cpp
        getline(cin, title);
        cout << "Enter author: ";
        getline(cin, author);
        cout << "Enter price: ";
        cin >> price;
        cin.ignore();  // Clear newline

        library[i].setDetails(title, author, price);
    }

    // Display all books
    cout << "\nLibrary Collection:" << endl;
    for (int i = 0; i < numBooks; i++) {
        cout << i+1 << ". ";
        library[i].display();
    }

    // Calculate total value
    double total = 0;
    for (int i = 0; i < numBooks; i++) {
        total += library[i].getPrice();
    }
    cout << "\nTotal value of library: $" << total << endl;

    // Proper deallocation
    delete [] library;

    return 0;
}
```

4. **Memory Leak Detection**

```cpp
#include <iostream>
using namespace std;

void createLeak() {
    int* leakyArray = new int[1000];
    // Oops! Forgot to delete - MEMORY LEAK
    // delete [] leakyArray;
}
```

```cpp
void noLeak() {
    int* safeArray = new int[1000];
    // Do something with array
    for (int i = 0; i < 1000; i++) {
        safeArray[i] = i;
    }

    // Proper cleanup
    delete [] safeArray;
    cout << "Memory properly deallocated" << endl;
}

int main() {
    cout << "Memory Management Demo" << endl;
    cout << "======================" << endl;

    cout << "\nCalling leaky function 5 times..." << endl;
    for (int i = 0; i < 5; i++) {
        createLeak();
    }
    cout << "Memory leaked! Check task manager." << endl;

    cout << "\nCalling safe function..." << endl;
    noLeak();

    cout << "\nProgram complete." << endl;
    cout << "Note: In real applications, always deallocate memory!" << endl;

    return 0;
}
```

5. **Dynamic 2D Array**

```cpp
#include <iostream>
using namespace std;

int main() {
    int rows, cols;
```

```
    cout << "Enter number of rows: ";
    cin >> rows;
    cout << "Enter number of columns: ";
    cin >> cols;

    // Allocate 2D array
    int** matrix = new int*[rows];
    for (int i = 0; i < rows; i++) {
        matrix[i] = new int[cols];
    }

    // Fill with values
    int value = 1;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = value++;
        }
    }

    // Display matrix
    cout << "\nMatrix:" << endl;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << matrix[i][j] << "\t";
        }
        cout << endl;
    }

    // Calculate sum of each row
    cout << "\nRow Sums:" << endl;
    for (int i = 0; i < rows; i++) {
        int rowSum = 0;
        for (int j = 0; j < cols; j++) {
            rowSum += matrix[i][j];
        }
        cout << "Row " << i << ": " << rowSum << endl;
    }

    // Proper deallocation
```

```
    for (int i = 0; i < rows; i++) {
        delete [] matrix[i];
    }
    delete [] matrix;

    cout << "\nMemory deallocated successfully!" << endl;

    return 0;
}
```

Conceptual Question

Why is proper memory management critical in C++ programming, and how do modern C++ practices improve upon traditional C-style memory management?
Answer: Proper memory management is critical because: 1) Prevents memory leaks that degrade performance, 2) Avoids dangling pointers causing crashes, 3) Ensures efficient resource use, 4) Prevents security vulnerabilities. Modern C++ improves through:
1) **RAII** (Resource Acquisition Is Initialization) tying resource lifetime to object lifetime,
2) **Smart pointers** (`unique_ptr`, `shared_ptr`) automating deallocation,
3) **Containers** (`vector`, `string`) managing memory automatically, 4) **Move semantics** enabling efficient transfers without copying, 5) **Exception safety** ensuring resources are cleaned up even when exceptions occur. These practices reduce manual memory management errors, leading to safer, more maintainable code while maintaining C++'s performance advantages.

Lecture 29: Friend Functions & Classes

- **Friend Functions:** Non-member functions granted access to private/protected members of a class
- **Declaration:** Inside class with `friend` keyword: `friend returnType functionName(params);`

- **Key Characteristics:**
  - Not member functions (no `this` pointer)
  - Can be declared in private, protected, or public section (location doesn't matter)
  - Called like normal functions (not with object and dot operator)
  - Have access to all members of class that declared them as friend
- **Friend Classes:** Entire class can be friend of another class
  - Syntax: `friend class ClassName;`
  - All member functions of friend class can access private/protected members
- **Limitations of Friendship:**
  - **Not transitive:** Friend of friend is not automatically friend
  - **Not associative:** Doesn't create mutual friendship automatically
  - **Not inherited:** Derived classes don't inherit friendship
- **Encapsulation & Data Hiding:** Friend functions somewhat violate encapsulation but provide controlled access
- **Use Cases for Friend Functions:**

1. Operator overloading (especially `<<`, `>>` operators)
2. Functions that need access to multiple classes' private data
3. Utility functions that logically don't belong as member functions
4. When member functions would have unnatural syntax

- **Example Applications:**
  - Mathematical classes (straight line: `y = mx + c`)
  - Quadratic equation: `y = ax² + bx + c`
  - Classes needing to share private data for calculations
- **Access Control:** Friend functions can access both private and public members
- **Declaration vs Definition:** Friend declaration inside class; function defined outside like normal function

MCQs

1. What is true about friend functions?

   a) They are member functions

   b) They can access private members

   c) Called with object and dot operator

   d) Inheritance applies to them

   Ans. b) They can access private members
2. Where is `friend` keyword placed?

   a) In function definition

   b) In function prototype inside class

   c) In main function

   d) In header only

   Ans. b) In function prototype inside class
3. What does friendship NOT provide?

   a) Access to private members

   b) Transitive relationship

   c) One-way relationship

   d) Can be given to non-members

   Ans. b) Transitive relationship
4. Can a class be friend of another class?

   a) Yes, with `friend class ClassName;`

   b) No, only functions can be friends

   c) Yes, automatically

   d) Only if same namespace

   Ans. a) Yes, with `friend class ClassName;`
5. What is limitation of friendship?

   a) Cannot access public members

   b) Violates encapsulation

   c) Cannot be overloaded

   d) Must be defined inside class

   Ans. b) Violates encapsulation

6. Which shows correct friend function declaration?

   a) `friend void func() {}`

   b) `void friend func();`

   c) `friend void func();`

   d) `void func() friend;`

   Ans. c) `friend void func();`

7. Can friend function access `this` pointer?

   a) Yes

   b) No

   c) Only if static

   d) Only if virtual

   Ans. b) No

8. What is "not transitive" mean for friendship?

   a) A friend of B, B friend of C ⇒ A friend of C

   b) A friend of B, B friend of C ⇏ A friend of C

   c) Friendship goes both ways

   d) Friendship inherits

   Ans. b) A friend of B, B friend of C ⇏ A friend of C

9. Where can friend declaration appear in class?

   a) Only in public section

   b) Only in private section

   c) Any section (public/private/protected)

   d) After class definition

   Ans. c) Any section (public/private/protected)

10. Why use friend functions for operator overloading?

    a) Better performance

    b) Natural syntax (e.g., `cout << obj`)

    c) Required by C++

    d) Easier to write

    Ans. b) Natural syntax (e.g., `cout << obj`)

11. What happens to friendship in derived classes?

    a) Inherited automatically

    b) Not inherited

    c) Inherited if public derivation

    d) Inherited if virtual

    Ans. b) Not inherited

12. Can friend functions be virtual?

    a) Yes

    b) No

    c) Only if member function

    d) Only in base class

    Ans. b) No

13. How are friend functions called?

    a) `obj.friendFunc()`

    b) `friendFunc(obj)`

    c) `Class::friendFunc()`

    d) `friendFunc()` without object

    Ans. b) `friendFunc(obj)`

14. What is "not associative" for friendship?

    a) A friend of B ⇒ B friend of A

    b) A friend of B ⇏ B friend of A

    c) Both ways automatically

    d) Requires mutual declaration

    Ans. b) A friend of B ⇏ B friend of A

15. When should friend functions be used?

    a) Always for efficiency

    b) When member function is unnatural

    c) To avoid encapsulation

    d) For all utility functions

    Ans. b) When member function is unnatural

Answer Questions

Q1. What is a friend function and why would you use one?

A: A friend function is a non-member function granted special access to private and protected members of a class. Used when: 1) Function needs access to private data but doesn't logically belong as member, 2) Operator overloading needs symmetric syntax, 3) Function works with multiple classes' private data.

Q2. Explain the statement "friendship is not transitive" with an example.

A: If Class A declares Class B as friend, and Class B declares Class C as friend, Class C does NOT automatically become friend of Class A. Each friendship must be explicitly declared. Example: Friend relationships don't chain automatically.

Q3. How do you declare a friend function inside a class?

A: Inside class definition, use: `friend returnType functionName(parameterList);`. The declaration can be in any access section (private, protected, or public). The function is defined outside class like normal function.

Q4. What are the advantages and disadvantages of using friend functions?

A: Advantages: 1) Allows natural syntax for some operations, 2) Enables access to multiple classes' private data, 3) Useful for operator overloading. Disadvantages: 1) Violates encapsulation, 2) Increases coupling, 3) Makes maintenance harder, 4) Can lead to less secure code.

Q5. Can a friend function be a member of another class?

A: Yes, a member function of one class can be declared as friend of another class. Syntax: `friend returnType OtherClass::functionName(params);` inside the class granting friendship.

Q6. How does friendship differ from inheritance?

A: Friendship grants access to private members but doesn't create is-a relationship. Inheritance creates is-a relationship and allows access to protected members (not private). Friendship is not inherited; inheritance relationships are.

Q7. What is a friend class and how is it declared?

A: A friend class is a class whose all member functions can access private/protected members of the class granting friendship. Declared as: `friend class ClassName;` inside the class definition.

Q8. Why are friend functions often used with operator overloading?

A: For operators that should be symmetric (like `<<`, `>>`), friend functions allow the operator to be defined outside the class with natural syntax. Example: `cout << obj` rather than `obj.output(cout)`.

Q9. Can constructors or destructors be friend functions?

A: No, constructors and destructors are always member functions of a class. They cannot be declared as friend functions of other classes, though they can be friends to other classes.

Q10. How do you make a function friend to multiple classes?

A: Declare the function as friend in each class that needs to grant access. The function definition remains independent. It can then access private members of all classes that declared it as friend.

Q11. What happens if you change private members of a class with friend functions?

A: Friend functions that depend on private implementation details may break if those details change. This is why friend functions increase coupling and should be used judiciously.

Q12. Can friend functions access protected members?

A: Yes, friend functions can access both private and protected members of the class that declared them as friend, just like member functions of that class.

Q13. Are friend functions in the class's scope?

A: No, friend functions are not in the class's scope. They are ordinary functions with

special access privileges. They cannot be called with class scope resolution (`ClassName::function`).

Q14. How do friend functions affect encapsulation?

A: Friend functions weaken encapsulation by allowing external functions to access implementation details. However, they provide controlled, explicit access rather than completely breaking encapsulation like making everything public.

Q15. When should you avoid using friend functions?

A: Avoid when: 1) Member function would work naturally, 2) The operation logically belongs to the class, 3) You can use public interface instead, 4) It would create tight coupling between classes, 5) Maintenance might be difficult.

Programming Exercise

1. **Basic Friend Function**

```cpp
#include <iostream>
using namespace std;

class SecretBox {
private:
    int secretNumber;
    string secretMessage;
public:
    SecretBox(int num, string msg) : secretNumber(num), secretMessage(msg) {}

    // Friend function declaration
    friend void revealSecret(const SecretBox& box);
};

// Friend function definition
void revealSecret(const SecretBox& box) {
    cout << "=== SECRET REVEALED ===" << endl;
    cout << "Secret Number: " << box.secretNumber << endl;
    cout << "Secret Message: " << box.secretMessage << endl;
```

```cpp
    cout << "======================" << endl;
}

int main() {
    SecretBox box1(42, "The answer to everything");
    SecretBox box2(777, "Lucky number");

    // Call friend function
    revealSecret(box1);
    revealSecret(box2);

    return 0;
}
```

2. **Friend Function for Two Classes**

```cpp
#include <iostream>
using namespace std;

// Forward declaration
class ClassB;

class ClassA {
private:
    int valueA;
public:
    ClassA(int val) : valueA(val) {}

    // Friend function declaration
    friend int sumValues(const ClassA& a, const ClassB& b);
};

class ClassB {
private:
    int valueB;
public:
    ClassB(int val) : valueB(val) {}

    // Same friend function declaration
    friend int sumValues(const ClassA& a, const ClassB& b);
```

```cpp
};

// Friend function accessing private members of both classes
int sumValues(const ClassA& a, const ClassB& b) {
    return a.valueA + b.valueB;
}

int main() {
    ClassA objA(100);
    ClassB objB(200);

    cout << "Value in ClassA: (private - cannot access directly)" << endl;
    cout << "Value in ClassB: (private - cannot access directly)" << endl;
    cout << "\nUsing friend function to calculate sum:" << endl;
    cout << "Sum = " << sumValues(objA, objB) << endl;

    return 0;
}
```

3. **Friend Class**

```cpp
#include <iostream>
using namespace std;

class PrivateData {
private:
    int secretCode;
    double secretValue;
    string secretName;
public:
    PrivateData(int code, double value, string name)
        : secretCode(code), secretValue(value), secretName(name) {}

    // Declare friend class
    friend class DataInspector;
};

class DataInspector {
public:
    void displayData(const PrivateData& data) {
```

```
        cout << "=== INSPECTION REPORT ===" << endl;
        cout << "Code: " << data.secretCode << endl;
        cout << "Value: " << data.secretValue << endl;
        cout << "Name: " << data.secretName << endl;
    }

    void modifyData(PrivateData& data, int newCode, double newValue, string newName)
{
        data.secretCode = newCode;
        data.secretValue = newValue;
        data.secretName = newName;
        cout << "Data modified successfully!" << endl;
    }
};

int main() {
    PrivateData myData(1234, 99.99, "Top Secret");
    DataInspector inspector;

    cout << "Initial data:" << endl;
    inspector.displayData(myData);

    cout << "\nModifying data..." << endl;
    inspector.modifyData(myData, 9999, 88.88, "Modified Secret");

    cout << "\nModified data:" << endl;
    inspector.displayData(myData);

    return 0;
}
```

4. **Friend Function for Operator Overloading**

```cpp
#include <iostream>
using namespace std;

class Complex {
private:
    double real;
    double imag;
```

```
public:
    Complex(double r = 0, double i = 0) : real(r), imag(i) {}

    // Friend function for output
    friend ostream& operator<<(ostream& out, const Complex& c);

    // Friend function for addition
    friend Complex operator+(const Complex& c1, const Complex& c2);

    void display() {
        cout << real << " + " << imag << "i";
    }
};

// Friend function definitions
ostream& operator<<(ostream& out, const Complex& c) {
    out << c.real << " + " << c.imag << "i";
    return out;
}

Complex operator+(const Complex& c1, const Complex& c2) {
    return Complex(c1.real + c2.real, c1.imag + c2.imag);
}

int main() {
    Complex c1(3.5, 2.5);
    Complex c2(1.5, 4.5);

    cout << "Complex Number 1: " << c1 << endl;
    cout << "Complex Number 2: " << c2 << endl;

    Complex sum = c1 + c2;
    cout << "\nSum: " << c1 << " + " << c2 << " = " << sum << endl;

    // Chain output
    cout << "\nAll numbers: " << c1 << ", " << c2 << ", " << sum << endl;

    return 0;
}
```

5. **Mathematical Classes with Friend Functions**

```cpp
#include <iostream>
#include <cmath>
using namespace std;

class Point;  // Forward declaration

class Line {
private:
    double slope;
    double intercept;
public:
    Line(double m, double c) : slope(m), intercept(c) {}

    // Friend function to find intersection
    friend Point findIntersection(const Line& l1, const Line& l2);

    void display() {
        cout << "y = " << slope << "x + " << intercept;
    }
};

class Point {
private:
    double x, y;
public:
    Point(double xVal, double yVal) : x(xVal), y(yVal) {}

    // Friend function declaration
    friend Point findIntersection(const Line& l1, const Line& l2);

    void display() {
        cout << "(" << x << ", " << y << ")";
    }
};

// Friend function accessing both classes
Point findIntersection(const Line& l1, const Line& l2) {
```

```
    if (l1.slope == l2.slope) {
        cout << "Lines are parallel, no intersection" << endl;
        return Point(0, 0);
    }

    double x = (l2.intercept - l1.intercept) / (l1.slope - l2.slope);
    double y = l1.slope * x + l1.intercept;

    return Point(x, y);
}

int main() {
    Line line1(2, 1);   // y = 2x + 1
    Line line2(-1, 5);  // y = -x + 5
    Line line3(2, 3);   // y = 2x + 3

    cout << "Line 1: ";
    line1.display();
    cout << endl;

    cout << "Line 2: ";
    line2.display();
    cout << endl;

    cout << "Line 3: ";
    line3.display();
    cout << endl;

    cout << "\nIntersection of Line 1 and Line 2:" << endl;
    Point intersection = findIntersection(line1, line2);
    cout << "Point: ";
    intersection.display();
    cout << endl;

    cout << "\nIntersection of Line 1 and Line 3:" << endl;
    Point intersection2 = findIntersection(line1, line3);
    if (intersection2.x == 0 && intersection2.y == 0) {
        cout << "(Parallel lines)" << endl;
    }
```

```
    return 0;
}
```

Conceptual Question

How do friend functions balance the need for controlled access to private members with the object-oriented principle of encapsulation, and when is their use justified in good software design?

Answer: Friend functions provide a mechanism for controlled, explicit violation of encapsulation when necessary. They balance by: 1) Requiring explicit declaration inside class (class controls who gets access), 2) Granting access only to specified functions (not open access like public), 3) Maintaining clear intent in code. Their use is justified when: 1) Operator overloading needs symmetric syntax (e.g., `<<`, `>>`), 2) Functions need access to multiple classes' implementations (e.g., bridge functions), 3) Performance-critical code needs direct access, 4) Implementing certain patterns or utilities that cross class boundaries. However, they should be used sparingly as they increase coupling and reduce maintainability. Good design often finds alternatives through better class design, public interfaces, or making functions members when appropriate.

Lecture 30: References in C++

- **References:** Alternative names (aliases) for existing variables
- **Declaration syntax:** `type &refName = varName;` (must be initialized)
- **Key Characteristics:**
  - Must be initialized when declared
  - Cannot be NULL
  - Cannot be reassigned to refer to different variable
  - Doesn't occupy separate memory (same address as original variable)

- **Reference vs Pointer:**
  - References: Safer, cleaner syntax, cannot be NULL, cannot be reassigned
  - Pointers: More flexible, can be NULL, can be reassigned, need dereferencing
- **Reference Parameters:** Used in functions for pass-by-reference
  - Syntax: `void func(int &param)`
  - Allows function to modify original argument
  - Avoids copying overhead for large objects
- **Constant References:** `const type &param`
  - Prevents modification of original
  - Allows passing temporaries/rvalues
  - Efficient for large read-only data
- **References as Return Values:**
  - Functions can return references
  - Allows function call on left side of assignment
  - Must ensure referred object still exists (avoid dangling references)
- **Dangling References:** References to destroyed objects (undefined behavior)
- **Swap Function Example:** Demonstrates reference parameters for swapping values
- **Call by Value vs Call by Reference:**
  - Value: Copy made, original unaffected
  - Reference: Works on original, changes persist
- **References with Large Structures:** More efficient than passing by value
- **Arrow vs Dot Operator:** Use `->` for pointers, `.` for references/objects
- **Reference Initialization:** Must refer to valid object, not literal (except const references)

MCQs

1. How is reference declared?

   a) `int *ref = &var;`

   b) `int &ref = var;`

c) `int ref = &var;`

d) `&int ref = var;`

Ans. b) `int &ref = var;`

2. What is NOT true about references?

   a) Must be initialized

   b) Can be NULL

   c) Cannot be reassigned

   d) Is an alias

   Ans. b) Can be NULL

3. What is output: `int x=5; int &y=x; y=10; cout<<x;`?

   a) 5

   b) 10

   c) Garbage

   d) Error

   Ans. b) 10

4. How to pass by reference in function?

   a) `void func(int x)`

   b) `void func(int *x)`

   c) `void func(int &x)`

   d) `void func(&int x)`

   Ans. c) `void func(int &x)`

5. What is dangling reference?

   a) Reference to constant

   b) Reference to destroyed object

   c) Reference to pointer

   d) Uninitialized reference

   Ans. b) Reference to destroyed object

6. What is purpose of `const` with reference?

   a) Allows modification

   b) Prevents copying but allows modification

c) Prevents modification but allows copying

d) Makes reference constant

Ans. c) Prevents modification but allows copying

7. How to return reference from function?

   a) `int& func()`

   b) `&int func()`

   c) `int func&()`

   d) `func() &int`

   Ans. a) `int& func()`

8. What is difference: `int &a, b;`?

   a) Both are references

   b) `a` is reference, `b` is normal int

   c) Both are normal ints

   d) Syntax error

   Ans. b) `a` is reference, `b` is normal int

9. Can reference refer to literal?

   a) Yes, always

   b) No, never

   c) Only with `const`

   d) Only for integers

   Ans. c) Only with `const`

10. What is safer: reference or pointer?

    a) Reference

    b) Pointer

    c) Both equally

    d) Depends on use

    Ans. a) Reference

11. How to swap two variables using references?

    a) `void swap(int a, int b)`

    b) `void swap(int &a, int &b)`

c) `void swap(int *a, int *b)`

d) Both b and c

Ans. d) Both b and c

12. What happens: `int &r;` (uninitialized)?

    a) Creates null reference

    b) Compilation error

    c) Runtime error

    d) Undefined behavior

    Ans. b) Compilation error

13. Can reference refer to another reference?

    a) Yes

    b) No

    c) Only with pointers

    d) Only in templates

    Ans. b) No

14. What is `int &&`?

    a) Reference to reference

    b) Rvalue reference (C++11)

    c) Syntax error

    d) Double reference

    Ans. b) Rvalue reference (C++11)

15. Why use reference parameters?

    a) To modify arguments

    b) To avoid copying

    c) For efficiency with large data

    d) All of the above

    Ans. d) All of the above

Answer Questions

Q1. What is a reference and how does it differ from a pointer?

A: A reference is an alias (alternative name) for an existing variable. Differences: 1) Reference must be initialized, pointer can be NULL; 2) Reference cannot be reassigned, pointer can; 3) Reference uses cleaner syntax (no `*` dereferencing); 4) Reference is generally safer.

Q2. Explain why references must be initialized.

A: References must be initialized because they are aliases, not separate storage. An uninitialized reference would be an alias to nothing/non-existent memory, which is meaningless and dangerous. The compiler enforces initialization.

Q3. How are reference parameters useful in functions?

A: Reference parameters: 1) Allow functions to modify original arguments, 2) Avoid copying overhead (efficient for large objects), 3) Enable functions to "return" multiple values, 4) Provide cleaner syntax than pointers.

Q4. What are constant references and when should they be used?

A: Constant references (`const T&`) are references that cannot modify the referred object. Used when: 1) Passing large objects efficiently without copying, 2) Preventing accidental modification, 3) Accepting both lvalues and rvalues, 4) Read-only access needed.

Q5. What is a dangling reference and how can it occur?

A: A dangling reference refers to memory that no longer contains the original object. Occurs when: 1) Returning reference to local variable, 2) Reference to dynamically allocated memory after `delete`, 3) Reference to object that went out of scope.

Q6. How can functions return references and why is this useful?

A: Functions return references using `Type& func()`. Useful for: 1) Allowing function calls on left side of assignment (e.g., `arr[i] = 5`), 2) Chaining operations, 3) Avoiding copying of large return values. Must ensure returned reference isn't dangling.

Q7. Compare call by value, call by reference, and call by pointer.

A: Call by value: copies argument, changes don't affect original. Call by reference: works

on original, changes persist, cleaner syntax. Call by pointer: works on original via address, more flexible (can be NULL, reassigned), needs dereferencing.

Q8. Why can't references be NULL?

A: References cannot be NULL because they must alias existing objects. This eliminates null reference errors common with pointers. If you need "no object" semantics, use pointers or optional types.

Q9. What is the syntax for declaring multiple references?

A: Each reference needs its own `&`: `int &a = x, &b = y, &c = z;`. Note: `int &a, b;` declares `a` as reference, `b` as normal int. The `&` binds to the variable name, not the type.

Q10. How do references work with large structures/classes?

A: References to large objects avoid copying entire objects when passing to functions. Use `const Type&` for read-only access or `Type&` for modification. Much more efficient than pass-by-value for large data.

Q11. Can you have a reference to a pointer or pointer to a reference?

A: You can have reference to pointer: `int*& refToPtr = ptr;`. Cannot have pointer to reference because references aren't objects that can have addresses taken (they're aliases, not separate storage).

Q12. What are rvalue references (C++11)?

A: Rvalue references (`Type&&`) bind to temporary objects (rvalues). Enable move semantics: efficient transfer of resources from temporary objects. Used for move constructors, move assignment, perfect forwarding.

Q13. How do you swap two variables using references?

A: `void swap(int &a, int &b) { int temp = a; a = b; b = temp; }`. Call as `swap(x, y)`. Cleaner than pointer version: `swap(&x, &y)` with `void swap(int *a, int *b)`.

Q14. What happens when you take address of a reference?

A: Taking address of reference (`&ref`) gives address of the original object, not the reference itself (references don't have separate addresses). `&ref` is same as `&originalVar`.

Q15. When should you use pointers instead of references?

A: Use pointers when: 1) Need "no object" state (NULL), 2) Need to reassign to point to different objects, 3) Implementing data structures (linked lists, trees), 4) Dynamic memory management, 5) Compatibility with C code.

Programming Exercise

1. **Basic Reference Demonstration**

```cpp
#include <iostream>
using namespace std;

int main() {
    int original = 100;

    // Create reference (alias)
    int& ref = original;

    cout << "Original variable:" << endl;
    cout << "Value: " << original << endl;
    cout << "Address: " << &original << endl;

    cout << "\nReference variable:" << endl;
    cout << "Value: " << ref << endl;
    cout << "Address: " << &ref << endl;

    // Modify through reference
    ref = 200;
    cout << "\nAfter modifying through reference:" << endl;
    cout << "Original value: " << original << endl;
    cout << "Reference value: " << ref << endl;

    // Modify through original
```

```cpp
    original = 300;
    cout << "\nAfter modifying through original:" << endl;
    cout << "Original value: " << original << endl;
    cout << "Reference value: " << ref << endl;

    return 0;
}
```

2. **Swap Function with References**

```cpp
#include <iostream>
using namespace std;

// Using references (clean syntax)
void swapRef(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

// Using pointers (traditional)
void swapPtr(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Using value (doesn't work)
void swapVal(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;

    cout << "Original values: x = " << x << ", y = " << y << endl;
```

```cpp
    swapVal(x, y);
    cout << "After swapVal: x = " << x << ", y = " << y << " (no change!)" << endl;

    swapRef(x, y);
    cout << "After swapRef: x = " << x << ", y = " << y << endl;

    swapPtr(&x, &y);
    cout << "After swapPtr: x = " << x << ", y = " << y << " (swapped back)" << endl;

    return 0;
}
```

3. **References with Large Structures**

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Employee {
    int id;
    string name;
    string department;
    double salary;
    string address[5];  // Large structure
};

// Pass by value (makes copy - inefficient)
void printEmployeeByValue(Employee emp) {
    cout << "Employee ID: " << emp.id << endl;
    cout << "Name: " << emp.name << endl;
    cout << "Department: " << emp.department << endl;
    cout << "Salary: $" << emp.salary << endl;
}

// Pass by reference (no copy - efficient)
void printEmployeeByRef(const Employee& emp) {
    cout << "Employee ID: " << emp.id << endl;
    cout << "Name: " << emp.name << endl;
    cout << "Department: " << emp.department << endl;
```

```
    cout << "Salary: $" << emp.salary << endl;
}

// Pass by reference with modification
void giveRaise(Employee& emp, double percentage) {
    emp.salary += emp.salary * (percentage / 100);
    cout << emp.name << " got a " << percentage << "% raise!" << endl;
}

int main() {
    Employee john = {101, "John Doe", "Engineering", 75000,
                     {"123 Main St", "Apt 4B", "New York", "NY", "10001"}};

    cout << "=== Passing by Value ===" << endl;
    printEmployeeByValue(john);

    cout << "\n=== Passing by Reference ===" << endl;
    printEmployeeByRef(john);

    cout << "\n=== Modifying through Reference ===" << endl;
    cout << "Before raise: $" << john.salary << endl;
    giveRaise(john, 10);
    cout << "After raise: $" << john.salary << endl;

    return 0;
}
```

4. **References as Return Values**

```cpp
#include <iostream>
using namespace std;

int globalCounter = 0;

// Returns reference to global variable
int& getCounter() {
    return globalCounter;
}
```

```cpp
// Returns reference to array element
int& getArrayElement(int arr[], int index) {
    return arr[index];
}

// DANGEROUS: Returns reference to local variable
int& badFunction() {
    int localVar = 999;
    return localVar;  // Dangling reference!
}

int main() {
    // Using reference return to modify global
    cout << "Initial counter: " << globalCounter << endl;

    getCounter() = 100;  // Modify through returned reference
    cout << "After modification: " << globalCounter << endl;

    getCounter() += 50;  // Increment through reference
    cout << "After increment: " << globalCounter << endl;

    // Using reference return with arrays
    int numbers[] = {10, 20, 30, 40, 50};

    cout << "\nOriginal array: ";
    for (int i = 0; i < 5; i++) {
        cout << numbers[i] << " ";
    }
    cout << endl;

    getArrayElement(numbers, 2) = 99;  // Modify array[2]
    cout << "After modifying element 2: ";
    for (int i = 0; i < 5; i++) {
        cout << numbers[i] << " ";
    }
    cout << endl;

    // Chain modifications
    getArrayElement(numbers, 0) = getArrayElement(numbers, 4) * 2;
```

```cpp
    cout << "After chaining: ";
    for (int i = 0; i < 5; i++) {
        cout << numbers[i] << " ";
    }
    cout << endl;

    // WARNING: Dangling reference
    // int& badRef = badFunction();  // DON'T DO THIS!
    // cout << badRef << endl;  // Undefined behavior!

    return 0;
}
```

5. **Constant References**

```cpp
#include <iostream>
#include <string>
using namespace std;

class Book {
private:
    string title;
    string author;
    int pages;
public:
    Book(string t, string a, int p) : title(t), author(a), pages(p) {}

    // Const member function - doesn't modify object
    void display() const {
        cout << title << " by " << author << " (" << pages << " pages)" << endl;
    }

    string getTitle() const { return title; }
    int getPages() const { return pages; }
};

// Accepts const reference - efficient and safe
void printBookInfo(const Book& book) {
    cout << "Book Information:" << endl;
```

```
    book.display();

    // Can call const member functions
    cout << "Title: " << book.getTitle() << endl;
    cout << "Pages: " << book.getPages() << endl;

    // Cannot modify book
    // book.pages = 0;  // Error: book is const
}

// Accepts non-const reference - can modify
void updateBookPages(Book& book, int newPages) {
    cout << "Updating pages from " << book.getPages()
         << " to " << newPages << endl;
    // Would need setter function to actually modify
}

int main() {
    Book myBook("C++ Programming", "John Smith", 500);

    cout << "=== Using Const Reference ===" << endl;
    printBookInfo(myBook);

    cout << "\n=== Temporary Objects with Const Reference ===" << endl;
    // Can pass temporary to const reference
    printBookInfo(Book("Java Basics", "Jane Doe", 350));

    // Cannot pass temporary to non-const reference
    // updateBookPages(Book("Temp", "Author", 100), 200); // Error!

    cout << "\n=== Reference vs Value ===" << endl;
    Book largeBook("Encyclopedia", "Various Authors", 2000);

    // Value parameter - makes copy (inefficient for large objects)
    // printBookInfoByValue(largeBook);  // Would copy 2000+ pages

    // Reference parameter - no copy (efficient)
    printBookInfo(largeBook);  // Just passes reference
```

```
    return 0;
}
```

Conceptual Question

How do references in C++ provide a safer and more intuitive alternative to pointers for many use cases, while still maintaining the efficiency benefits of direct memory access?

Answer: References provide safety by: 1) Requiring initialization (no uninitialized references), 2) Prohibiting NULL values (eliminating null reference errors), 3) Preventing reassignment (clearer code intent). They maintain efficiency by: 1) Being implemented as constant pointers internally (same performance as pointers), 2) Avoiding copying overhead when passing large objects, 3) Enabling direct access to memory without dereferencing syntax. References offer intuitive syntax (looks like value semantics) while providing reference semantics underneath. This balance makes code cleaner (no `*` or `&` operators at call sites) while keeping performance. However, references aren't a complete replacement for pointers—they serve different purposes, with references excelling at parameter passing and return values, while pointers remain essential for dynamic data structures, optional values, and low-level memory manipulation.